



Universidad de Jaén

*Escuela Politécnica Superior de
Jaén*

Simulación predictiva y visualización de escenarios naturales integrando información multisensorial

Estudiante: Alberto Jiménez Expósito

Grado: Ingeniería Informática

**Dirección del TFG: Juan Manuel Jurado Rodríguez, David Jurado Rodríguez
Departamento de Informática**

Fecha: Junio de 2026

Índice de contenidos

1. INTRODUCCIÓN.....	10
1.1 Motivación.....	10
1.2 Contexto del problema y antecedentes.....	11
1.2.1 Evolución y fundamentos de los SDM	12
1.2.2. Tipos de datos y el desafío de la observación.....	12
1.2.3. La transición hacia el Machine Learning	13
1.2.4. Plataformas y sistemas colaborativos	13
1.2.5. Marcos de proyección climática: CMIP6 y escenarios SSP..	13
1.3 Planteamiento del problema	14
1.3.1. Limitaciones de los SIG tradicionales.....	14
1.3.2. Solución al problema	15
1.4 Objetivos	16
1.4.1. Objetivo general	16
1.4.2. Objetivos específicos.....	16
1.5 Metodología de trabajo	18
1.5.1. Evolución y contexto de las metodologías de trabajo	18
1.5.2. Comparación de algunas metodologías de trabajo	18
1.5.3. Descripción de las fases del proyecto	19
1.6 Estructura de la memoria	21
2. ESTADO DEL ARTE	24
2.1 Modelos de distribución de especies (SDMs)	24
2.1.1. Definición y fundamentos teóricos.....	24
2.1.2. Evolución: de la estadística al Machine Learning	24
2.1.3. Interpretación del modelo: curvas de respuesta ecológicas .	24
2.1.4. MaxEnt vs RandomForest: análisis comparativo	25
2.1.4. Otros algoritmos y enfoques relacionados	27
2.2 Herramientas geoespaciales en C++	28
2.2.1. GDAL (Geospatial Data Abstraction Library)	28
2.2.2. OpenCV (Open Source Computer Vision Library)	28
2.2.3. Evaluación de alternativas de Machine Learning en C++	29
2.2.4. Otras librerías de soporte técnico	30
2.3 Tecnologías y visores espaciales para Gemelos Digitales	30
2.3.1. Motores nativos vs tecnologías WebGL	31
2.3.2. Visores globales y análisis comparativo: CesiumJS.....	31
2.3.3. Gemelos Digitales interactivos: Three.js	31

2.4 Visualización web de datos geoespaciales	32
2.4.1. CesiumJS y globos terráqueos virtuales	32
2.4.2. Three.js y renderizado 3D en navegador	33
2.4.3. Aplicaciones web de ciencia ciudadana y divulgación.....	33
3. PLANIFICACIÓN	36
3.1 Marco metodológico y gestión del proyecto	36
3.1.1. Gestión del ciclo de vida y herramientas ALM.....	36
3.1.2. Justificación desde la Ingeniería del Software (IS)	37
3.1.3. Gestión y análisis de riesgos	37
3.2 Identificación de tareas y roles.....	39
3.2.1. Herramienta de seguimiento y control (Kanban)	39
3.2.2. Roles	40
3.2.3. Estructura de desglose de trabajo (EDT)	40
3.3 Planificación temporal	42
3.3.1 Calendario de hitos.....	42
3.3.2 Diagrama de Gantt	43
3.4 Estimación de costes	44
3.4.1 Costes de recursos humanos	44
3.4.2 Costes de hardware	45
3.4.3 Costes de software	47
3.4.4 Costes totales.....	48
4. MATERIALES Y HERRAMIENTAS	50
4.1 Zonas de estudio.....	50
4.1.1 Sierra Mágina/Puerto de la Mata	50
4.1.2 Parque Nacional de Ordesa y Monte Perdido / Pirineos	52
4.1.3 Justificación de las zonas de estudio.....	53
4.2 Fuentes de datos	53
4.2.1 GBIF	53
4.2.2 WorldClim	57
4.2.3 IGN	62
4.3 Entorno de desarrollo	64
4.3.1 Lenguaje y entorno de programación (IDE).....	64
4.3.2 Arquitectura base: motor GEU.....	65
4.3.3 Integración del módulo de simulación	65
4.3.4 Librerías de terceros (GEU).....	65
4.3.5 Plataforma web de visualización: TerraPredict.....	66
5. ANÁLISIS Y DISEÑO	70

5.1 Ingeniería de requisitos	70
5.1.1 Identificación de actores	70
5.1.2 Catálogo de requisitos funcionales (RF)	71
5.1.3 Catálogo de requisitos no funcionales (RNF)	73
5.2 Modelado de objetivos y funcionalidad	74
5.2.1 Modelo de dependencias estratégicas (i-Star / Strategic Dependency Model)	74
5.2.2 Diagrama de casos de uso (UML)	75
5.2.3 Diagrama de actividades (UML)	76
5.3 Arquitectura del sistema.....	78
5.3.1 Diagrama de componentes.....	78
5.3.2 Patrones de diseño.....	79
5.3.3 Diagrama de despliegue (UML).....	80
5.4 Diseño detallado	81
5.4.1 Diseño estructural (Diagrama de clases).....	81
5.4.2 Diseño de comportamiento (Diagrama de secuencia).....	83
5.4.3 Diseño del ciclo de vida (Diagrama de Máquina de Estados).....	84
5.4.4 Diseño de la arquitectura web (Diagrama de componentes TerraPredict)	85
6. IMPLEMENTACIÓN	89
6.1 Incremento I: Subsistema de datos (GeoTIFF/GDAL)	89
6.2 Incremento II: Subsistema de IA (OpenCV – RF/MaxEnt)	95
6.2.1 Algoritmo Random Forest.....	97
6.2.2 Algoritmo MaxEnt (Máxima Entropía).....	104
6.3 Incremento III: Exportación y renderizado en GEU	109
6.4 Incremento IV: Subsistema de interfaz GEU (ImGui).....	112
6.4.1 Arquitectura del adaptador GUI	112
6.4.2 Secciones del panel de control.....	113
6.4.3 Progreso asíncrono y máquina de estados	113
6.4.4 Localización.....	114
6.5 Incremento V: Subsistema frontend web	115
7. RESULTADOS Y EVALUACIÓN	117
7.1 Métricas de rendimiento del modelo	117
7.2 Mapas de predicción: actual y escenarios futuros.....	117
7.3 Visualización en TerraPredict: capturas y análisis	117
7.4 Comparativa de escenarios: áreas de hábitat.....	117
7.5 Rendimiento computacional y tiempos de ejecución.....	117
7.6 Validación visual	117

8. CONCLUSIONES Y LÍNEAS FUTURAS	119
8.1 Cumplimiento de objetivos	119
8.2 Aportaciones del trabajo	119
8.3 Limitaciones del sistema	119
8.4 Líneas futuras de trabajo	119
Referencias bibliográficas	120
Anexos	124
Anexo I – Código fuente GEU	124
Anexo II – Manual de usuario GEU (módulo de predicción de especies).....	151

Índice de ilustraciones

Ilustración 1 : Modelos de distribución de especies, SDM (autor)	12
Ilustración 2 : Gráfica con diferentes escenarios SSPs [8]	14
Ilustración 3 : Interfaz de QGIS [9]	15
Ilustración 4 : Evolución de las metodologías de trabajo (autor)	18
Ilustración 5 : Curva de respuesta (autor).....	25
Ilustración 6 : Comparativa MaxEnt vs RandomForest (autor)	27
Ilustración 7 : Ejemplo de gemelo digital [16]	31
Ilustración 8 : Ejemplo de distribución de especies (SDM) [18]	32
Ilustración 9: Tablero Kanban (autor)	39
Ilustración 10 : Diagrama de Gantt del proyecto (autor)	43
Ilustración 11 : Sierra Mágina [42]	50
Ilustración 12 : Dolina [43]	51
Ilustración 13 : Lapiaces [43]	51
Ilustración 14 : Sima [44]	51
Ilustración 15 : Puerto de la Mata [48]	51
Ilustración 16 : Pinus Nigra [50].....	52
Ilustración 17 : Parque Nacional de Ordesa [52]	52
Ilustración 18 : Monte Perdido [53]	52
Ilustración 19 : Búsqueda de ocurrencias en GBIF(autor)	55
Ilustración 20 : Opciones de descarga GBIF (autor).....	55
Ilustración 21 : Descarga datos presencia GBIF (autor)	56
Ilustración 22 : Formato CSV GBIF (I) (autor)	56
Ilustración 23 : Formato CSV GBIF (II) (autor)	56
Ilustración 24 : Datos históricos WorldClim	58
Ilustración 25 : Descarga de datos futuros en WorldClim (autor)	59
Ilustración 26 : Ángulos de la Tierra (meridianos) [57]	60
Ilustración 27 : Visualización GeoTIFF WorldClim en QGIS (autor)	61
Ilustración 28 : Resolución baja con GeoTIFF WorldClim (autor)	62
Ilustración 29 : Descarga de MDT en IGN (autor)	63
Ilustración 30 : Visualización DEM en QGIS (autor)	63
Ilustración 31 : Diagrama i-Star (autor).....	75
Ilustración 32 : Diagrama de casos de uso (autor)	76
Ilustración 33 : Diagrama de actividad (autor)	78
Ilustración 34 : Diagrama de componentes (autor)	79
Ilustración 35 : Diagrama de despliegue (autor)	81
Ilustración 36 : Diagrama de clases (autor)	83
Ilustración 37 : Diagrama de secuencia: Entrenamiento y evaluación (autor).....	84
Ilustración 38 : Diagrama de máquina de estados (autor)	85
Ilustración 39 : Diagrama de componentes TerraPredict (autor)	87
Ilustración 40 : Estructura del código en GEU (autor).....	90
Ilustración 41 : Flujo completo de predicción (autor)	92
Ilustración 42 : Ejemplo de matriz de Pearson (autor)	94
Ilustración 43 : Flujo de ejecución Random Forest (autor)	98
Ilustración 44 : Ejemplo árbol de decisión Random Forest (autor)	101
Ilustración 45 : Gráfico de importancia de variables en GEU (autor) ...	103
Ilustración 46 : Flujo contextualizado Random Forest (autor).....	104
Ilustración 47 : Flujo de predicción MaxEnt (autor).....	105

Ilustración 48 : Curva de respuesta MaxEnt (autor)	108
Ilustración 49 : Flujo de predicción MaxEnt contextualizado	109
Ilustración 50 : Nube de puntos 3D en GEU (autor)	110
Ilustración 51 : Heatmap 2D (autor).....	110
Ilustración 52 : Mapa de diferencias actual-futuro (autor).....	110
Ilustración 53 : Botones de exports de archivos	112
Ilustración 54 : Archivo de localización de cadenas de texto para idiomas (autor).....	115

Índice de tablas

Tabla 1 : Comparación metodologías de trabajo	19
Tabla 2 : MaxEnt vs RandomForest	27
Tabla 3 : Riesgos del proyecto	38
Tabla 4 : Costes de recursos humanos	45
Tabla 5 : Componentes equipo (PC) personal.....	45
Tabla 6 : Componentes equipo (PC) laboratorio	46
Tabla 7 : Coste total equipo personal	46
Tabla 8 : Coste total equipo laboratorio	47
Tabla 9 : Costes totales de hardware	47
Tabla 10 : Coste de software	48
Tabla 11 : Costes totales del proyecto.....	48
Tabla 12 : Stack tecnológico TerraPredict	67
Tabla 13 : Requisitos funcionales (I).....	71
Tabla 14 : Requisitos funcionales (II).....	72
Tabla 15 : Requisitos funcionales (III).....	73
Tabla 16 : Componentes principales módulo predicción de especies ...	92
Tabla 17 : Hiperparámetros Random Forest	100
Tabla 18 : Hiperparámetros MaxEnt.....	108
Tabla 19 : Archivos exportados de GEU.....	111
Tabla 20 : Secciones interfaz gráfica GEU	113

Índice de códigos fuente

Código 1 : Carga de archivos de ocurrencias CSV de GBIF (autor)....	126
Código 2 : Carga de archivos GeoTIFF multibanda WorldClim (autor)	128
Código 3 : Construcción matriz de entrenamiento (autor)	130
Código 4 : Generación de puntos de fondo/ausencias (autor).....	132
Código 5 : Proceso de thinning espacial sobre ocurrencias (autor).....	132
Código 6 : Proceso de jittering geográfico sobre ocurrencias (autor) ..	133
Código 7 : Selección de variables con Pearson y VIF (autor).....	135
Código 8 : Cálculo del VIF para variables bioclimáticas (autor).....	135
Código 9 : Entrenamiento Random Forest (autor)	137
Código 10 : Predicción Random Forest (autor)	137
Código 11 : Obtención de importancia de variables Random Forest (autor)	138
Código 12 : Entrenamiento MaxEnt (autor)	140
Código 13 : Expansión de características cuadráticas (autor).....	140

Código 14 : Predicción MaxEnt (autor)	140
Código 15 : Obtención de importancia de variables MaxEnt (autor)....	141
Código 16 : Validación cruzada (autor).....	142
Código 17 : Paso de probabilidad a color para heatmap (autor)	143
Código 18 : Generar heatmap (autor).....	145
Código 19 : Generar nube de puntos 3D con colores del heatmap (autor)	
.....	147
Código 20 : Generación textura de diferencia entre escenarios (autor)	
.....	149
Código 21 : Renderización de la interfaz del módulo con ImGui (autor)	
.....	149
Código 22 : Realizar tarea asíncrona para entrenamiento o predicción	
(autor).....	150
Código 23 : Comprobar finalización tarea asíncrona (autor)	151

CAPÍTULO 1 INTRODUCCIÓN

1. INTRODUCCIÓN

1.1 Motivación

Frente a nosotros se nos presenta un grave problema que crea una paradoja tecnológica: ahora mismo tenemos muchísima capacidad a nivel tecnológico e informática para procesar, crear y estudiar datos, sin embargo, existe una amenaza impredecible contra la que quizás no tenemos tantas herramientas para combatir: el cambio climático. Esta situación no es algo lejano, que podamos abandonar, sino que ya está empezando a causar repercusiones en distintos puntos de nuestra sociedad. Desde la industria alimentario, por lo que el cambio climático provoca en sectores agrícolas, hasta la estabilidad de nuestros ecosistemas que proveen de bienes esenciales como el agua, flora y fauna. El impacto es global, independiente y agresivo.

La conservación de la biodiversidad ha dejado de ser una cuestión puramente ética para convertirse en una estrategia de supervivencia. La pérdida de especies animales y vegetales no solo empobrece el patrimonio natural, sino que debilita la resiliencia de nuestro entorno frente a eventos extremos. En este contexto, la gestión reactiva ya no es suficiente; el mundo demanda una capacidad de anticipación que solo la vanguardia tecnológica puede ofrecer.

La conservación de la biodiversidad de la que nos hablan en los colegios o institutos ha dejado de ser una mera cuestión política y ética, convirtiéndose en un punto clave para la supervivencia de nuestra especie y del planeta que habitamos. Además de perder especies animales y vegetales, se debilita nuestra resiliencia frente a eventos extremos. En este punto, la gestión reactiva no será suficiente, por lo que debemos proporcionar una capacidad de anticipación que solo las herramientas tecnológicas pueden ofrecer al mundo.

La revolución digital ha puesto a nuestro alcance herramientas que hasta hace una década parecían imposibles. Tenemos a nuestro alcance muchísima cantidad de datos disponible. Estos datos existen gracias a diferentes tecnologías y herramientas que los proporcionan, como son los satélites, avionetas, drones, etc. Sin embargo, lo verdaderamente importante no es el dato o cómo este es capturado, sino cómo lo interpretamos para llegar a una conclusión. Aquí es donde, la Inteligencia Artificial se convierte en nuestro aliado. Mediante el uso de diferentes algoritmos de Machine Learning y Deep Learning, es posible procesar todos estos datos provenientes de distintas fuentes para encontrar patrones y conseguir resultados precisos. Al integrar esta herramienta con fuentes de datos como WorldClim [1] y GBIF [2], la IA nos permite construir "máquinas del tiempo": modelos de nicho ecológico que simulan cómo reaccionará una especie antes los escenarios futuros del IPCC (CMIP6) [3], [4], mostrando la distribución de la misma a lo largo del tiempo. Esto proporciona una clara herramienta para no sanar, sino prevenir de la herida. Más adelante se hablará de qué son estos escenarios futuros, cómo trabajaremos con ellos y cómo funcionan.

En esta visión de futuro, el concepto de Gemelo Digital (Digital Twin) [3] emerge como la piedra angular para la gestión de ecosistemas complejos. Este no es únicamente un modelo 3D estático, sino que representa una réplica virtual dinámica, interactiva y fiel de un escenario real, alimentado por datos multisensoriales y algoritmos predictivos (en nuestro caso) para reflejar su estado y propiedades actuales, así como anticipar su comportamiento bajo diferentes condiciones.

En el contexto de este Trabajo de Fin de Grado, el uso de un Gemelo Digital permite transformar entornos agrícolas en un entorno virtual donde es posible experimentar con los escenarios climáticos CMIP6 sin riesgo alguno para el entorno físico. Con esta tecnología, se podrá manejar el entorno para realizar pruebas y proporcionar resultados visuales y precisos sobre diferentes zonas de estudio de interés individual para el usuario que utilice la herramienta. Esta capacidad de “predecir el futuro” convierte a los gemelos digitales en una herramienta de decisión sin precedentes, permitiendo una mejor toma de decisiones sobre la gestión del cambio climático y que la preservación de la biodiversidad y la planificación agrícola pasen de ser reactivas (reaccionar frente a las consecuencias) a ser proactivas (prevenir antes de que sea tarde) y basadas en datos, asegurando así la sostenibilidad de nuestros recursos naturales más críticos, entre los que se encuentran las especies vegetales.

En resumen, necesitamos herramientas que nos permitan gestionar y llevar el seguimiento del cómo el cambio climático afecta en distintos campos. La IA podría ser gran parte de la solución, y, bien alimentada, podría servir como vía de escape hacia un ecosistema mucho más verde. A veces, las personas no son capaces de visualizar que podría pasar si seguimos contaminando el planeta. Sin embargo, con tecnologías como la que se va a desarrollar para este Trabajo de Fin de Grado, las personas no especializadas en el mundo tecnológico, podrán ver claramente como podría llegar a afectar el cambio climático en diferentes entornos.

En resumen, necesitamos herramientas que nos permitan gestionar y llevar un seguimiento de cómo el cambio climático afecta en distintos campos. La IA podría ser una gran parte de la solución, y, bien alimentada, podría servir como vía de escape hacia un ecosistema mucho más verde. Con simples palabras, las personas no somos capaces de visualizar que podría pasar si seguimos contaminando, o con simples datos como un porcentaje. Sin embargo, este Trabajo de Fin de Grado busca que cualquier persona pueda ver claramente cómo podría llegar a afectar el cambio climática en diferentes entornos de una manera muy visual, para concienciar de lo que estamos haciendo, de qué puede pasar, pero también de que si ponemos solución ahora, podemos revertir el cambio climático.

1.2 Contexto del problema y antecedentes

El estudio de la distribución de una especie ha pasado de ser una disciplina descriptiva a una ciencia predictiva de alta complejidad computacional y que involucra una cantidad de información, datos y herramientas bastante

grande. En este apartado se detalla la evolución de los modelos y tecnologías base para este proyecto.

1.2.1 Evolución y fundamentos de los SDM

Los Modelos de Distribución de Especies nacieron inicialmente como herramientas para el inventario de recursos y el mapeo de conservación. Sin embargo, su propósito ha evolucionado radicalmente con el aumento de la disponibilidad de datos digital, métodos estadísticos avanzados [4] y el auge de herramientas tecnológicas imprescindibles para representar estas distribuciones (Gemelos Digitales, mapas de distribución, etc.). A nivel conceptual, estos modelos se basan en la teoría del nicho ecológico, que asume que una especie solo puede persistir bajo ciertas condiciones ambientales específicas, es decir, cada especie tiene un rango de valores para ciertas variables climáticas y biológicas (temperatura, humedad, elevación, etc.) donde esta vive de forma cómoda.

Históricamente, los SDM se han utilizado para cuatro aplicaciones principales, estudiar los efectos del cambio climático, identificar áreas protegidas, predecir la expansión de especies invasoras y mapear el riesgo de enfermedades transmitidas por vectores [4]. El proceso estándar implica relacionar localizaciones geográficas (presencias) con capas ambientales (variables bioclimáticas) mediante un algoritmo que genera la superficie de “idoneidad de hábitat”. En la Ilustración 1 se observa un pequeño diagrama representativo del proceso que sigue un SDM descrito anteriormente.

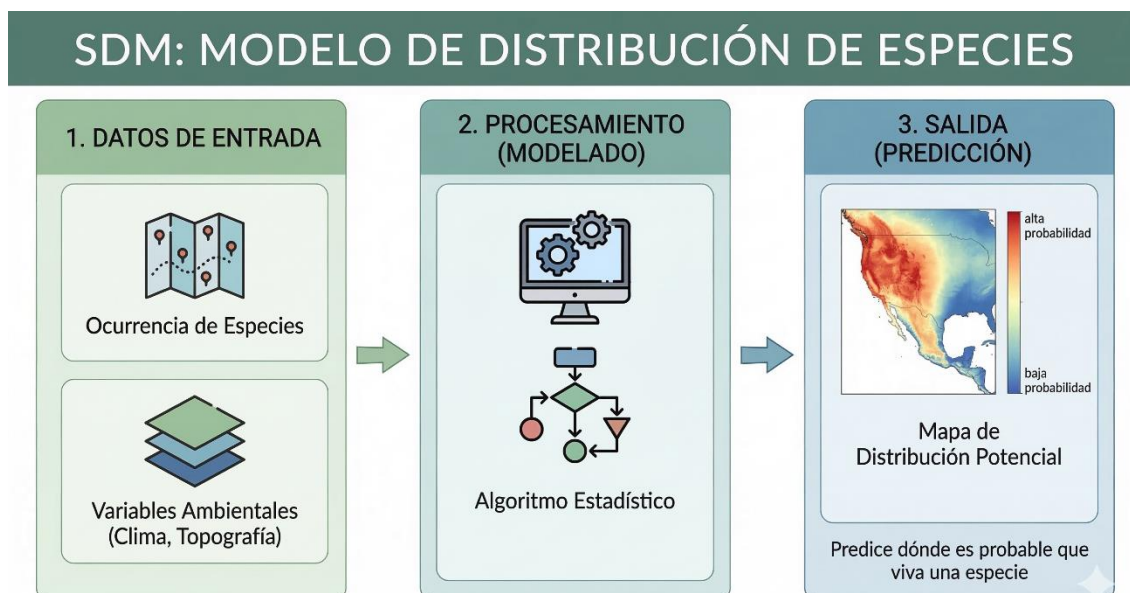


Ilustración 1 : Modelos de distribución de especies, SDM (autor)

1.2.2. Tipos de datos y el desafío de la observación

La calidad de un SDM depende críticamente de los datos de entrada. Tradicionalmente, se distinguen varios tipos de datos biológicos según su origen y estructura:

- **Observaciones puntuales (*Point observations*):** registros GPS de avistamientos.
- **Listas de control (*Checklists*):** registros de especies presentes en un área definida.
- **Encuestas en cuadrículas (*Gridded surveys*):** muestreos sistemáticos que suelen incluir datos de ausencia.
- **Mapas de expertos:** rangos geográficos delimitados por especialistas [5].

Uno de los mayores retos actuales es el manejo de los llamados "datos de solo presencia", donde no se tiene constancia de dónde no está la especie, lo que obliga a los investigadores a utilizar técnicas de generación de "pseudo-ausencias" o "puntos de fondo" (background points) para poder entrenar los modelos [5].

1.2.3. La transición hacia el Machine Learning

Si bien los primeros modelos utilizaban regresiones estadísticas simples (como GLM o GAM), el campo ha experimentado un giro hacia el Machine Learning (ML) debido a su capacidad para modelar relaciones no lineales y complejas entre las variables [5].

Algoritmos como Random Forest, MaxEnt (Máxima Entropía) y Gradient Boosting se han convertido en el estándar de la industria. Estos modelos permiten integrar no solo datos climáticos (temperatura, lluvia), sino también variables dinámicas derivadas de sensores remotos, como índices de vegetación o nubes de puntos de satélite, mejorando significativamente la precisión de las predicciones [6].

1.2.4. Plataformas y sistemas colaborativos

Actualmente, el procesamiento de estos modelos se está desplazando hacia la nube. Herramientas como Google Earth Engine (GEE) han democratizado el acceso a petabytes de datos satelitales y capacidad de computación masiva, permitiendo que análisis que antes tardaban días se realicen en minutos [6].

Además, la tendencia moderna es el desarrollo de sistemas colaborativos como BioModelos, que integran el modelado automatizado con la validación de expertos humanos. Esto demuestra que la tecnología por sí sola no es suficiente; se requiere una interfaz que permita a los expertos interpretar y refinar los resultados obtenidos por los algoritmos [7].

1.2.5. Marcos de proyección climática: CMIP6 y escenarios SSP

Para que los SDM puedan proyectar la distribución futura de las especies, es imprescindible contar con modelos climáticos globales fiables. El estándar internacional actual lo define el IPCC (Intergovernmental Panel on Climate Change) a través del CMIP6 (Coupled Model Intercomparison Project Phase 6). Este proyecto coordina experimentos de modelado climático en todo el mundo para proporcionar proyecciones precisas de temperatura y precipitación.

Una de las innovaciones del CMIP6 respecto a versiones anteriores es la introducción de los SSPs (Shared Socioeconomic Pathways). Estos escenarios representan diferentes trayectorias de desarrollo socioeconómico global y sus correspondientes emisiones de gases de efecto invernadero (Ilustración 2). Van desde el SSP1 (desarrollo sostenible) hasta el SSP5 (desarrollo basado en combustibles fósiles). La integración de estos escenarios en los modelos de distribución de especies permite a los investigadores y técnicos prever cómo se desplazarán los nichos ecológicos bajo diferentes realidades políticas, económicas y climáticas mundiales.

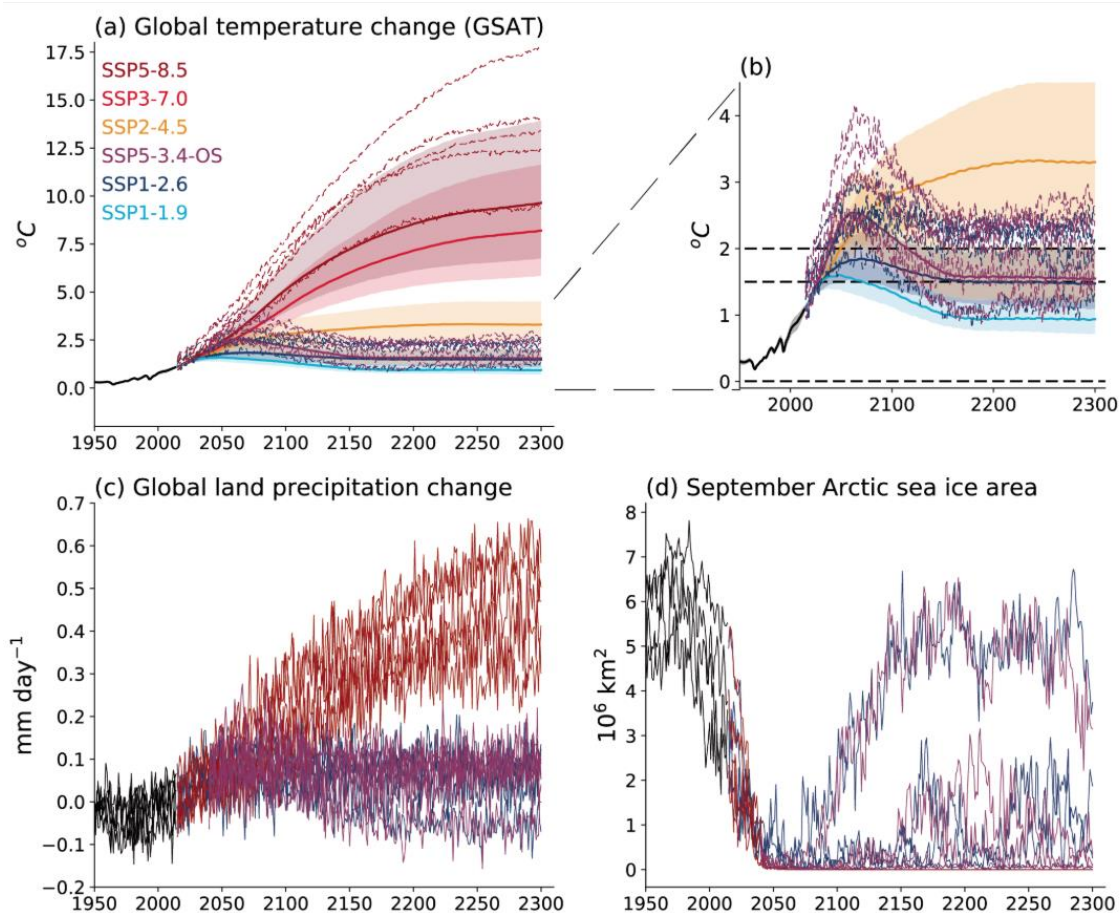


Ilustración 2 : Gráfica con diferentes escenarios SSPs [8]

1.3 Planteamiento del problema

A pesar del avance exponencial en las capacidades de computación y en la disponibilidad de datos abiertos sobre biodiversidad y clima, existe una desconexión técnica y funcional entre las herramientas de modelización científica y los entornos de visualización avanzada. Esta problemática se puede desglosar en los siguientes puntos fundamentales.

1.3.1. Limitaciones de los SIG tradicionales

En la actualidad, la comunidad científica depende mayoritariamente de herramientas SIG en dos dimensiones (2D), como QGIS o ArcGIS, para la

generación y visualización de Modelos de Distribución de Especies (SDM). Aunque estas herramientas son sumamente potentes para el análisis estadístico, presentan limitaciones intrínsecas en la representación de fenómenos complejos en terrenos con orografías abruptas. El analista se ve obligado a interpretar mapas de calor planos que, en ocasiones, no reflejan con fidelidad la realidad física del ecosistema [4].

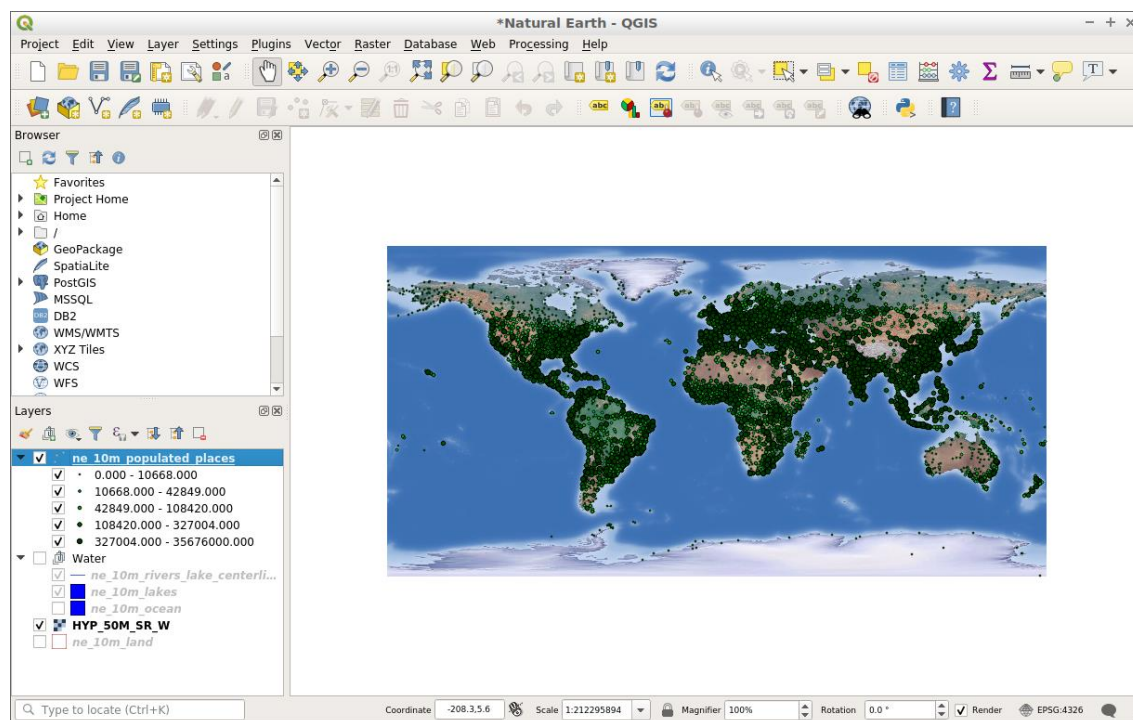


Ilustración 3 : Interfaz de QGIS [9]

Esta abstracción bidimensional genera una barrera en la comunicación de resultados. Como se ha mencionado anteriormente, el impacto del cambio climático es a menudo difícil de cuantificar para el público no experto o incluso para los gestores de políticas ambientales. Un informe basado en tablas de temperatura o mapas 2D estáticos no logra transmitir la urgencia de la situación de la misma manera que una representación inmersiva. Existe una necesidad real de herramientas que permitan proyectar el estado de un terreno en el futuro (por ejemplo, a 100 años vista) de forma que el usuario pueda "ver" la degradación o el desplazamiento de las especies principales. La capacidad de representación de un modelo 3D permite que las consecuencias de fenómenos como el efecto invernadero o el agotamiento de recursos no renovables dejen de ser conceptos teóricos y se conviertan en evidencias visuales impactantes.

1.3.2. Solución al problema

En nuestro caso, utilizaremos un motor de renderizado, visualización y fusión de datos multisensoriales llamado GEU, desarrollado por el GGGJ (Grupo de Gráficos y Geomáticos de Jaén). En la actualidad, GEU está siendo ampliado con nuevas funcionalidades en forma de módulo que se adhieren al core del mismo. Para este Trabajo de Fin de Grado, se pretende crear un nuevo módulo que sea capaz de gestionar los SDM, realizar procesos de Machine Learning mediante librerías correspondientes y visualizar resultados cuantitativos y

cualitativos que permitan al usuario obtener conclusiones verídicas y lo más precisas posibles. Este nuevo módulo será capaz de manejar variables bioclimáticas y otros datos para realizar ese procesamiento sobre la distribución de especies.

Actualmente, si un investigador desea visualizar un escenario futuro basado en el CMIP6 y los escenarios SSPs, debe realizar el procesamiento en software externo (generalmente en lenguajes como R o Python) y, posteriormente, intentar importar los resultados en un visor, perdiendo en el proceso la capacidad de análisis en tiempo real. Falta un "puente" técnico que integre algoritmos de aprendizaje automático (como Random Forest o MaxEnt) dentro de la arquitectura de alto rendimiento de GEU (C++). La ausencia de esta integración impide realizar una monitorización dinámica donde se puedan ajustar variables y observar los cambios en la distribución de la vegetación de forma instantánea sobre el gemelo digital del terreno.

Finalmente, hay que tener en cuenta que estos motores de renderizado y visualización nativos (GEU, QGIS, etc.) llegan a ser complicados para usuarios no especializados en áreas tecnológicas o industriales. Esto complica el acceso para una cantidad importante de usuarios. Es por ello que para la visualización de datos nace la necesidad de la implementación de una herramienta web que facilite el proceso de obtención de resultados, creando así una plataforma que sea mucho más accesible para todos los públicos que quieran estudiar los SDMs. Por tanto, también se desarrollará una plataforma web interactiva como resultado final de este Trabajo de Fin de Grado.

1.4 Objetivos

1.4.1. Objetivo general

El objetivo principal de este Trabajo de Fin de Grado es el diseño, desarrollo e implementación de un pipeline integral compuesta por un nuevo módulo de simulación predictiva integrado en el motor GEU, desarrollado en C++ (con sus librerías correspondientes, como CGAL, GDAL, PCL, etc.) y una plataforma web interactiva para la visualización de resultados obtenidos en GEU de forma que estos se expongan de manera intuitiva a los usuarios finales. En el motor se integrarán algoritmos de Machine Learning aplicados a Modelos de Distribución de Especies (SDM), permitiendo también en el propio motor la visualización de resultados (nubes de puntos, mapas de calor, métricas, etc.) bajo diferentes escenarios climáticos (SSPs).

1.4.2. Objetivos específicos

Para alcanzar el objetivo general propuesto, se han definido los siguientes objetivos específicos, ordenados según la progresión técnica del proyecto:

- **O1 - Investigación y estado del arte:** realizar una revisión bibliográfica profunda sobre los Modelos de Distribución de Especies (SDMs), la aplicación de la Inteligencia Artificial en la conservación de la biodiversidad y las técnicas actuales de visualización ambiental interactiva.

- **O2 - Análisis de la arquitectura GEU:** estudiar el funcionamiento interno y el flujo de renderizado del motor 3D GEU para asegurar una integración nativa y eficiente del nuevo módulo, respetando los estándares de rendimiento del software.
- **O3 – Búsqueda de datos climáticos, biológicos y geográficos de interés:** investigar todos los datos que pueden resultar factibles y necesarios para realizar el entrenamiento de los modelos de nicho ecológico usando los algoritmos pertinentes de Machine Learning.
- **O4 - Integración de librerías geoespaciales:** implementar el soporte para la gestión de datos ráster y vectoriales mediante la librería GDAL, permitiendo la lectura de archivos que contienen variables ambientales y topográficas (elevación, relieve, humedad).
- **O5 - Desarrollo del motor de Inteligencia Artificial:** implementar y adaptar algoritmos de aprendizaje automático (específicamente Random Forest y MaxEnt) en el entorno C++ de GEU, capaces de modelar el nicho ecológico de las especies a partir de los datos multisensoriales y climáticos.
- **O6 - Simulación de escenarios futuros:** integrar las proyecciones climáticas globales del CMIP6 y los escenarios socioeconómicos SSPs, permitiendo al sistema realizar cálculos predictivos sobre la distribución de la vegetación en diferentes horizontes temporales (ej. años 2070 o 2100).
- **O7 - Diseño de interfaz de usuario (GUI):** crear una interfaz de control interactiva dentro del software que permita al usuario gestionar capas de datos, seleccionar especies y configurar los parámetros de las simulaciones de forma intuitiva.
- **O8 - Visualización avanzada y validación:** desarrollar un sistema de renderizado que transforme los resultados numéricos de los modelos en representaciones visuales coherentes sobre el gemelo digital 3D, validando la precisión del sistema mediante pruebas con datos reales del área de estudio.
- **O9 – Estudio e implementación de algoritmos de nicho ecológico basados en Machine Learning orientados a SDM:** desarrollar algoritmos basados en Machine Learning orientados al problema (SDM) que procesen variables bioclimáticas y datos geográficos para su posterior predicción futura.
- **O10 - Desarrollo y despliegue de una plataforma web interactiva para visualización de resultados predictivos:** desarrollar una herramienta web para los usuarios finales que incluya diferentes componentes de visualización de resultados (mapas, modelos 3D, métricas, etc.) usando librerías convenientes para ello (CesiumJS, Three.js, etc.).

1.5 Metodología de trabajo

La gestión de un proyecto de ingeniería de software requiere la selección de un marco de trabajo que garantice la calidad, la trazabilidad y la correcta gestión de los riesgos. Dada la complejidad técnica de integrar un motor 3D, algoritmos de aprendizaje automático, datos climáticos y la implementación de una plataforma web, la elección de la metodología es una decisión crítica de diseño.

1.5.1. Evolución y contexto de las metodologías de trabajo

Históricamente, el desarrollo de software ha evolucionado desde modelos lineales y rígidos, como el modelo en cascada (heredado de la ingeniería civil), hacia enfoques más dinámicos. Mientras que los modelos tradicionales asumen que los requisitos son inmutables desde el inicio, la ingeniería de software moderna ha pivotado hacia modelos iterativos e incrementales y metodologías ágiles (como Scrum o Kanban). Esta evolución responde a la necesidad de gestionar la incertidumbre y permitir que el software evolucione conforme se profundiza en el conocimiento del dominio del problema.

En este TFG, se ha optado por un modelo incremental, que permite construir el sistema a través de pequeñas piezas funcionales, asegurando que el núcleo del software base (GEU) permanezca estable mientras se añaden nuevas capacidades.

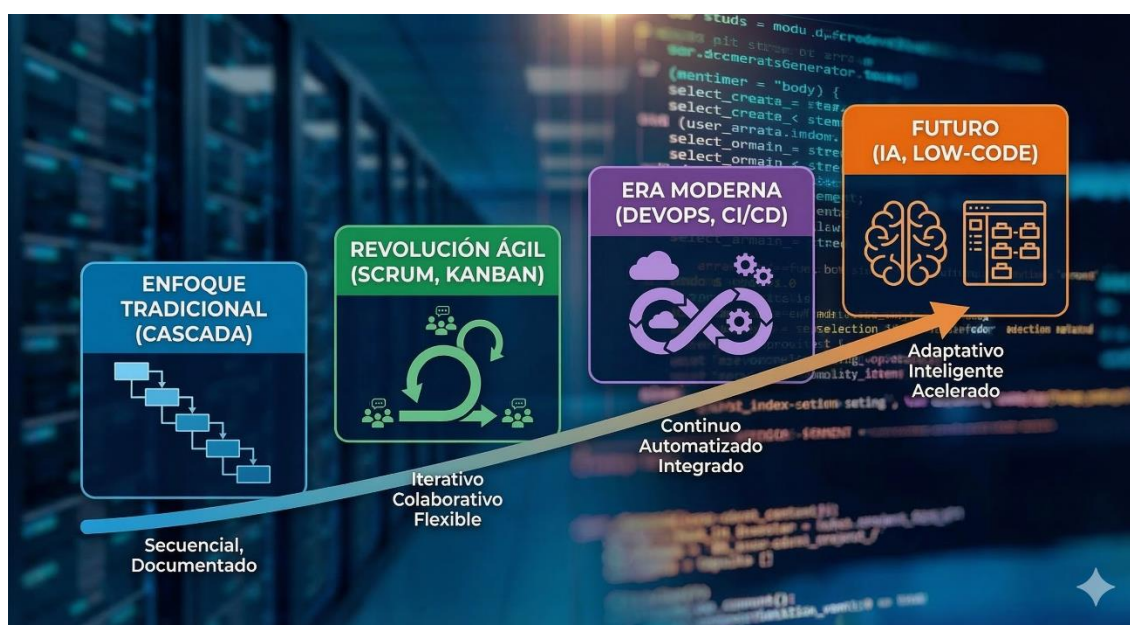


Ilustración 4 : Evolución de las metodologías de trabajo (autor)

1.5.2. Comparación de algunas metodologías de trabajo

Se han evaluado diferentes modelos de Ciclo de Vida del Software (SDLC) antes de la implementación:

Metodologías	Ventajas	Desventajas	Aplicabilidad
Cascada (tradicional)	Estructura clara y fases bien definidas	Muy rígido, los cambios en requisitos provocan un bloqueo enorme en el desarrollo del proyecto	Baja: la integración de datos climáticos puede requerir cambios en el diseño
Espiral	Excelente gestión de riesgos y prototipado	Alta complejidad de gestión y costes temporales	Media: útil para la parte de investigación, pero pesada para el desarrollo
Incremental	Entregas parciales funcionales y gran flexibilidad.	Requiere una arquitectura inicial muy sólida	Alta: permite validar el motor IA antes de desarrollar la GUI

Tabla 1 : Comparación metodologías de trabajo

De las posibles metodologías de desarrollo, se ha seleccionado la incremental por su flexibilidad. Esa metodología nos permite sobre todo reaccionar antes cambios en los requisitos, cuestión clave en la gestión de un proyecto.

1.5.3. Descripción de las fases del proyecto

Siguiendo el enfoque incremental, el desarrollo se ha estructurado en las siguientes fases de ingeniería, garantizando que cada incremento aporte un valor real:

1. Ingeniería de requisitos y análisis

En esta fase se van a definir los requisitos funcionales (ej. procesamiento de archivos, ejecución de algoritmos de nicho ecológico, etc.) y no funcionales (ej. rendimiento en tiempo real, eficiencia en el uso de memoria RAM en C++). Se realizará un análisis de los casos de uso para entender cómo interactuará el biólogo o el técnico forestal con la herramienta.

2. Diseño de la arquitectura e integración

Se diseñará el módulo siguiendo principios de Diseño Orientado a Objetos (POO) para asegurar la mantenibilidad. Se va a prestar especial atención a la interoperabilidad con la arquitectura existente de GEU, diseñando las interfaces

de comunicación necesarias para que el nuevo módulo no afecte al núcleo de renderizado del motor.

3. Implementación por incrementos técnicos

La codificación se ha dividido en incrementos funcionales para facilitar las pruebas:

- i) Subsistema de datos: se implementarán las funcionalidades para tratar datos geoespaciales y otros usando GDAL (GeoTIFF, CSV, etc.).
- ii) Subsistema de Machine Learning: se implementarán la funcionalidades correspondientes a los motores de inferencia, es decir, algoritmos de Random Forest (OpenCV), MaxEnt, entrenamiento, predicción, validación, métricas, etc.
- iii) Exportación y renderizado: se implementarán distintas formas de visualización en el software base (heatmaps 2D, nubes de puntos 3D, etc.) y diferentes métricas de validación del modelo, además de posibles gráficas para un mayor análisis. Finalmente se implementará un sistema de exportación de archivos para su visualización de forma externa e independiente al software utilizado.
- iv) Subsistema de interfaz: se llevará a cabo la implementación de una interfaz siguiendo estándares del software base utilizado y sistemas de internacionalización.
- v) Frontend web: se implementará una herramienta web interactiva donde los usuarios finales podrán ver los resultados de las predicciones ejecutadas en el software.

4. Verificación y pruebas (QA)

Se aplicarán técnicas de ingeniería de software para la validación del sistema, incluyendo pruebas unitarias para los algoritmos de predicción y pruebas de integración para asegurar que la visualización 3D en GEU se mantiene fluida (FPS estables) tras la carga de los nuevos modelos. Además, la GUI y sus propiedades (usabilidad, facilidad de uso, adaptabilidad, etc.) se valorarán con usuarios de prueba. También se probará la interoperabilidad de los datos exportados de GEU para la aplicación web, de forma que sean compatibles entre sí. Además, se testeará la usabilidad y accesibilidad de la web con usuarios voluntarios.

5. Mantenimiento y documentación

Redacción de la memoria técnica y generación de documentación de código, facilitando que el módulo sea extensible en el futuro para nuevas especies o nuevos escenarios.

1.6 Estructura de la memoria

La presente memoria se ha organizado siguiendo una estructura lógica que refleja tanto el proceso de investigación científica como el ciclo de desarrollo de ingeniería de software. A continuación, se resume el contenido de cada uno de los capítulos que componen este trabajo:

- **Capítulo 1: Introducción:** en este capítulo se establece el marco general del trabajo. Se detalla la motivación ligada al cambio climático, se plantea el problema técnico de la falta de herramientas integradas y se definen tanto el objetivo general como los específicos que guiarán el desarrollo.
- **Capítulo 2: Estado del arte:** se realiza una revisión técnica y científica de los conceptos clave. Se analizan los Modelos de Distribución de Especies (SDMs), comparando algoritmos como MaxEnt y Random Forest. Además, se estudian las librerías geoespaciales líderes en el sector (GDAL, OpenCV) y la situación actual de la visualización 3D científica.
- **Capítulo 3: Planificación y costes:** como pilar fundamental de la Ingeniería de Software, este capítulo detalla la metodología de desarrollo incremental seleccionada. Se incluye la planificación temporal mediante un diagrama de Gantt y el presupuesto detallado del proyecto, abarcando recursos hardware, software y costes de personal.
- **Capítulo 4: Materiales y datos:** describe los elementos utilizados para la experimentación. Se detalla la zona de estudio (Sierra Mágina/Puerto de la Mata y Parque Nacional de Ordesa/Monte Perdido), el origen de los datos biológicos (GBIF) y climáticos (WorldClim/CMIP6), así como el entorno tecnológico basado en C++, Visual Studio 2022 y el motor base GEU.
- **Capítulo 5: Análisis y diseño del sistema:** se aborda el proyecto desde una perspectiva puramente de ingeniería. Se definen los requisitos funcionales y no funcionales, se diseña la arquitectura del nuevo módulo y su integración con el núcleo de GEU, y se detalla el diseño lógico del algoritmo de predicción.
- **Capítulo 6: Implementación:** recoge el desarrollo a nivel de código del proyecto propuesto, tanto del motor GEU como de la web interactiva.
- **Capítulo 7: Resultados y evaluación:** aquí se exponen los resultados obtenidos a nivel gráfico (imágenes, terreno 3D) y cuantitativo (métricas, variables utilizadas, etc.)

- **Capítulo 8: Conclusiones y líneas futuras:** se sintetizan los logros alcanzados en relación con los objetivos propuestos, se discuten las aportaciones del trabajo y se proponen mejoras o extensiones para futuras versiones del software.

CAPÍTULO 2

ESTADO DEL ARTE

2. ESTADO DEL ARTE

2.1 Modelos de distribución de especies (SDMs)

2.1.1. Definición y fundamentos teóricos

Los Modelos de Distribución de Especies (SDM), también denominados modelos de nicho ecológico o modelos de idoneidad de hábitat, son herramientas computacionales que relacionan las observaciones de la ocurrencia de una especie con las características ambientales de los lugares donde se encuentra [4]. El objetivo fundamental es caracterizar las condiciones ambientales que permiten la persistencia de una especie y proyectar ese conocimiento sobre un espacio geográfico para identificar áreas de presencia potencial.

Desde una perspectiva teórica, los SDM se basan en el concepto de nicho ecológico de Hutchinson, que define el hipervolumen de n dimensiones de condiciones ambientales (temperatura, precipitación, altitud, etc.) dentro del cual una especie puede mantener una población a largo plazo [5]. En la práctica, estos modelos transforman datos puntuales de presencia en superficies continuas de probabilidad o idoneidad.

2.1.2. Evolución: de la estadística al Machine Learning

Históricamente, los SDM se han basado en métodos estadísticos tradicionales como los Modelos Lineales Generalizados (GLM) o los Modelos Aditivos Generalizados (GAM). Sin embargo, la creciente complejidad de los datos geospaciales y la necesidad de modelar relaciones no lineales han impulsado el uso de algoritmos de Aprendizaje Automático (Machine Learning).

Como señala Beery et al. (2021) [5], el enfoque de Machine Learning permite manejar grandes volúmenes de datos multisensoriales y variables ambientales con altas correlaciones, superando las limitaciones de los métodos estadísticos clásicos en términos de capacidad predictiva y robustez.

2.1.3. Interpretación del modelo: curvas de respuesta ecológicas

Para interpretar el funcionamiento de nuestro modelo y su posterior comprobación correcta, no basta con visualizar resultados como un mapa o una nube de puntos con la distribución potencial tras la predicción realizada, sino que debemos de analizar el nicho ecológico de cada especie, analizando que influencia tiene cada variable sobre la predicción específica para una especie utilizando un modelo de Machine Learning u otro. Para ello se llevan a cabo las conocidas “curvas de respuesta” [10], que tratan de aislar una variable utilizada en el entrenamiento del modelo, manteniendo el resto fijas. Esto nos permite ver la variación de esta variable a lo largo de su espectro para identificar puntos de

comodidad e incomodidad de la especie, es decir, su nicho ecológico. A continuación, en la

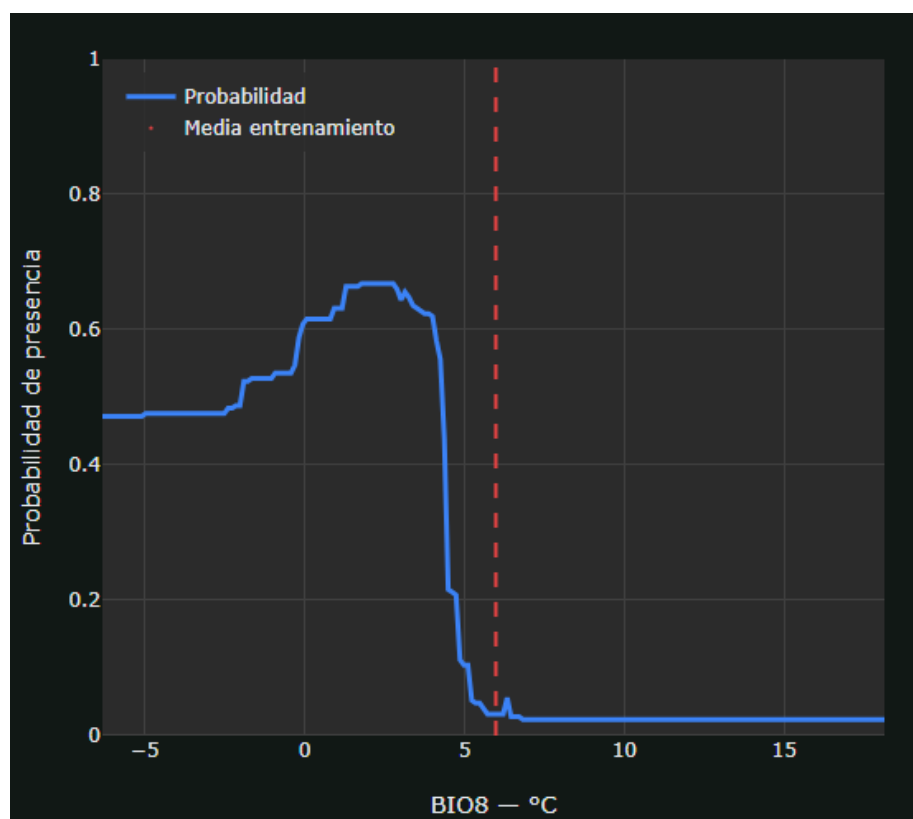


Ilustración 5 : Curva de respuesta (autor)

2.1.4. MaxEnt vs RandomForest: análisis comparativo

En la actualidad, la transición hacia el Machine Learning ha permitido superar las limitaciones de los modelos estadísticos clásicos. A continuación, se detallan los dos algoritmos con mayor impacto en la literatura científica y que forman el núcleo de este estudio.

- **MaxEnt (Maximum Entropy Modeling):** MaxEnt es un método basado en la teoría de la información que estima la distribución de probabilidad más uniforme (máxima entropía), sujeta a un conjunto de restricciones dadas por los datos ambientales observados [7]. Para la integración en este proyecto, no se utiliza el software original dependiente de Java, sino que se ha desarrollado computacionalmente una aproximación nativa en C++ equivalente mediante una Regresión Logística con penalización L2 [13]. Para replicar el comportamiento ecológico característico de MaxEnt, el modelo variables cuadráticas (permitiendo capturar rangos de tolerancia ambiental no lineales) y aplica un balanceo de pesos de clase (Class Weighting). Esta arquitectura implementada permite abordar el problema crítico de los SDM: modelar la probabilidad de idoneidad contrastando un número reducido de presencias reales

frente a una cantidad grande de puntos de fondo (pseudo-ausencias).

- Fundamento técnico: en lugar de intentar modelar la probabilidad de presencia absoluta, el algoritmo aproxima el "nicho ecológico" comparando las condiciones ambientales de las coordenadas con presencia confirmada frente a la distribución ambiental de toda el área de estudio (background points).
- Gestión de datos: es el estándar en ecología porque permite trabajar exclusivamente con datos de solo-presencia (presence-only). Esto elimina el sesgo estadístico que supondría asumir falsas ausencias en zonas que simplemente no han sido muestreadas.
- Regularización: el uso de la penalización L2 actúa como método de regularización para evitar el sobreajuste (overfitting), castigando la dependencia excesiva de variables individuales. Esto permite que el modelo sea lo suficientemente generalizado y robusto como para proyectarse sobre escenarios de cambio climático futuros [5].
- **Random Forest (RF)**: se trata de un método de aprendizaje supervisado de tipo ensamblado (Ensemble Learning) que combina múltiples árboles de decisión para obtener una predicción más precisa y estable [4].
 - Funcionamiento (Bagging): el algoritmo utiliza una técnica llamada Bootstrap Aggregating. Crea múltiples subconjuntos de datos al azar y entrena un árbol de decisión independiente para cada uno. La predicción final es el resultado del promedio (regresión) o votación (clasificación) de todos los árboles.
 - Importancia de variables: una de sus mayores ventajas para la ingeniería es su capacidad intrínseca para evaluar la importancia de cada variable (ej. ¿influye más la temperatura de verano o la altitud?). Esto permite simplificar el modelo eliminando datos redundantes.
 - Robustez: al promediar los resultados de muchos árboles, es extremadamente resistente al ruido en los datos y a los valores atípicos (outliers), algo común cuando se trabaja con datos multisensoriales de UAVs o nubes de puntos LiDAR [5].

Característica	MaxEnt	RandomForest
----------------	--------	--------------

Tipo de Algoritmo	Probabilístico / Entropía	Ensamblado (árboles)
Requisitos de datos	Solo presencia	Presencia / Ausencia
Complejidad de interpretación	Media	Alta
Rendimiento computacional	Alto	Muy alto (paralelizable)

Tabla 2 : MaxEnt vs RandomForest

A continuación, también la Ilustración 6, que es una comparativa más visual entre ambos algoritmos.

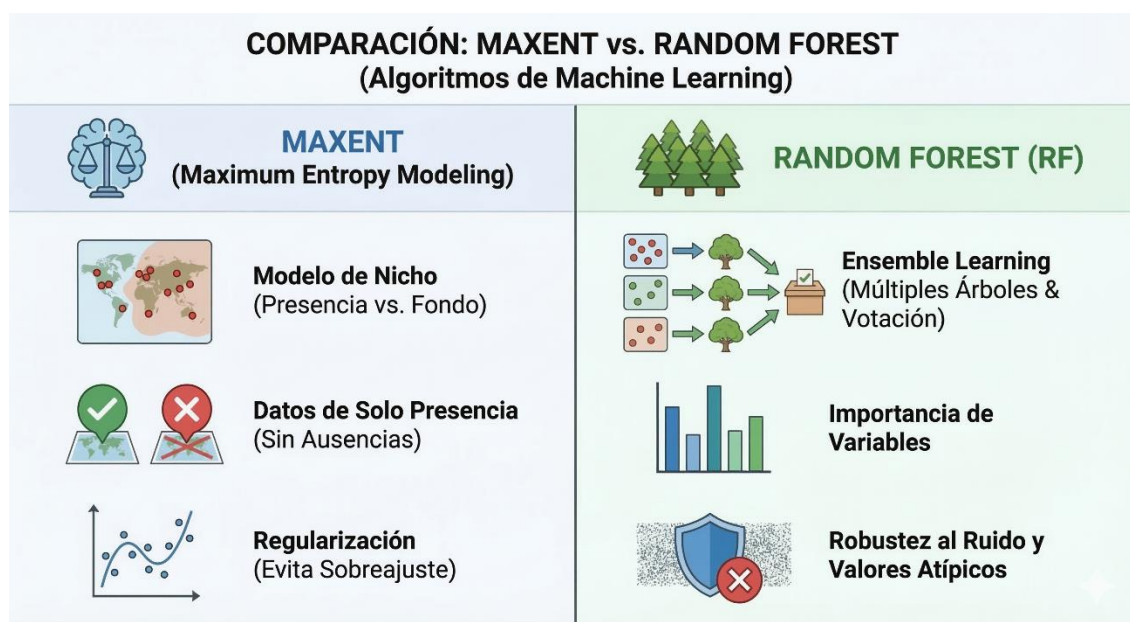


Ilustración 6 : Comparativa MaxEnt vs RandomForest (autor)

2.1.4. Otros algoritmos y enfoques relacionados

Aunque MaxEnt y Random Forest lideran el sector, existen otros enfoques que merece la pena mencionar por su relevancia histórica o emergente:

- **GLM (Generalized Linear Models):** modelos estadísticos clásicos que extienden la regresión lineal. Aunque son fáciles de interpretar, sufren ante relaciones biológicas no lineales complejas.
- **GAM (Generalized Additive Models):** similares a los GLM pero permiten funciones de suavizado, adaptándose mejor a curvas de respuesta biológica reales.
- **BRT (Boosted Regression Trees):** un algoritmo potente que, a diferencia de Random Forest, construye árboles de forma secuencial (cada uno corrige los errores del anterior).

- **SVM (Support Vector Machines):** utilizado para encontrar el hiperplano óptimo que separa las zonas de presencia de las de ausencia en espacios de alta dimensión.
- **Redes Neuronales y Deep Learning:** enfoques emergentes, como las Redes Neuronales Convolucionales (CNN), que se están empezando a aplicar para analizar directamente imágenes de satélite o drones en lugar de variables procesadas [5].

2.2 Herramientas geoespaciales en C++

Para desarrollar software geoespacial de forma eficiente y eficaz, se necesitan diferentes herramientas que permitan una gestión rápida y fiable de la memoria y del procesamiento masivo de datos de distinta índole (ráster, vectorial, etc.). Al trabajar en el entorno GEU, desarrollado en C++, nos apoyaremos en bibliotecas estándar, que garanticen una interoperabilidad fluida y tiempos de respuesta aceptables para la visualización 3D en tiempo real y un procesamiento rápido de las predicciones a realizar.

2.2.1. GDAL (Geospatial Data Abstraction Library)

GDAL es considerada la biblioteca estándar de la industria para la lectura y escritura de formatos de datos geoespaciales [11]. Actúa como una capa de abstracción que permite al desarrollador interactuar con más de 200 formatos de datos (como GeoTIFF, HDF5 o netCDF) mediante una API unificada.

- **Importancia en el proyecto:** en este TFG, GDAL es la pieza clave para la ingesta de las variables bioclimáticas de WorldClim y las proyecciones CMIP6. Dado que estos datos suelen distribuirse en formato GeoTIFF de alta resolución, GDAL permite acceder a los metadatos geográficos (proyección, sistema de coordenadas, dimensiones del píxel) y a los valores numéricos de cada celda de forma eficiente.
- **Capacidades de procesamiento:** además de la lectura, GDAL facilita operaciones críticas como el re-muestreo (resampling), el recorte de capas (clipping) y la reproyección de sistemas de coordenadas, tareas necesarias para alinear los datos a la zona de estudio.

2.2.2. OpenCV (Open Source Computer Vision Library)

Aunque nació orientada a la visión por computador, OpenCV se ha convertido en una herramienta indispensable y en un pilar central de este proyecto debido a su doble funcionalidad: la manipulación de matrices espaciales y su módulo integrado de aprendizaje automático [15].

- **Modelado predictivo (Machine Learning):** a través de su módulo `cv::ml`, OpenCV proporciona una implementación altamente optimizada de Random Forest (RTrees). En este TFG, se utiliza directamente esta librería para entrenar el ensamble de árboles de decisión, procesando la matriz de variables climáticas para generar las superficies de probabilidad espacial, evitando así dependencias externas adicionales.
- **Procesamiento de mapas y exportación visual:** OpenCV resulta fundamental para el posprocesado de los resultados. Se utiliza para aplicar algoritmos de upsampling bicúbico (`cv::resize`) a los mapas de predicción, aplicar rampas de color personalizadas (`colormap`) y exportar los resultados predictivos y las curvas de respuesta a formatos estándar como PNG.

2.2.3. Evaluación de alternativas de Machine Learning en C++

A diferencia del ecosistema Python (dominado por Scikit-Learn), la implementación de Random Forest en C++ requiere librerías que equilibren la facilidad de uso con el rendimiento computacional. Existen diversas opciones de código abierto que permiten integrar modelos de aprendizaje automático de forma nativa:

- **MLPACK:** una biblioteca de aprendizaje automático de C++ intuitiva, rápida y flexible que pone énfasis en la escalabilidad y la pureza del código C++ [12].
- **Dlib / Shark:** otras alternativas potentes que ofrecen implementaciones robustas de árboles de decisión y bosques aleatorios, permitiendo que el entrenamiento y la inferencia se realicen dentro del mismo hilo de ejecución del motor gráfico.
- **Ventaja competitiva:** el uso de estas librerías nativas evita la latencia que supondría llamar a scripts externos de Python o R, permitiendo que la simulación de cambio climático sea dinámica e interactiva dentro de GEU.

A diferencia del ecosistema Python (donde el estándar es Scikit-Learn), la implementación de algoritmos predictivos en C++ requiere evaluar librerías que equilibren la facilidad de uso con el rendimiento computacional. Existen diversas alternativas a OpenCV que estudiaron como propuesta y se exponen a continuación:

- **MLPACK:** una biblioteca de aprendizaje automático de C++ intuitiva, rápida y flexible que pone énfasis en la escalabilidad y la pureza del código C++ [12].
- **Dlib / Shark:** otras alternativas potentes que ofrecen implementaciones robustas de árboles de decisión y bosques

aleatorios, permitiendo que el entrenamiento y la inferencia se realicen dentro del mismo hilo de ejecución del motor gráfico.

Aunque librerías como MLPACK son potentes y robustas, finalmente se optó por descartarlas y centralizar el procesamiento de datos en OpenCV, pues esta es una librería que ya se usa en GEU, por lo que, teniendo en cuenta esto y que OpenCV nos proporciona suficientes herramientas para llevar a cabo el uso del Random Forest en el proyecto, se eligió como propuesta final.

2.2.4. Otras librerías de soporte técnico

Para garantizar la integridad del sistema y la correcta visualización de la información multisensorial, se requiere el apoyo de librerías especializadas en geometría computacional y desarrollo de interfaces:

- **Dear ImGui:** para la implementación de la interfaz de usuario (GUI) interactiva dentro de un motor gráfico basado en C++, ImGui es una de las opciones más eficientes. Al ser una librería de "interfaz inmediata", permite crear paneles de control, gráficas de variables climáticas y selectores de escenarios con un impacto mínimo en el rendimiento de renderizado del motor [13]. Además, se integra con ImPlot, una extensión para la renderización matemática en tiempo real de diferentes gráficas necesarias para mostrar resultados y validación del modelo.
- **PROJ:** aunque GDAL gestiona formatos de archivos, la librería PROJ es el motor subyacente que realiza las transformaciones matemáticas entre diferentes sistemas de coordenadas y proyecciones cartográficas. Su uso es vital para asegurar que los datos locales del UAV (normalmente en coordenadas UTM) se alineen perfectamente con las capas climáticas globales (coordenadas geográficas) [14].
- **Eigen:** se trata de una librería de plantillas de C++ para álgebra lineal. Es fundamental para realizar las operaciones matemáticas de alta velocidad que requieren tanto los algoritmos de Machine Learning como las transformaciones geométricas de los objetos 3D en el entorno virtual.

2.3 Tecnologías y visores espaciales para Gemelos Digitales

La representación visual de datos científicos ha evolucionado de gráficos estáticos a entornos interactivos que permiten una comprensión mucho más profunda como el cambio climático. En esta sección se analiza cómo la tecnología de los Gemelos Digitales [15] y las arquitecturas web modernas se aplican a la visualización de la biodiversidad.



Ilustración 7 : Ejemplo de gemelo digital [16]

2.3.1. Motores nativos vs tecnologías WebGL

Tradicionalmente, la simulación geoespacial con una alta carga computacional requería de motores nativos en lenguajes de bajo nivel (como C++ en GEU) para ser capaz de proporcionar en tiempo real una respuesta al usuario y procesar eficientemente los datos necesarios. Sin embargo, exigir a un usuario la instalación de un motor pesado (aún más si no tiene experiencia en este ámbito) supone una barrera significativa.

Para solucionar esto, se apuesta por la separación en dos capas: un motor de procesamiento nativo (GEU) y un frontend de visualización basado en WebGL (Web Graphics Library [17]). Este es una especificación estándar que permite renderizar gráficos interactivos en 3D y 2D acelerados por hardware, pero directamente en el navegador web, sin necesidad de plugins.

2.3.2. Visores globales y análisis comparativo: CesiumJS

Para la visualización a macroescala, la representación del nicho ecológico requiere un motor capaz de manejar coordenadas esféricas globales (WGS84). CesiumJS se ha posicionado como la biblioteca líder para la creación de globos terráqueos virtuales en 3D.

En nuestro contexto (SDM), CesiumJS permite superponer los mapas de idoneidad (tras ser exportados de GEU) sobre una cartografía base. Esta tecnología proporciona la capacidad de mostrar al usuario las diferentes distribuciones a lo largo del tiempo, comparativas entre escenarios, etc.

2.3.3. Gemelos Digitales interactivos: Three.js

Mientras que CesiumJS es idóneo para el contexto global, Three.js proporciona una exploración y control más fino sobre el ecosistema local, mediante la manipulación de la geometría con shaders.

Mediante Three.js, el Gemelo Digital proporciona al usuario final una herramienta de navegación a través del relieve tridimensional en tiempo real, observando directamente cómo la probabilidad de supervivencia de la especie varía según la orografía del terreno (valles, picos, laderas, etc.).

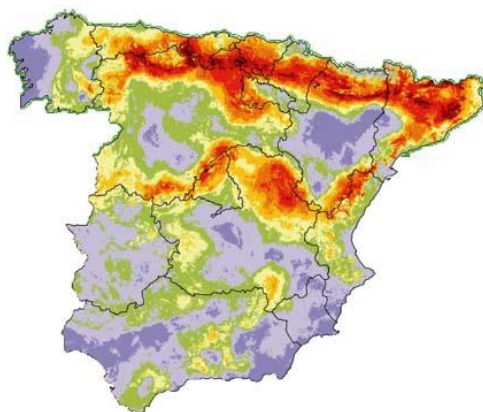


Ilustración 8 : Ejemplo de distribución de especies (SDM) [18]

2.4 Visualización web de datos geoespaciales

El descubrimiento y expansión de datos geoespaciales de alta resolución ha impulsado el desarrollo de tecnologías web que representan información compleja directamente en el navegador, sin necesidad de instalar software externo. Esta capacidad resulta realmente relevante en el ámbito de la divulgación científica y la toma de decisiones ambientales, donde los resultados de modelos predictivos deben ser accesibles para los gestores, investigadores y usuarios generales que quieran realizar algún estudio. A continuación se describen las tecnologías web comunes para este tipo de proyectos.

2.4.1. CesiumJS y globos terráqueos virtuales

CesiumJS es una biblioteca de código abierto basada en WebGL que permite la visualización de datos geoespaciales en un globo terráqueo virtual 3D [19]. Con esta librería se representa la tierra como un elipsoide geodésico, que básicamente garantiza una precisión cartográfica correcta. Entre las características a destacar están:

- **Renderizado de imágenes satelitales y capas ráster:** permite superponer ortofotos, mapas de calor (nuestro caso para los SDMs) y modelos digitales del terreno (MDT).
- **Soporte nativo para GeoJSON, KML y CZML:** posibilita la carga de datos vectoriales con atributos asociados, para representar puntos de predicción y probabilidades.
- **Navegación interactiva:** zoom, rotación, etc.

- **Terrain provider:** integración con Cesium World Terrain o datos propios para representar relieve.

En nuestro contexto, CesiumJS nos proporcionara sus capacidades para mostrar en el visor geoespacial (2D/2.5D) diferentes resultados de la predicción de distribución de las especies.

2.4.2. Three.js y renderizado 3D en navegador

Three.js es una biblioteca JavaScript de alto nivel que abstrae la API WebGL para facilitar la creación de escenas tridimensionales interactivas directamente en el navegador [20]. Mientras que CesiumJS nos puede proporcionar una visión más global en un entorno del globo completo, Three.js nos ayuda a controlar de forma local una escena 3D de la zona de estudio en específico, asilando la orografía de la misma en un Gemelo Digital.

Entre las capacidades de Three.js se encuentran:

- **Geometría personalizada:** generación de mallas a partir de datos DEM.
- **Shaders personalizados:** uso de shaders para aplicar texturas personalizadas con transiciones, atenuación, brillo, etc.
- **Nubes de puntos:** renderizado eficiente de grandes volúmenes de puntos.
- **Controles de cámara:** navegación orbital alrededor del modelo.
- **Post-procesado y tone mapping:** reproducción de color personalizado y controlado.
- **Point picking:** posibilita la funcionalidad de hacer click en un punto y que proporcione toda la información asociada (en nuestro caso: probabilidad, altura, color, etc.).

Con CesiumJS y Three.js se ofrece una experiencia de usuario híbrida y completa, al igual que fácil de usar y público, ya que solo se precisa del navegador para funcionar.

2.4.3. Aplicaciones web de ciencia ciudadana y divulgación

Las plataformas web interactivas han demostrado ser herramientas efectivas para la comunicación de resultados científicos a usuarios no especializados. Herramientas como BioModelos [7] o Google Earth Engine [6] combinan datos abiertos con interfaces accesibles, permitiendo que investigadores, gestores y ciudadanos comunes exploren distintos escenarios sin conocimiento técnicos.

En este Trabajo de Fin de Grado se persigue un objetivo dual:

- **Análisis técnico:** por un lado, se busca proporcionar a los investigadores gráficos de métricas, histogramas y demás, que faciliten la interpretación directa de los modelos entrenados.
- **Divulgación y comunicación:** por otro se pretende ofrece una visualización inmersiva del cambio climático sobre diferentes zonas de estudio para concienciar de la amenaza que este puede llegar a ser (o que ya es, realmente).

CAPÍTULO 3 PLANIFICACIÓN Y COSTES

3. PLANIFICACIÓN

3.1 Marco metodológico y gestión del proyecto

Dada la naturaleza de este Trabajo de Fin de Grado, que integra el procesamiento de grandes volúmenes de datos de distinto carácter (.tif, .json, etc.), algoritmos de aprendizaje automática y renderizado 3D, se exige un marco metodológico que garantice la robustez y escalabilidad del software, así como su capacidad para reaccionar ante cambios en requisitos y propuestas. Desde la perspectiva de la Ingeniería de Software, se ha rechazado el uso de modelos lineales tradicionales (como el modelo en cascada) debido a su pobre adaptabilidad ante cambios.

En su lugar, como se ha comentado anteriormente, se ha adoptado una metodología incremental e iterativa con enfoques de gestión ágil. Este modelo permite descomponer el sistema final en diferentes incrementos funcionales que aporten valor, añadiendo distintas funcionalidades en cada entregable, permitiendo así también una validación continua de la integración de librerías (GDAL, OpenCV, ImGui, CesiumJS, etc.), procesamiento de datos mediante Machine Learning, usabilidad de la aplicación web, etc.

3.1.1. Gestión del ciclo de vida y herramientas ALM

Para asegurar la calidad del producto y la trazabilidad de los requisitos desde su concepción hasta su implementación, se ha hecho uso de herramientas de Gestión del Ciclo de Vida de las Aplicaciones (ALM) [21], [22].

- **Control de versiones y repositorio:** se utiliza Git junto con la plataforma GitHub, aprovechando que el núcleo de GEU se encuentra bajo este ecosistema. Esto permite una gestión eficiente de las ramas de desarrollo (branching), facilitando la experimentación con nuevos algoritmos sin comprometer la estabilidad de la rama principal (main).
- **Gestión de tareas y trazabilidad:** para el seguimiento de las tareas y la gestión del backlog, se ha empleado GitHub Projects. Esta herramienta permite mapear los requisitos funcionales con los "issues" y commits específicos, garantizando que cada funcionalidad implementada responde a una necesidad analizada previamente.
- **Comunicación y feedback con stakeholders:** siguiendo principios ágiles, se han establecido ciclos de revisión quincenales con los directores del proyecto (tutores). Estas reuniones actúan como "Sprint Reviews", donde se valida el incremento desarrollado

y se re-priorizan los requisitos del siguiente ciclo en función de los resultados técnicos obtenidos.

3.1.2. Justificación desde la Ingeniería del Software (IS)

La elección de este marco metodológico responde a tres pilares fundamentales de nuestra rama:

1. **Mitigación de riesgos técnicos:** la integración de código C++ en un motor complejo como GEU conlleva riesgos de gestión de memoria y punteros. Las iteraciones cortas permiten identificar fugas de memoria (memory leaks) o conflictos de librerías de forma temprana.
2. **Evolución de requisitos:** en el ámbito de la simulación climática, el análisis de los datos (CMIP6/SSPs) puede revelar necesidades no previstas inicialmente. Una metodología ágil permite adaptar el diseño del software sin penalizar el cronograma.
3. **Calidad del software (QA):** al realizar pruebas unitarias y de integración al finalizar cada incremento, se asegura que el software es estable y que las nuevas funcionalidades no introducen regresiones en el comportamiento base del motor.

3.1.3. Gestión y análisis de riesgos

En el marco de la Ingeniería de Software, la gestión de riesgos no es una actividad puntual, sino un proceso continuo de identificación y planificación que permite anticiparse a posibles desviaciones del proyecto. Para este trabajo, se ha realizado un análisis cualitativo y cuantitativo siguiendo estándares de gestión (como el PMBOK [23]), categorizando los riesgos en tres áreas críticas: Técnicos (T), de Gestión (G) y de Datos (D).

Para evaluar la severidad de cada riesgo, se utiliza la métrica de exposición al riesgo (E), calculada mediante el producto de la probabilidad (P) de que el riesgo ocurra y el impacto (I) que tendría sobre el proyecto, ambos valorados en una escala del 1 al 5.

ID	Categoría	Riesgo	P	I	E	Estrategia de mitigación (preventiva)	Plan de contingencia (reactiva)
R01	T	Incompatibilidad de APIs (C++, GDAL, GEU)	3	5	15	Estudio previo de versiones y prototipado rápido de enlace de librerías.	Uso de envoltorios (wrappers [24]) o degradación a versiones estables anteriores.
R02	T	Curva de aprendizaje del motor GEU y C++	3	4	12	Formación intensiva inicial y consulta de documentación técnica/wikis internas.	Simplificación de la arquitectura

							del módulo o apoyo en tutores.
R03	D	Calidad insuficiente o ruido en datos UAV/LiDAR	2	5	10	Preprocesado riguroso mediante filtros en OpenCV y PCL.	Recurrir a datasets de respaldo del grupo de investigación (Sierra Mágina).
R04	G	Desviación del cronograma (Diagrama de Gantt)	4	3	12	Aplicación de márgenes de seguridad (buffer) en tareas críticas.	Re-priorización de requisitos mediante técnica MoSCoW [25] (eliminar Could-haves).
R05	T	Exceso de consumo de recursos (RAM, CPU, GPU)	4	3	12	Diseño de algoritmos eficientes y uso de punteros inteligentes en C++.	Implementación de técnicas de optimización (LOD [26]) y reducción de resolución ráster.
R06	D	Inconsistencia entre datos	2	4	8	Normalización de sistemas de coordenadas (PROJ) y re-muestreo.	Ajuste manual de parámetros de escala y alineación cartográfica.

Tabla 3 : Riesgos del proyecto

En la tabla anterior podemos observar el desglose detallado de los factores que podrían comprometer la viabilidad del TFG. El valor de exposición (E) nos permite priorizar los esfuerzos de gestión: aquellos riesgos con una puntuación superior a 10 (como R01, R02, R04 y R05) se consideran críticos o de alta prioridad, exigiendo un seguimiento constante en cada incremento de software.

Para cada elemento, se han definido dos líneas de defensa:

- **Estrategia de mitigación:** representa las acciones preventivas que se han tomado desde el inicio del proyecto para reducir la probabilidad de que el riesgo se materialice. Un ejemplo claro es el uso de prototipos rápidos para asegurar la compatibilidad de librerías (R01).
- **Plan de contingencia:** establece la respuesta reactiva en caso de que el riesgo llegue a ocurrir. Esto garantiza que el proyecto no se detenga ante imprevistos, disponiendo de soluciones alternativas como la re-priorización de funcionalidades mediante la técnica MoSCoW o el uso de datos de respaldo.

Este enfoque proactivo asegura que la planificación temporal y los costes estimados se mantengan dentro de los márgenes aceptables, minimizando el

impacto de la incertidumbre técnica inherente al desarrollo con motores gráficos y algoritmos de Inteligencia Artificial.

3.2 Identificación de tareas y roles

Dado que este proyecto abarca el ciclo completo de vida del software, desde la concepción hasta el despliegue, es necesario definir una estructura organizativa clara. Aunque la ejecución recae sobre un único ingeniero, se han identificado distintos roles profesionales que se asumen en las diferentes fases del proyecto. Esta distinción es fundamental para la posterior estimación económica, ya que cada perfil tiene un coste de mercado diferente.

3.2.1. Herramienta de seguimiento y control (Kanban)

Para la gestión operativa del proyecto y el seguimiento de los Sprints, se seguirá la filosofía Kanban. Dentro de esta, existe una herramienta conocida como tableros, que definen las tareas y la estructuran, dividiéndolas en diferentes categorías, definidas por el ciclo de vida del proyecto. Se utilizará dicho tablero a lo largo del proyecto para gestionar las tareas a realizar, definir nuevas. De esta forma, se podrán organizar las tareas de una forma fácil, eficiente y eficaz.

Como se observa en la Ilustración 9, se observan las distintas categorías (típicas de la metodología Kanban) en las que se repartirán las tareas definidas anteriormente:

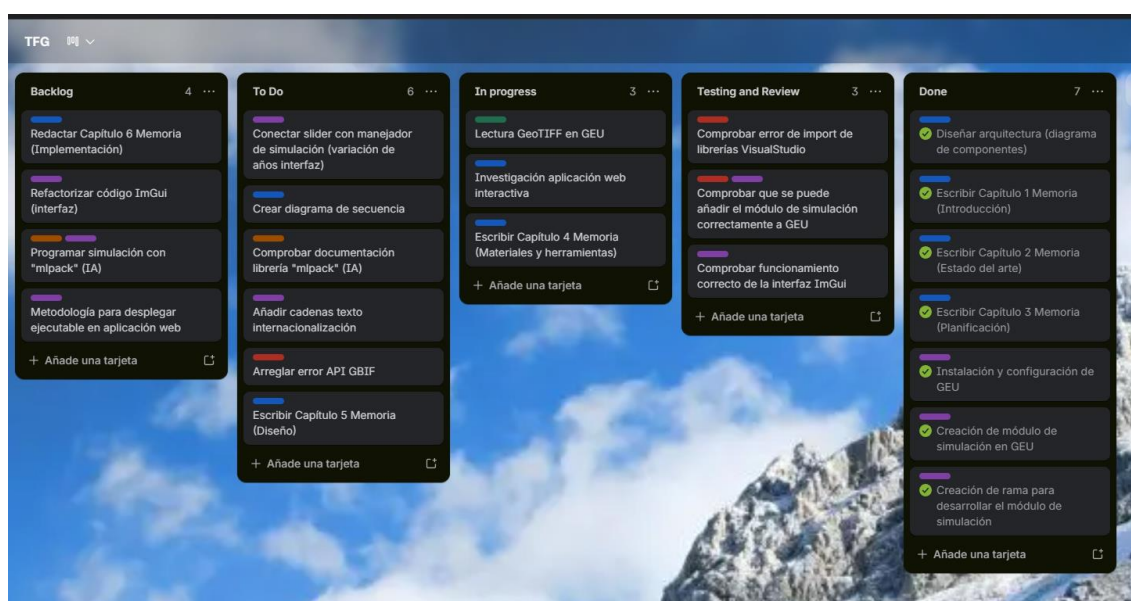


Ilustración 9: Tablero Kanban (autor)

3.2.2. Roles

Se han definido cuatro perfiles técnicos que cubrirán las necesidades del desarrollo:

- **Jefe de proyecto (Project Manager - PM):** responsable de la planificación temporal, la gestión de riesgos (analizados en el punto 3.1.3) y la comunicación con los directores del TFG (Stakeholders). Su objetivo es asegurar que se cumplen los hitos del diagrama de Gantt.
- **Analista de software:** encargado de la ingeniería de requisitos. Su labor incluye la especificación de requisitos funcionales y no funcionales, así como el modelado del sistema (diagramas UML, i-Star) antes de la fase de codificación.
- **Ingeniero de desarrollo (Full-Stack Developer):** el rol más extenso en tiempo. Se encarga de la implementación técnica en C++, la integración de las librerías (GDAL, OpenCV, ImGui, etc.), la escritura de código para el procesamiento de los datos correspondientes, exportación y visualización de resultados e implementación de la herramienta web interactiva (frontend) con las librerías correspondientes y componentes necesarios (shaders, CesiumJS, Three.js).
- **Ingeniero de calidad (QA / Tester):** responsable del diseño y ejecución del plan de pruebas (unitarias y de integración) para validar que la simulación climática es correcta y que el rendimiento (FPS) es óptimo.

3.2.3. Estructura de desglose de trabajo (EDT)

Para garantizar una gestión granular del proyecto, se han descompuesto las actividades en una Estructura de Desglose de Trabajo (WBS - Work Breakdown Structure). Las tareas se han agrupado en Paquetes de Trabajo (PT) lógicos, evitando una lista plana y desordenada.

A continuación, se detallan las tareas específicas para cada paquete:

PT1: Gestión y planificación (Rol: Jefe de proyecto)

- T1.1: Definición del alcance y objetivos del TFG.
- T1.2: Elaboración del cronograma y diagrama de Gantt inicial.
- T1.3: Análisis de riesgos y planes de contingencia.

- T1.4: Reuniones de seguimiento quincenal con tutores (Sprints Reviews).

PT2: Investigación y estado del arte (Rol: Analista)

- T2.1: Revisión bibliográfica sobre Modelos de Distribución de Especies (SDMs).
- T2.2: Estudio comparativo de algoritmos: MaxEnt vs Random Forest.
- T2.3: Análisis de librerías C++ para GIS (GDAL) y Machine Learning (OpenCV), así como tecnologías WebGL.
- T2.4: Estudio de la arquitectura interna del motor GEU.

PT3: Análisis y diseño (Rol: Analista)

- T3.1: Ingeniería de requisitos (funcionales y no funcionales).
- T3.2: Modelado de actores y dependencias (Diagrama i-Star).
- T3.3: Diseño de la arquitectura del módulo (Diagramas de Clases y Secuencia).
- T3.4: Diseño de la interfaz de usuario (mockups para ImGui).

PT4: Implementación y desarrollo (Rol: Desarrollador)

- T4.1: Subsistema de datos: implementación de la carga de GeoTIFFs (WorldClim/CMIP6) y normalización de coordenadas con GDAL/PROJ.
- T4.2: Subsistema de IA: integración y entrenamiento del algoritmo Random Forest en C++.
- T4.3: Subsistema de visualización: desarrollo de shaders y lógica de renderizado en GEU para representar los mapas de calor y la vegetación.
- T4.4: Subsistema de interfaz: programación de la GUI con Dear ImGui (selectores de escenario, años, variables).
- T4.5: Subsistema frontend web: desarrollo de la plataforma TerraPredict (Vite/VanillaJS), integración del visor global (CesiumJS), gemelo digital local (Three.js) y gráficos analíticos (Plotly).

PT5: Pruebas y validación (Rol: QA)

- T5.1: Diseño de casos de prueba.
- T5.2: Ejecución de pruebas unitarias (algoritmo de predicción).
- T5.3: Pruebas de integración interoperabilidad de archivos (GeoJSON/PLY) y rendimiento visual en el entorno web.
- T5.4: Validación visual de escenarios con datos históricos.

PT6: Documentación y defensa (Rol: Analista/PM)

- T6.1: Redacción de la memoria técnica.
- T6.2: Elaboración del manual de usuario y documentación de código (Doxygen).
- T6.3: Preparación de la presentación y materiales de defensa.

3.3 Planificación temporal

Partiendo de la Estructura de Desglose de Trabajo (EDT) definida en el apartado anterior, se ha elaborado una planificación temporal detallada para cubrir el ciclo de desarrollo estimado de 4 meses (equivalente a la carga lectiva de 12 créditos ECTS del TFG, unas 300 horas de dedicación).

3.3.1 Calendario de hitos

Antes de detallar el cronograma diario, se han establecido los siguientes Hitos de Control (Milestones). Estos eventos no tienen duración, sino que marcan la finalización de fases críticas y sirven como puntos de revisión con los directores del proyecto:

- Hito 1 (M1): aprobación de requisitos y diseño de la arquitectura en dos capas (GEU/Frontend)
- Hito 2 (M2): prototipo funcional de los algoritmos de IA en C++ (GEU) y validación del sistema de exportación de datos espaciales
- Hito 3 (M3): despliegue de la plataforma web interactiva (TerraPredict) e integración visual del Gemelo Digital 3D
- Hito 4 (M4): validación integral del sistema, pruebas de usuario (usabilidad) y entrega de la memoria final

3.3.2 Diagrama de Gantt

Para la gestión del cronograma se ha tenido en cuenta la dependencia lógica entre tareas (por ejemplo, no se puede entrenar el modelo sin tener los datos normalizados). En la siguiente figura se muestra el Diagrama de Gantt resultante, donde se aprecia el camino crítico del proyecto.

Para la gestión del cronograma se ha tenido en cuenta la dependencia lógica entre tarea (por ejemplo, no se puede desplegar el visor 3D web sin tener los datos climáticos previamente procesados y exportados por GEU). En la siguiente Ilustración 10 se muestra el Diagrama de Gantt resultante.

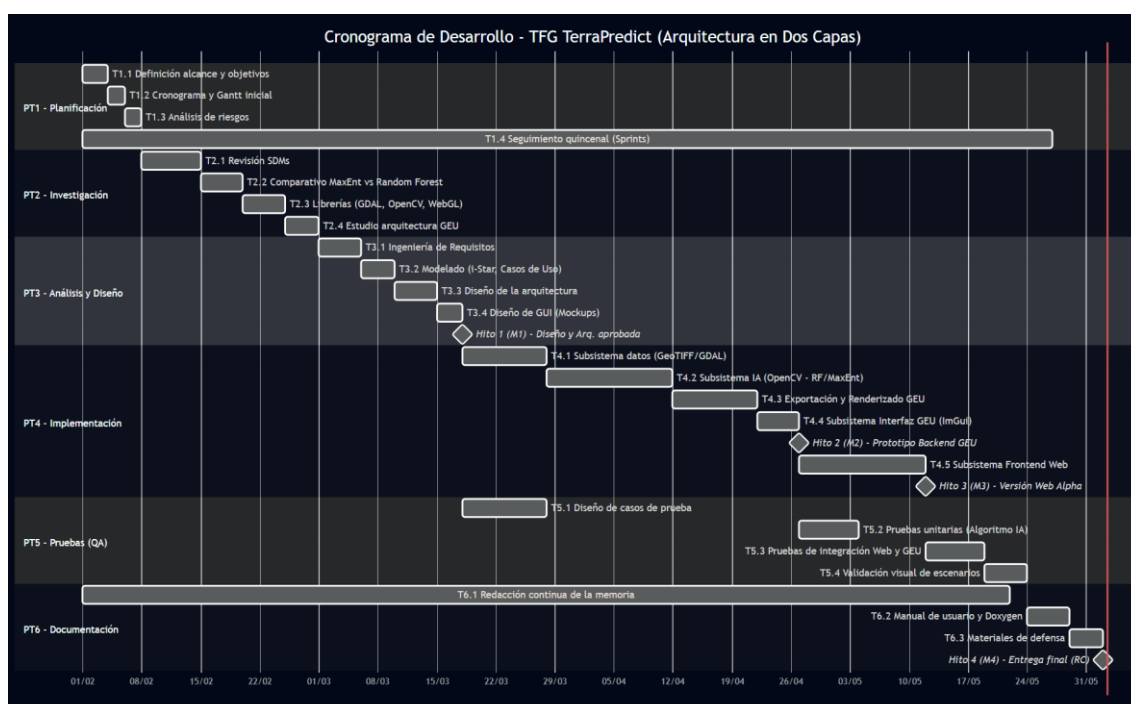


Ilustración 10 : Diagrama de Gantt del proyecto (autor)

Como se puede apreciar, la planificación propuesta rompe con el esquema tradicional de "cascada" estricta en favor de un enfoque más dinámico y eficiente. Cabe destacar dos decisiones estratégicas en la distribución de las tareas:

1. **Documentación continua y transversal (PT6):** a diferencia de las metodologías clásicas donde la redacción de la memoria se relega a la fase final, en este proyecto se ha planificado como una actividad paralela y continua (barra inferior del diagrama) que comienza desde la fase de investigación.
 - (i) Justificación: esta estrategia permite documentar los hallazgos técnicos (Estado del Arte) y las decisiones de arquitectura (Diseño) en "caliente", reduciendo el riesgo de pérdida de información y asegurando que la memoria final

sea un reflejo fiel y técnico del proceso, evitando la "deuda técnica" documental en las semanas previas a la entrega.

2. **Dependencias críticas:** se ha respetado estrictamente la lógica de ingeniería: no se inicia la implementación del subsistema de IA (T4.2) hasta que no se ha validado el subsistema de datos (T4.1) y el diseño de arquitectura (Hito 1). Esto minimiza la probabilidad de refactorizaciones costosas por errores de base.
3. **Solapamiento de calidad (QA):** las pruebas unitarias (PT5) no esperan a la finalización absoluta del código, sino que se inician en cuanto la interfaz de usuario y la visualización son funcionales, permitiendo un ciclo de corrección de errores (*bug fixing*) más ágil antes del hito de integración final.
4. **Desarrollo modular y desacoplado:** al separar procesamiento masivo (GEU) de la visualización final (CesiumJS/Three.js) el cronograma permite trabajar en el refinamiento visual de los gráficos analíticos (Plotly) de forma simultánea a la optimización matemática del algoritmo en C++, sin que el rendimiento de uno bloquee el desarrollo del otro.

3.4 Estimación de costes

En un proyecto siempre se deben de analizar los costes que puede requerir el mismo. Para ello necesitamos conocer que recursos vamos a necesitar y durante cuánto tiempo para poder calcular un presupuesto certero de forma que se pueda llevar la mejor planificación posible, teniendo en cuenta todos los factores que toman parte en la estimación. Para dicha estimación se tendrá en cuenta que la duración del proyecto corresponde a 4 meses (300 horas), dado que el TFG proporciona una carga lectiva de 12 créditos ETCS [27]. A continuación, se detallan las distintas categorías de costes a analizar:

- Costes de recursos humanos
- Costes de hardware
- Costes de software

3.4.1 Costes de recursos humanos

Aunque el proyecto es ejecutado por un único ingeniero, desde el punto de vista de la gestión (punto 3.2), se asumen diferentes roles profesionales con responsabilidades y costes de mercado diferenciados.

Para determinar el coste por hora, se han analizado las tablas salariales actuales en España para perfiles junior en el sector TIC, aplicando un coste de Seguridad Social y gastos estructurales (~30%) sobre el salario bruto anual [28], [29], [30], [31], [32]:

Rol	PT Asociado	% Carga	Horas est.	Coste/hora	Coste total
-----	-------------	---------	------------	------------	-------------

Project manager	Gestión y planificación	10%	30 h	20,00 €	600,00 €
Analista	Investigación y requisitos	20%	60 h	19,00 €	1.140,00 €
Desarrollador	Implementación (C++)	55%	165 h	17,50 €	2.887,50 €
QA Engineer	Pruebas y validación	15%	45 h	15,00 €	675,00 €
TOTAL		100%	300 h		5.302,50 €

Tabla 4 : Costes de recursos humanos

Teniendo en cuenta los sueldos promedios para los roles descritos en el proyecto, la suma total teniendo en cuenta el tiempo trabajado correspondiente a los 12 créditos ECTS del TFG, el coste en recursos humanos ascendería a 5.302,50 €, siendo esta una suma importante pero necesaria dado el calibre del trabajo de investigación.

3.4.2 Costes de hardware

El coste de hardware para este proyecto correspondería a los dispositivos y herramientas físicas utilizadas para el diseño e implementación del mismo. En este caso, para el desarrollo del proyecto solo necesitaremos un ordenador (de sobremesa) de gama medio-alta para poder hacer funcionar nuestro modelo de predicción a un rendimiento bastante notable. En este equipo se implementarán, diseñarán y ejecutarán los distintos prototipos que se vayan realizando a lo largo del recorrido del proyecto. Además de ello, se utilizará otro ordenador de sobremesa para poder trabajar desde distintos puntos geográficos (laboratorio y remoto). Además de ello, debemos sumar los periféricos de uso común (monitor, teclado, ratón, etc.). Las siguientes tablas muestran los equipos con los que se trabajarán para calcular el coste total a posteriori.

Concepto	Precio
NVIDIA Asus RTX 4060 Ti 16 GB	460,00 €
RAM Corsair Vengeance 8x2 16 GB DDR4	258,00 €
AMD Ryzen 7 5800X	188,95 €
Disco duro WD Black SN770 M.2 2TB	256,09 €
Placa base Asus Ex-320M-Gaming	85,00 €
Caja PC MSI MAG PANO M100R PZ	89,90 €
Fuente alimentación Mars Gaming 1000W Gold	79,90 €
Total	1.417,84 €

Tabla 5 : Componentes equipo (PC) personal

Concepto	Precio
NVIDIA Asus RTX 5060	328,99 €
RAM Corsar Vengeance 8x2 16 GB DDR4	258,00 €

Intel Core Ultra 7 265F	323,99 €
Disco Duro Kingston NV3 1TB	159,95 €
Placa Base GIGABYTE B760 GAMING X Intel B760	133,40 €
Torre PC Caja UNYKAch Aero C35	89,90 €
Tacens RADIX VII AG 700S Fuente Alimentación ATX 700W	54,90 €
Total	1.349,13 €

Tabla 6 : Componentes equipo (PC) laboratorio

Tras analizar en profundidad los componentes de los equipos principales a utilizar durante el desarrollo de este proyecto y su coste asociado [33], [34], podemos analizar una última tabla con el coste de hardware total, teniendo en cuenta los periféricos que se van a utilizar y su coste asociado también [33], [34]. También debemos conocer la vida útil media de cada uno de los dispositivos, para calcular el coste asociado a los mismos por el tiempo de uso que le vamos a dar (que será de 4 meses, asumiendo que el Trabajo de Fin de Grado tiene esa duración estimada) [35], [36], [37], [38]. La fórmula para calcular la amortización de los equipos sería la siguiente:

$$Coste_{\{Imputado\}} = \left(\frac{Precio_{\{Compra\}}}{VidaUtil_{\{Meses\}}} \right) * Duracion_{\{Proyecto\}}$$

En las tablas siguientes se muestran el coste de los equipos y periféricos utilizados para el desarrollo del proyecto.

Concepto	Vida útil	Coste/mes (€)	Tiempo uso	Coste	Coste total
PC Personal	48 meses	29,54 €	4 meses	1.417,84 €	118,16 €
Teclado 65% Royal Kludge	120 meses	3,38 €	4 meses	60,00 €	13,52 €
Ratón Logitech G402	48 meses	1,25 €	4 meses	60,00 €	5,00 €
Samsung Monitor Gaming Odyssey OLED G6	96 meses	6,77 €	4 meses	650,00 €	27,08 €
Total				2.187,84 €	163,76 €

Tabla 7 : Coste total equipo personal

Concepto	Vida útil	Coste/mes (€)	Tiempo uso	Coste	Coste total
----------	-----------	---------------	------------	-------	-------------

PC Laboratorio	48 meses	11,00 €	4 meses	1.349,13 €	112,41 €
Raton básico	36 meses	0,24 €	4 meses	8,59 €	0,96 €
Teclado básico	60 meses	0,24 €	4 meses	14,69 €	0,96 €
Monitor básico	64 meses	0,78 €	4 meses	50,00 €	3,12 €
Total				1.422,41 €	117,45 €

Tabla 8 : Coste total equipo laboratorio

Se ha supuesto un total de 4 meses de trabajo, por lo que cada uno de los dispositivos a utilizar supondrán un coste, que se calculará con su coste total y el intervalo de tiempo de trabajo definido. Dada esta información podemos proceder a calcular el coste total de los equipos de software.

Concepto	Coste	Coste total
Equipo Laboratorio	1.422,41 €	117,45 €
Equipo Personal	2.187,84 €	163,76 €
Total	3.610,25 €	281,21 €

Tabla 9 : Costes totales de hardware

3.4.3 Costes de software

En este caso, se analizará el coste asociado a licencias de uso de aplicaciones y herramientas software para el desarrollo del proyecto en su totalidad. En gran parte se han utilizado tecnologías y software *Open Source* [39], por lo que sus costes se reducen a cero. Por tanto, la tabla resultante de costes software es la siguiente.

Concepto	Herramienta elegida	Coste	Alternativa comercial	Ahorro generado
IDE Desarrollo	Visual Studio Community	0,00 €	Visual Studio Pro	180,00 €
IDE Web	Visual Studio Code	0,00€	WebStorm	70,00€
Motor GIS/Datos	GDAL	0,00 €	ArcGIS Desktop Basic	1.500,00 €
Librería IA	OpenCV	0,00 €	MATLAB + Toolboxes	800,00 €
Motor gráfico	GEU (C++)	0,00 €	Unity Pro / Unreal Enterprise	1.800,00 €
Dataset	WorldClim/GBIF	0,00 €	Datasets privados	500,00 €

Visor web 3D	CesiumJS / Three.js	0,00€	Mapbox Pro / ArcGIS Online	600,00€
Total		0,00 €		4.650,00€

Tabla 10 : Coste de software

GEU [40] es una infraestructura de software propio desarrollada por el grupo GGGJ de la Universidad de Jaén, especializada en la visualización y análisis interactivo de nubes de puntos masivas y datos multisensoriales procedentes de sensores térmicos y espectrales. Dicho usa el IDE Visual Studio 2022 Community, el cual es de uso libre, por tanto, el coste es nulo. Por otro lado, los dataset que vamos a utilizar también son de uso libre y gratis, ya que los hemos descargado de herramientas como WorldClim, GBIF, etc. Dichas herramientas serán comentadas en puntos posteriores. A su vez, para el desarrollo web se utiliza Visual Studio Code (gratuito), usando herramientas como CesiumJS y Three.js, de código abierto y por tanto también gratuitas. Se han elegido herramientas sin coste para desarrollar el proyecto y comparar con herramientas de pago, de forma que sea vea el ahorro acumulado que esto produce.

3.4.4 Costes totales

Finalmente, se presenta el presupuesto consolidado del proyecto. Se incluye una partida de contingencia y costes indirectos (10%) para cubrir gastos no inventariables como el consumo eléctrico, la conexión a internet y posibles desviaciones en la duración del proyecto, conforme al análisis de riesgos (R04).

Concepto	Coste total	% del presupuesto
Costes recursos humanos	5.302,50 €	86,50%
Costes hardware	271,68 €	4,40%
Costes software	0,00 €	0,00%
Subtotal	5.574,18 €	
Contingencia y costes indirectos	557,42 €	9,10%
Total	6.131,60 €	100,00%

Tabla 11 : Costes totales del proyecto

CAPÍTULO 4

MATERIALES Y HERRAMIENTAS

4. MATERIALES Y HERRAMIENTAS

4.1 Zonas de estudio

4.1.1 Sierra Mágina/Puerto de la Mata

Es importante conocer la orografía del terreno y de la zona donde vamos a realizar nuestro estudio, pues esta va a determinar bajo qué condiciones se encuentran las especies que podemos encontrar en dicha zona. Por ejemplo, una especie que se encuentre en pendiente estará sometida a unas condiciones meteorológicas y topológicas diferentes (y más extremas) que otra que esté en un valle sobre una superficie más o menos plana en la que no afecte tanto el viento, erosión, etc.

Bien, ahora pasaremos a comentar ambas zonas de estudio y los factores a tener en cuenta para nuestro modelo de predicción:

- **Sierra Mágina:** es un macizo situado en el sur de la provincia de Jaén. Constituye uno de los macizos más elevados de la Cordillera Subbética, consiguiendo una extensión de 20.000 hectáreas. Su núcleo central lo compone un gran bloque de roca caliza y está rodeada por un mar de olivos como ya bien conocemos. Alberga a su vez la cumbre más alta de la provincia, el Pico Mágina (2.164 m) [41].

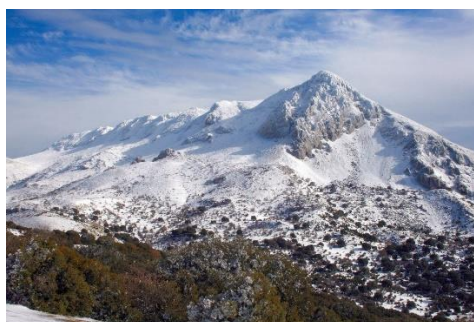


Ilustración 11 : Sierra Mágina [42]

- Orografía y relieve: se caracteriza por un relieve agreste y quebrado, dada su intensa erosión. Predomina el modelado kárstico, siendo común encontrar formaciones como dolinas, lapiaces y simas. Las pendientes suelen ser fuertes, llegando a superar el 30-40%. Esto condicionará el uso del suelo y la distribución espacial de la vegetación, factor clave para este proyecto.



Ilustración 12 : Dolina
[43]



Ilustración 13 : Lapiaces
[43]



Ilustración 14 : Sima
[44]

- Condiciones meteorológicas: en el clima predomina el mediterráneo, pero con una marcada continentalidad. Los inviernos son duros, con heladas frecuentes; los veranos son secos y bastante cálidos, aunque la altura a la que se encuentra Sierra Mágina suaviza la temperatura. En cuanto a precipitaciones, se encuentran entre 600 y 1000 mm anuales, con presencia de nieve en invierno de forma regular (actúa como reserva hídrica). Destaca la presencia de fuertes vientos, por lo que las condiciones son duras para la vegetación.
- Flora: predominan distintas especies según la altura (capa) a la que se encuentren. Entre ellas podemos hablar de la *sabina rastrera*, *jurinea fontqueri* y los pinos (*pinus nigra* [45]). En este proyecto se analizarán algunas especies en concreto para ver su evolución y distribución espacial con el modelo de predicción a diseñar e implementar.
- **Puerto de la Mata:** es uno de los pasos de montaña más emblemáticos y elevados de Sierra Mágina, con unos 1650 metros de altitud. Ofrece un perfil geológico y biológico muy variado en una superficie relativamente pequeña [46], [47].



Ilustración 15 : Puerto de la Mata [48]

- Orografía y topografía: predomina una alta pendiente, con laderas que caen hacia el valle. Presenta pequeñas depresiones y zonas de acumulación de derrubios, lo que supone un reto para la generación de Modelos Digitales del Terreno (MDT) precisos, necesarios para este proyecto.

- Condiciones meteorológicas: al ser un puerto, es propenso a vientos, lo cual favorece la evapotranspiración [49] de las plantas. Es una de las zonas donde la nieve persiste durante más tiempo durante la primavera, factor a tener en cuenta para nuestro modelo.
- Flora: abunda el *pinus nigra* (pino salgareño), capaz de resistir nieve y hielo. También aparecen los piornales (matorrales) que protegen al suelo de la erosión.



Ilustración 16 : Pinus Nigra [50]

4.1.2 Parque Nacional de Ordesa y Monte Perdido / Pirineos

Situado en el Pirineo oscense, constituye uno de los ecosistemas alpinos más extremos y representativos de la Península Ibérica. Con elevaciones que superan los 3.300 metros, ofrece un contraste climático drástico respecto a Sierra Mágina, siendo el escenario ideal para validar la escalabilidad y la capacidad de generalización del algoritmo predictivo a macroescala [51].



Ilustración 17 : Parque Nacional de Ordesa [52]



Ilustración 18 : Monte Perdido [53]

- Orografía y topografía: está dominada por abruptos valles de origen glaciar, cañones profundos y paredes verticales continuas. Este enorme gradiente altitudinal en un espacio tan grande exige el procesamiento de Modelos Digitales del Terreno precisos, lo que pone a prueba el procesamiento de los datos llevados a cabo en GEU.
- Condiciones meteorológicas: presenta un clima de alta montaña caracterizado por precipitaciones abundantes, humedad constante y

temperaturas extremas bajo cero en invierno. A diferencia del estrés hídrico de la zona sur, aquí los factores limitantes son el frío y la radiación solar, lo que permite evaluar cómo los modelos (MaxEnt y Random Forest) ponderan dinámicamente distintas variables bioclimáticas según el bioma.

- **Flora:** el estudio en esta zona se centra en especies [52], [54] que sobreviven al límite de su nicho ecológico. Destaca el pino negro (*Pinus uncinata*), que marca el límite altitudinal del bosque (*timberline* [55]); el abeto blanco (*Abies alba*), una especie de requerimientos fríos muy vulnerable a las proyecciones de cambio climático; y el haya común (*Fagus sylvatica*), cuya estricta dependencia de la umbría sirve para validar algorítmicamente la orientación de laderas calculada por el motor topográfico.

4.1.3 Justificación de las zonas de estudio

La elección de estos dos enclaves geográficos (sur peninsular y Pirineos) responde a dos criterios científicos fundamentales para el desarrollo de este proyecto:

- **Modelización de nichos críticos:** al tratarse de áreas con un gradiente altitudinal tan marcado, las especies presentes están viviendo al límite de su nicho ecológico. Esto garantiza que cualquier ligera variación térmica en las simulaciones de la Inteligencia Artificial (MaxEnt, Random Forest) se refleje rápidamente en desplazamientos evidentes de la idoneidad del hábitat en nuestros mapas de calor.
- **Centinelas del cambio climático:** la flora de alta montaña de ambos ecosistemas es extremadamente vulnerable al aumento de temperaturas. Esto convierte a estas zonas en el escenario perfecto para aplicar las proyecciones climáticas del CMIP6 y los escenarios socioeconómicos (SSPs), permitiendo observar visualmente si las especies tenderán a migrar hacia cotas más altas o se enfrentarán a la reducción crítica de su hábitat.

4.2 Fuentes de datos

Para la construcción del Gemelo Digital y la alimentación de los modelos predictivos, el sistema se ha diseñado para ingerir y fusionar datos de tres fuentes heterogéneas: registros biológicos puntuales, mallas climáticas globales y modelos digitales de elevación de las zonas de estudio.

4.2.1 GBIF

El Sistema Global de Información sobre Biodiversidad (GBIF por sus siglas en inglés) es una red internacional de datos para dar a cualquier persona en cualquier lugar acceso a información sobre todas las formas de vida en la Tierra (especies existentes). Esta herramienta permite almacenar, obtener y registrar datos sobre dónde y cuándo se han registrado las especies, mediante

técnicas más antiguas como especímenes en museos hasta tecnologías actuales basadas en fotografías geoetiquetadas de dispositivos inteligentes [2].

Su funcionamiento principal se rige por los siguientes puntos:

- **Estándar Darwin Core:** es un estándar de metadatos para organizar la información sobre especies de una manera en concreto para que siempre se pueda interpretar de la misma forma.
- **Integrated Publishing Toolkit (IPT):** es una herramienta software que se usa para publicar conjuntos de datos. El IPT genera un archivo que luego GBIF indexa y modifica según sus necesidades.
- **Procesamiento de datos:** cuando se registra un dataset, se realizan los siguientes pasos de forma interna en GBIF:
 - Crawling: descargar el dataset del servidor
 - Parsing: paso de datos brutos a individuales
 - Interpretación: normalización y validación de coordenadas geográficas
- **Acceso vía API:** GBIF proporciona un API REST para consultar desde una aplicación propia de forma automática los datos necesarios para utilizarlos directamente en nuestro programa.

Los datos de prueba se han obtenido de forma manual mediante el buscador que proporciona GBIF de ocurrencias de especies. Dicho buscador permite una personalización muy potente y completa [7]. Se permite elegir un polígono con unas coordenadas concretas, filtrar por especie, fecha, etc. Esto nos da la posibilidad de buscar especies concretas que se encuentren en la zona de estudio para centrarnos en la que más abunde en dicha localización. De esta forma, utilizando el buscador que proporcione GBIF podemos realizar un recorte a nuestro gusto para las zonas de estudio comentadas anteriormente, filtrar por las especies que queramos y descargar los datos de presencia de las mismas.

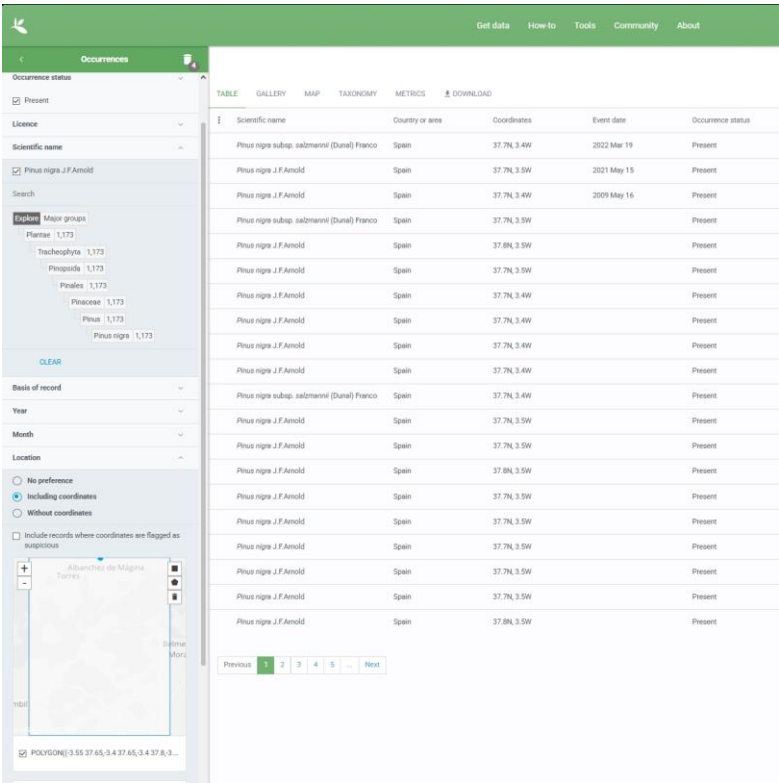


Ilustración 19 : Búsqueda de ocurrencias en GBIF(autor)

Tras la búsqueda de los datos necesarios, nos dirigimos al apartado “Descargas”, en el que podremos iniciar sesión para realizar la descarga. Hay distintos formatos para descargar la información geoespacial de las especies elegidas en el buscador de ocurrencias. Una de ellas es “Simple”, con el cual obtenemos un archivo .csv con toda la información directamente. Dicho formato será con el que trabajemos habitualmente en código. En la siguiente página se nos mostrará algo similar a la Ilustración 20.

OPCIONES DE DESCARGA						
	Datos sin procesar	Datos interpretados	Multimedia	Coordenadas	Formato	Tamaño de datos estimados
📄 CSV	X	✓	X	✓ (si está disponible)	CSV delimitado por tabulaciones (para usar en Excel, etc.) ⓘ	18 KB (comprimido para descargar)
📄 ARCHIVO DARWIN CORE	✓	✓	✓ (vínculos)	✓ (si está disponible)	CSV delimitado por tabulaciones (para usar en Excel, etc.) ⓘ	54 KB (comprimido para descargar)
📄 LISTAS DE ESPECIES	X	✓	X	X	CSV delimitado por tabulaciones (para usar en Excel, etc.) ⓘ	
📄 CUBO	X	✓	X	✓ (si está seleccionado)	CSV delimitado por tabulaciones (para usar en Excel, etc.) ⓘ	

Ilustración 20 : Opciones de descarga GBIF (autor)

Para el caso de este proyecto, trabajaremos con la opción simple del csv, ya que lo que necesitamos son las coordenadas de las especies para registrar la presencia de las mismas en el terreno de estudio. Tras hacer click en la opción elegida, se nos mostrará la siguiente pantalla (Ilustración 21).

FILTRO APLICADO EL VOLVER A EJECUTAR LA CONSULTA

La descarga ha comenzado y está siendo procesada en estos momentos.
 Por favor, espere hasta 3 horas para completar la descarga. La mayoría de descargas se completan en 15 minutos.
 Una vez preparada, se le enviará un correo electrónico de notificación a la siguiente dirección, incluyendo un enlace para descargar los resultados. aje00009@red.ujaen.es

Licencia: Sin especificar
 Asegúrese de leer el [acuerdo para usuarios de datos](#) y la [guía para citar](#).

API

Y

- Geometría POLYGON((-3.62369 37.70738,-3.54121 37.70738,-3.54121 37.76659,-3.62369 37.76659,-3.62369 37.70738))
- Incluye coordenada true
- Presenta un problema geoespacial false
- Estado del registro biológico present
- Nombre científico Pinus nigra J.F.Arnold

Ilustración 21 : Descarga datos presencia GBIF (autor)

Finalmente, tras unos minutos se enviará un correo con el enlace final de descarga, con el cual obtendremos un .zip con un archivo csv que ya incluye los datos solicitados. El formato de un archivo como estos es bastante completo, ya que incluye mucha información, a continuación, se muestran algunas de las columnas más importantes.

gbifID	datasetKey	occurrenceID	kingdom	phylum	class	order	family	genus	species	infraspecificEpithet
728501850	fab4c599-802a-4bfc-8a59-fc7515001bfa	MAGRANA:IFN3:205922	Plantae	Tracheophyta	Pinopsida	Pinales	Pinaceae	Pinus	Pinus nigra	
728501848	fab4c599-802a-4bfc-8a59-fc7515001bfa	MAGRANA:IFN3:205920	Plantae	Tracheophyta	Pinopsida	Pinales	Pinaceae	Pinus	Pinus nigra	
728501800	fab4c599-802a-4bfc-8a59-fc7515001bfa	MAGRANA:IFN3:205878	Plantae	Tracheophyta	Pinopsida	Pinales	Pinaceae	Pinus	Pinus nigra	
728501512	fab4c599-802a-4bfc-8a59-fc7515001bfa	MAGRANA:IFN3:205799	Plantae	Tracheophyta	Pinopsida	Pinales	Pinaceae	Pinus	Pinus nigra	
728501465	fab4c599-802a-4bfc-8a59-fc7515001bfa	MAGRANA:IFN3:205756	Plantae	Tracheophyta	Pinopsida	Pinales	Pinaceae	Pinus	Pinus nigra	salzmannii
5.933E+09	50c9509d-22c7-4a22-a47d-8c48425ef4a7	https://www.inaturalist.org/observations/109783440	Plantae	Tracheophyta	Pinopsida	Pinales	Pinaceae	Pinus	Pinus nigra	
3.457E+09	50c9509d-22c7-4a22-a47d-8c48425ef4a7	https://www.inaturalist.org/observations/102985020	Plantae	Tracheophyta	Pinopsida	Pinales	Pinaceae	Pinus	Pinus nigra	

Ilustración 22 : Formato CSV GBIF (I) (autor)

decimalLatitude	decimalLongitude	coordinateUncertaintyInMeters
377118	-34311	250
377118	-34425	250
377209	-34312	250
377477	-34881	250
377567	-34768	250
37731235	-3425874	130
37722817	-3425616	1000
37741103	-3500903	1420
37734211	-3472697	10
37736971	-3467687	10
37745027	-3442426	10
37742495	-3454101	10
37744566	-3445215	10

Ilustración 23 : Formato CSV GBIF (II) (autor)

Tras saber cómo funciona GBIF y que datos nos puede proporcionar, debemos decidir que datos necesitamos. Dadas las zonas de estudio, se han elegido tres especies por zona de estudio (Sierra Mágina/Puerto de la Mata y Ordesa/Monte Perdido), formando un total de seis especies para realizar la predicción. En referencia a la primera zona de estudio, se han elegido las siguientes especies:

- *Pinus nigra* (pino salgareño).
- *Pinus halapensis* (pino carrasco).
- *Juniperus communis* (enebro común).

Por otro lado, para la segunda zona de estudio se han escogido estas tres especies:

- *Abies alba* (abeto común).
- *Fagus sylvatica* (haya).
- *Pinus uncinata* (pino negro).

Cabe mencionar que se han intentado elegir especies que se distribuyan por distintas zonas para realizar una evaluación exhaustiva del modelo en diferentes zonas y bajo diferentes condiciones determinadas por el nicho ecológica de la especie. Por otro lado, seguramente surge la duda de por qué no se ha escogido como especie el olivo (*Olea Europaea*). Esto es porque el olivo es una especie manipulada por un agente externo, las personas. Los olivos se riegan por los humanos, por tanto sería una variable externa no contemplada por los datos bioclimáticos que se manejan, por lo que se ha descartado como opción.

4.2.2 WorldClim

WorldClim es una base de datos de alta resolución espacial de información meteorológica y climática global. Esta herramienta permite obtener datos tanto presentes como futuros. Es una fuente muy apropiada para alimentar los modelos de nicho ecológico como MaxEnt o Random Forest, es decir, para este proyecto [1].

Los aspectos a destacar sobre WorldClim son los siguientes:

- **Funcionamiento técnico:** no son mediciones directas, sino que se usan diferentes algoritmos (*thin-plate smoothing splines* [56]) que estiman los posibles valores climáticos, utilizando la latitud, longitud y la elevación (DEM) como variables predictoras.
- **Variables clave:** además de la temperatura y precipitación como valores principales, incluye 19 variables bioclimáticas estructuradas en modo multibanda. Estas variables, por ejemplo, pueden ser “precipitación del mes más cálido”, “temperatura del mes más frío”, etc. De esta forma y con estas variables, seremos capaces de profundizar aún más en la predicción con nuestro modelo de IA, ya que estos valores resultan clave para analizar los factores biológicos y climáticos a los que se someten las especies a estudiar.

En la página web de WorldClim encontramos diferentes recursos a los que podemos acceder de forma gratuita. Por un lado encontramos datos históricos (1970-2000), los cuales utilizaremos como la situación “actual” de nuestro modelo.

Historical climate data

This is WorldClim version 2.1 climate data for 1970-2000. This version was released in January 2020. [Climate modeling tools](#)

There are monthly climate data for minimum, mean, and maximum temperature, precipitation, solar radiation, wind speed, water vapor pressure, and for total precipitation. There are also 19 "bioclimatic" variables.

The data is available at the four spatial resolutions, between 30 seconds (~1 km²) to 10 minutes (~340 km²). Each download is a "zip" file containing 12 GeoTiff (.tif) files, one for each month of the year (January is 1; December is 12).

variable	10 minutes	5 minutes	2.5 minutes	30 seconds
minimum temperature (°C)	tmin 10m	tmin 5m	tmin 2.5m	tmin 30s
maximum temperature (°C)	tmax 10m	tmax 5m	tmax 2.5m	tmax 30s
average temperature (°C)	tavg 10m	tavg 5m	tavg 2.5m	tavg 30s
precipitation (mm)	prec 10m	prec 5m	prec 2.5m	prec 30s
solar radiation (kJ m ⁻² day ⁻¹)	srad 10m	srad 5m	srad 2.5m	srad 30s
wind speed (m s ⁻¹)	wind 10m	wind 5m	wind 2.5m	wind 30s
water vapor pressure (kPa)	vapr 10m	vapr 5m	vapr 2.5m	vapr 30s

Below you can download the standard (19) WorldClim [Bioclimatic variables](#) for WorldClim version 2. They are the average for the years 1970-2000. Each download is a "zip" file containing 19 GeoTiff (.tif) files, one for each month of the [variables](#).

variable	10 minutes	5 minutes	2.5 minutes	30 seconds
Bioclimatic variables	bio 10m	bio 5m	bio 2.5m	bio 30s

Ilustración 24 : Datos históricos WorldClim

Aquí visualizamos muchos enlaces de descarga de distintos tipos de datos. Lo que nos interesa para este Trabajo de Fin de Grado es proporcionar al modelo de la mayor cantidad de información de interés posible. Es por ello que descargaremos las variables bioclimáticas que se encuentran abajo en la Ilustración 24. Estas 19 variables bioclimáticas nos proporcionan información en muchos aspectos:

- BIO1 = Temperatura media anual
- BIO2 = Rango diurno medio (Media mensual de (temperatura máxima - temperatura mínima))
- BIO3 = Isotermalidad (BIO2/BIO7) (×100)
- BIO4 = Estacionalidad de la temperatura (desviación estándar ×100)
- BIO5 = Temperatura máxima del mes más cálido
- BIO6 = Temperatura mínima del mes más frío
- BIO7 = Rango anual de temperatura (BIO5-BIO6)
- BIO8 = Temperatura media del trimestre más húmedo
- BIO9 = Temperatura media del trimestre más seco
- BIO10 = Temperatura media del trimestre más cálido
- BIO11 = Temperatura media del trimestre más frío
- BIO12 = Precipitación anual
- BIO13 = Precipitación del mes más húmedo
- BIO14 = Precipitación del mes más seco
- BIO15 = Estacionalidad de la precipitación (Coeficiente de variación)
- BIO16 = Precipitación del trimestre más húmedo
- BIO17 = Precipitación del trimestre más seco
- BIO18 = Precipitación del trimestre más cálido
- BIO19 = Precipitación del trimestre más frío

Con estos datos el modelo va a ser entrenado para realizar las predicciones futuras posteriormente. Una vez tenemos los datos actuales ahora nos falta conseguir las variables bioclimáticas futuras. WorldClim nos proporciona una estructura de datos que sigue los escenarios SSPs (CMIP6) que hemos comentado anteriormente. Si nos dirigimos a la sección de datos futuros (concretamente eligiendo la resolución espacial de 30 segundos) encontraremos lo siguiente (Ilustración 25).

2041-2060

GCM	ssp126	ssp245	ssp370	ssp585
ACCESS-CM2	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
BCC-CSM2-MR	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
CMCC-ESM2	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
EC-Earth3-Veg	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
FIO-ESM-2-0	tn, tx, pr, bc	tn, tx, pr, bc		tn, tx, pr, bc
GFDL-ESM4	tn, tx, pr, bc		tn, tx, pr, bc	pr
GISS-E2-1-G	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
HadGEM3-GC31-LL	tn, tx, pr, bc	tn, tx, pr, bc		tn, tx, pr, bc
INM-CM5-0	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
IPSL-CM6A-LR	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
MIROC6	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
MPI-ESM1-2-HR	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
MRI-ESM2-0	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc
UKESM1-0-LL	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc	tn, tx, pr, bc

Ilustración 25 : Descarga de datos futuros en WorldClim (autor)

En esta página observamos distintos datos organizados por:

- **Etapas:** comprende diferentes épocas, desde 2021-2040 hasta 2081-2100
- **GCM (Global Climate Model):** indican las diferentes simulaciones que se han hecho desde distintos laboratorios de investigación. Dada la gran superficie terrestre, el CMIP6 agrupa múltiples GCMs para abarcar la mayor diversidad de situaciones posibles dada la zona específica desde donde se realice el estudio. Cada uno de estos laboratorios pueden utilizar variaciones a la hora del cálculo de las variables bioclimáticas. Para nuestro caso de estudio nos interesa la opción “MPI-ESM1-2-HR”, el cual se corresponde con “Max Planck Institute Earth System model versión 1.2 (High Resolution)”. Este nombre nos proporciona dos grandes rasgos importantes: el hecho de que es de resolución alta (HR) y que ha sido desarrollado por un instituto altamente reconocido en Alemania, lo que también nos conviene dado que estudiarán Europa con mayor detenimiento al ser un territorio relativamente cercano a nuestra zona de estudio.
- **Escenarios SSPs:** encontramos los escenarios SSP de los que hemos hablado con anterioridad (desde el ssp126 hasta el 585). De este modo tenemos múltiples escenarios con los que realizar predicciones y probar la veracidad y robustez del modelo.

- **Dato a descargar:** finalmente encontramos cuatro opciones, entre las cuales está “bc”, que se corresponde con las 19 variables bioclimáticas a las que queremos acceder, por tanto serán las que descarguemos para construir el dataset con el que trabajará el modelo.

Es importante destacar cómo se mide la resolución espacial en WorldClim y que opciones de descarga tenemos para saber cómo vamos a trabajar después. Bien, para explicar esto necesitamos entender un concepto previo. La Tierra se puede dividir en 360 grados. Cada uno de esos grados corresponden a 60 minutos de arco, que es la medida que utiliza WorldClim para medir la resolución espacial de sus datos.

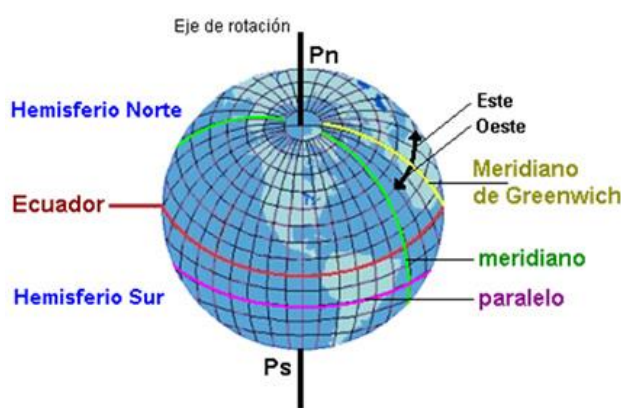


Ilustración 26 : Ángulos de la Tierra (meridianos) [57]

Para pasar de minutos de arco (medida en WorldClim) a superficie que representa un pixel, podemos aplicar la siguiente fórmula:

$$S = \left(\frac{C}{21.600} \times n \right) \times \cos(\phi)$$

Donde:

- S = resolución espacial en metros o kilómetros (longitud pixel)
- C = circunferencia ecuatorial de la Tierra (40.075 km)
- **21.600** = divisor de minutos obtenido a partir del número total de minutos de arco en un círculo completo (360°)
- n = resolución angular que obtenemos de WorldClim expresado en minutos (30 segundos corresponde a $n = 0,5$)
- **cos(ϕ)** = factor de corrección de latitud. Los meridianos se van estrechando conforme nos acercamos a los polos. Por tanto, en el ecuador, no afecta, ya que ϕ es 0 (coseno 1). Sin embargo, en nuestro

caso, $\varphi \approx 37,7^\circ$, reduciendo así la longitud (horizontal) del pixel. De esta forma ajustamos la resolución para que se adecue a la zona de estudio.

Para simplificar, la resolución con la que vamos a trabajar es de aproximadamente 1 km/pixel. Claramente esta resolución es bastante baja. Sin embargo, se aplicarán técnicas para mejorar los resultados finales y poder proyectar un mapa de calor continuo sobre el relieve tridimensional del terreno, garantizando que la visualización final sea topográficamente coherente.

Una vez hemos descargado los datos, ya los podemos usar para alimentar nuestro modelo. Sin embargo, deberíamos ver cómo son estos datos y que formato tienen. El formato es el conocido GeoTIFF [58], en el que se disponen los datos de distinta forma según descarguemos las variables históricas/actuales o las futuras:

- **Históricas/actuales:** se genera un archivo .tif (GeoTIFF) por cada variable bioclimática.
- **Futuras:** se genera un único archivo .tif multibanda (19 bandas, una para cada variable bioclimática).

A continuación se muestra una imagen (Ilustración 27) de un GeoTIFF con datos históricos abierto en el visor QGIS.

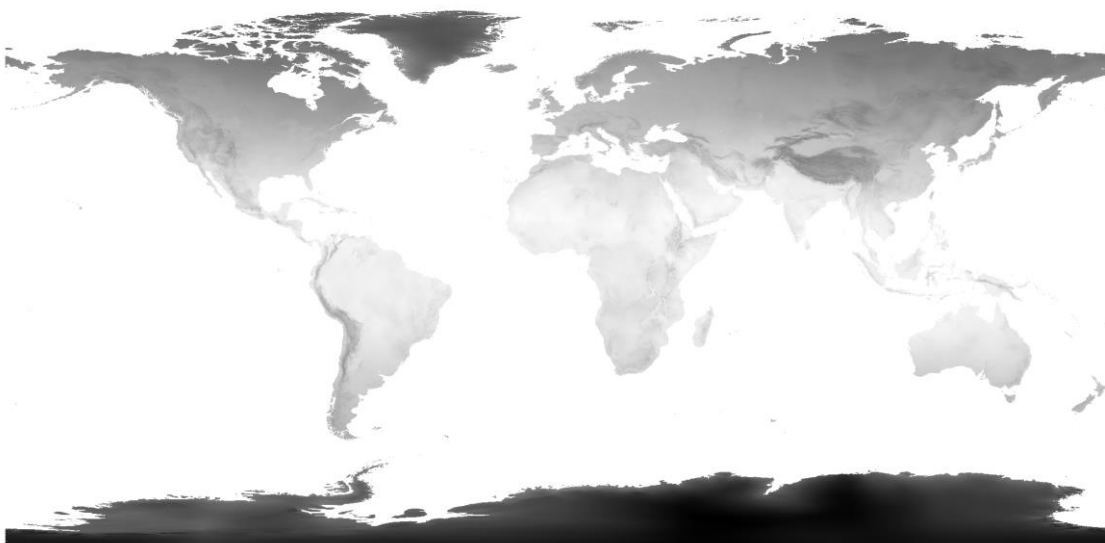


Ilustración 27 : Visualización GeoTIFF WorldClim en QGIS (autor)

En la Ilustración 28 se observa el gran problema de la resolución que proporciona WorldClim para una zona de estudio concreta.



Ilustración 28 : Resolución baja con GeoTIFF WorldClim (autor)

4.2.3 IGN

Hasta ahora, tenemos información actual y futura sobre el clima de la zona de estudio e información de presencias para los algoritmos de Machine Learning. Sin embargo, nos falta una serie de datos que van a proporcionar la posibilidad de representar la orografía del terreno mediante un Gemelo Digital 3D. Para ello necesitamos información sobre la altura de cada punto geográfico, de forma que se pueda representar el relieve del terreno. Para ello, una de las opciones predilectas es IGN (Instituto Geográfico Nacional). En concreto, usaremos la herramienta CNIG [59] (Centro Nacional de Información Geográfica), en el que encontraremos muchas herramientas útiles para diferentes objetivos. En este caso, usaremos el buscador de mapa, de forma que encontremos la misma zona de estudio que estamos analizando.

Como se observa en la Ilustración 29, podemos visualizar el buscador del CNIG. Tras seleccionar la zona de estudio deseada (ya sea mediante un bounding box, coordenadas geográficas, etc.), nos aparecen muchos resultados: desde mapas básicos (ej. Autonómicos) hasta otros datos más complejos como LIDAR. En este contexto, nos interesa obtener los Modelos Digitales del Terreno, los cuales contienen un raster (.tif) con información de altura del terreno. Cabe mencionar que se elegirá la variante “MDT05”, la cual proyecta una resolución de 5m/pixel, siendo muy buena esta resolución frente a la de WorldClim. Una vez seleccionamos los datos que queremos nos saldrán diferentes ficheros a descargar. En este caso, representa diferentes *tiles* (diferentes partes del DEM completo) con información de altura.

MDT05 - 1ª cobertura : 4 ficheros

Nombre	Formato	Fecha	Resolución	Altura	Acciones	Descarga
MDT05-1ª cobertura (5m)	COD	2010	5m	45.11		
MDT05-1ª cobertura (10m)	COD	2010	10m	45.11		
MDT05-1ª cobertura (20m)	COD	2010	20m	45.11		
MDT05-1ª cobertura (30m)	COD	2010	30m	45.11		

Ilustración 29 : Descarga de MDT en IGN (autor)

Tras descargar los datos, encontramos múltiples archivos .tif. Aquí se realiza un procesamiento previo para unir todos los tiles del DEM y poder trabajar con un único archivo. Para ello, nos apoyamos en QGIS y, en concreto, su consola OSGeo4W Shell. En ella, tras ubicar la carpeta donde se encuentran los datos, deberemos ejecutar los siguientes comandos:

```
gdalbuildvrt mosaico_virtual.vrt *.tif
```

```
gdal_translate -of GTiff mosaico_virtual.vrt DEM_unido_final.tif
```

Tras esto, nos quedará un único archivo .tif representando el DEM completo de la zona de estudio. Aún así, habrá zonas que no podremos representar con altura dado que dependemos de que haya datos en ese punto en concreto recogidos en IGN. Se visualiza perfectamente en la Ilusadw X. Hay zonas sin datos de altura que no podremos representar dado que la cobertura no tiene en cuenta dichos puntos concretos.

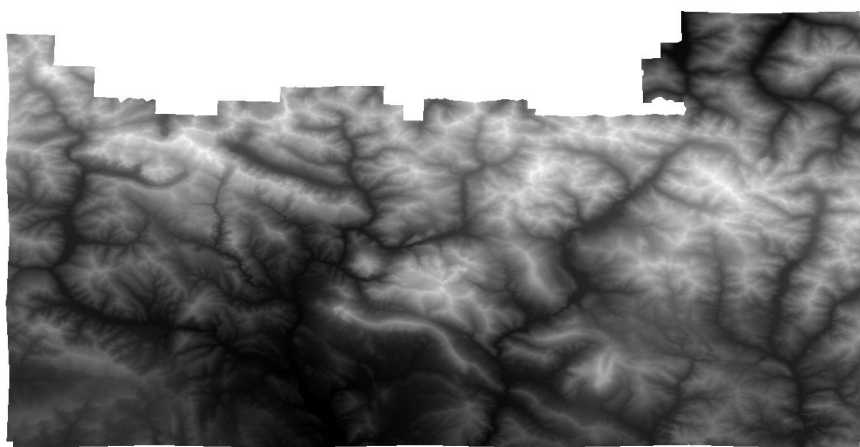


Ilustración 30 : Visualización DEM en QGIS (autor)

4.3 Entorno de desarrollo

El desarrollo del sistema no parte de cero, sino que se fundamenta en una infraestructura de software de investigación consolidada. Esta estrategia permite centrar los esfuerzos de ingeniería en la lógica de simulación predictiva y no en la creación de un motor gráfico desde la base.

El desarrollo del sistema no parte de cero, sino que se fundamenta en una infraestructura de software de investigación consolidada y en tecnologías web estandarizadas. Esta estrategia permite centrar los esfuerzos de ingeniería en la lógica de la simulación predictiva y en la accesibilidad de la información necesaria.

4.3.1 Lenguaje y entorno de programación (IDE)

El núcleo del proyecto se ha implementado utilizando C++ moderno (estándar C++17) bajo el entorno de desarrollo Microsoft Visual Studio 2022. Esta elección tecnológica responde a requisitos no funcionales críticos:

- **Rendimiento en tiempo real:** necesidad de gestión manual de memoria y punteros para la manipulación de texturas de alta resolución (GeoTIFFs climáticos) sin la latencia introducida por recolectores de basura (Garbage Collectors) de lenguajes como C# o Java.
- **Optimización hardware:** acceso de bajo nivel a la API gráfica (OpenGL/Vulkan) y a las instrucciones de la GPU (NVIDIA CUDA cores) para el renderizado masivo de geometría.

Dada la arquitectura en dos capas del proyecto, el desarrollo se ha dividido en dos entornos específicos:

- **Backend (Visual Studio 2022):** el núcleo de procesamiento nativo se ha implementado usando C++ (17). Esta elección se debe a que GEU ya trabaja con este IDE desde su creación, por tanto, para trabajar en cada módulo de forma unificada, se usa este entorno. También responde a requisitos no funcionales críticos para un software de este tipo: manipulación de matrices espaciales (GeoTIFFs), gestión manual de memoria, optimización de elementos 3D (nubes de puntos), etc.
- **Frontend (Visual Studio Code):** la herramienta web (TerraPredict) se ha desarrollado utilizando JavaScript/TypeScript y herramientas de empaquetado (Vite). Esto produce un entorno ligero donde se visualicen resultados de forma fácil y accesible para un usuario común, pero también cierto personalización con la programación de shaders WebGL.

4.3.2 Arquitectura base: motor GEU

A diferencia de los SIG tradicionales, GEU implementa una arquitectura basada en Grafo de Escena (Scene Graph), similar a motores comerciales como Unity o Blender. Esto permite organizar la información geográfica en una jerarquía de entidades, facilitando la gestión de transformaciones espaciales y la herencia de propiedades. El motor expone un Inspector de Propiedades que permite la manipulación en tiempo real de los atributos de cada objeto (posición, escala, material).

La plataforma anfitriona para el procesamiento de datos es GEU (Geoespacial and Enviromental tools of UJA) [40], un motor gráfico interactivo desarrollado por el Grupo de Gráficos y Geomática de Jaén (GGGJ [60]). GEU implementado una arquitectura basada en Grafo de Escena (Scene Graph [61]). Este software está dividido en distintos módulos, conteniendo cada uno una funcionalidad principal diferente (GEUCore, GEUApp, GEUHiperespectral, etc.). Esto hace que sea una herramienta potente para procesamiento de datos multisensoriales, visualización de nubes de puntos gigantes, modelado y muchos otros aspectos de un SIG.

Una de las características más interesantes y relevantes de GEU para este TFG es su capacidad para leer y fusionar distintos tipos de datos espaciales en un mismo espacio de coordenadas:

1. **Modelos de elevación:** soporte optimizado para el instanciado de mallas topográficas a partir de Modelos Digitales del Terreno (MDT), permitiendo procesar así la orografía de las zonas de estudio mencionadas en apartados anteriores.
2. **Información raster:** capacidad de lectura de formatos geospaciales como GeoTIFF (.tif), que es el estándar utilizado por WorldClim para las variables bioclimáticas.
3. **Información vectorial:** soporte para primitivas vectoriales (.GeoJSON, .SHP), lo cual facilita bastante la delimitación precisa de las zonas de estudio (importante en este caso ya que WorldClim trabaja con variables a nivel global) y la exportación estandarizada de métricas.

4.3.3 Integración del módulo de simulación

El trabajo desarrollado en este TFG se integra como un nuevo módulo funcional a los ya existentes dentro del ciclo de vida de GEU. Este nuevo módulo (GEUPredicciónEspecies) centralizará toda la lógica algorítmica de Machine Learning, leyendo capas climáticas, presencia de especies, generación de probabilidad espacial (mapas de calor, nubes 3D) y exportación de resultados para la visualización web posterior.

4.3.4 Librerías de terceros (GEU)

Para extender las capacidades base de GEU hacia la simulación predictiva, se han integrado las siguientes librerías de código abierto:

- Librerías de backend (GEU / C++):
 - **GDAL (Geospatial Data Abstraction Library):** utilizada para la abstracción de datos ráster. Permite decodificar los ficheros .tif de WorldClim, gestionar los sistemas de coordenadas y extraer matrices numéricas con las que trabajar.
 - **OpenCV (Open Source Computer Vision Library):** aunque es conocida para la visión artificial y en GEU se utilizaba con otro objetivo, su módulo *cv:mv* se utiliza en este proyecto como el motor principal de Machine Learning encargado del entrenamiento e inferencia espacial del modelo predictivo Random Forest.
 - **Dear ImGui/ImPlot:** librería para la construcción de interfaces gráficas inmediatas. Se emplea para desarrollar los paneles de control del simulador (selectores de año, escenarios SSP, umbrales de probabilidad) integrados visualmente en la interfaz nativa de GEU. librerías para la construcción de interfaces gráficas inmediatas dentro del motor. Se emplean para desarrollar el panel de control del simulador (datos, modelo, resultados, etc.) y renderizar las curvas de respuesta ecológicas de las especies.
 - **GLM (OpenGL Mathematics):** proporciona estructuras matemáticas (vectores, matrices) necesarias para los cálculos espaciales.
- Librerías de frontend (Web / JavaScript):
 - **CesiumJS [19]:** motor de renderizado geoespacial 3D a nivel global (*Virtual Globe*), empleado para representar resultados a macroescala y realizar comparativas entre escenarios climáticos (SSPs).
 - **Three.js [20]:** librería 3D de bajo nivel sobre WebGL, utilizada para instanciar los Gemelos Digitales a escala local sobre las zonas de estudio, inyectando también los mapas de calor desarrollados en GEU sobre la geometría del terreno mediante el uso de shaders.

4.3.5 Plataforma web de visualización: TerraPredict

Además del motor de procesamiento GEU, este proyecto, como se ha comentado, contempla el desarrollo de una aplicación web (TerraPredict) para visualizar e interactuar con los resultados

espaciales exportados previamente. Es por ello que es importante mencionar cómo se va a elaborar este entorno web, que herramientas se van a utilizar para desarrollarlo, etc.

Se **seguirá** una arquitectura Single Page Application (SPA [62]) utilizando el siguiente stack tecnológico:

Capa	Tecnología	Función
Bundler y servidor de desarrollo	Vite 5	Empaquetado de módulos, recarga en caliente (hot reload), optimización de build
Estilos	Tailwind CSS 3 + PostCSS	Framework de utilidades CSS para diseño responsive y tema oscuro
Lógica de aplicación	Vanilla JavaScript	Gestión de estado, eventos y componentes
Visor geoespacial 2D	CesiumJS 1.118	Globo terráqueo virtual con capas de heatmap, GeoJSON y terrain provider
Visor 3D local	Three.js 0.184	Escena tridimensional del DEM con heatmap y nubes de puntos PLY
Gráficos analíticos	Plotly.js 2.33	Curvas de respuesta e histogramas de probabilidad

Tabla 12 : Stack tecnológico TerraPredict

Para que TerraPredict consuma de forma automática todos los archivos exportados y que sea fácil de ubicar más si se añade otra especie, periodo o algoritmo, se ha seguido la siguiente estructura:

- ***_Actual.geojson** y ***_SSP_Periodo.geojson**: puntos de predicción con propiedades (probabilidad, especie, escenario, métricas).
- ***_Actual.png** y ***_SSP_Periodo.png**: mapas de calor RGB con la paleta de probabilidad de GEU.
- ***_Actual.tif**: ráster GeoTIFF para análisis en GIS externos.
- ***_ResponseCurves.csv**: curvas de respuesta de cada variable bioclimática.
- ***_Metrics.json**: métricas de rendimiento del modelo.
- ***_Config.json**: configuración del modelo (hiperparámetros, variables seleccionadas, etc.).
- ***_Diferencias.png**: mapa visual de pérdida/ganancia del hábitat con respecto a la actualidad.
- ***.ply**: nubes de puntos 3D para el visor tridimensional.
- **terrain.json** y **terrain.bin**: modelo digital del terreno preprocesado para Three.js.

CAPÍTULO 5

ANÁLISIS Y DISEÑO

5. ANÁLISIS Y DISEÑO

A continuación, se detallan las funciones específicas que el sistema debe implementar. Dada la arquitectura en dos capas (Backend/frontend), los requisitos se han dividido y se ha aplicado la técnica de priorización MoSCoW (*Must, Should, Could, Won't*) para alinear el desarrollo con la planificación temporal.

5.1 Ingeniería de requisitos

El primer paso en la fase de análisis consiste en formalizar las necesidades del sistema. Para ello, se ha seguido una especificación basada en el estándar IEEE 830, adaptada al ciclo de vida incremental del proyecto. El objetivo es delimitar claramente la frontera entre el motor anfitrión (GEU) y el módulo de simulación desarrollado en este TFG.

5.1.1 Identificación de actores

Se han identificado los siguientes actores, entendidos como las entidades (humanas o digitales) que interactúan directamente con el sistema para intercambiar información:

- **ACT-01: Motor GEU (sistema anfitrión):** es el perfil técnico encargado de operar el motor GEU. Interactúa con el entorno C++ para cargar los MDT, suministrar las matrices climáticas pesadas (GeoTIFFs), ejecutar el entrenamiento y predicciones del modelo de IA (OpenCV) y exportar los resultados espaciales.
- **ACT-02: Usuario final (frontend):** es el usuario de la plataforma web *TerraPredict*. Su objetivo es el análisis científico, por lo que no requiere conocimientos de programación. Interactúa con la interfaz gráfica web para navegar por los Gemelos Digitales (CesiumJS/Three.js), comparar mapas de calor de las diferentes distribuciones de diferentes especies con diversos escenarios climáticos y visualizar curvas de respuesta y otras métricas del modelo entrenado.
- **ACT-03: API de biodiversidad (GBIF):** sistema externo pasivo del que el motor se alimenta con las coordenadas de ocurrencia de especies para el estudio.
- **ACT-04: Repositorio climático (WorldClim):** fuente de datos externa proveedora de las variables bioclimáticas históricas y de las proyecciones climáticas futuras (GCMs) bajo el estándar CMIP6/SSPs.

5.1.2 Catálogo de requisitos funcionales (RF)

A continuación, se detallan las funciones específicas que el módulo debe implementar. Se ha aplicado la técnica de priorización MoSCoW (Must, Should, Could, Won't) para alinear el desarrollo con la planificación temporal.

Bloque 1: Ingesta de datos y exportación (GEU)

ID	Nombre requisito	Descripción	Prioridad
RF-01	Descarga de ocurrencias biológicas	El sistema debe permitir procesar registros de ocurrencia (ej. <i>Pinus nigra</i> , <i>Pinus uncinata</i>) filtrados por área geográfica procedente de bases de datos como GBIF	Must
RF-02	Ingesta de variables climáticas	El módulo de predicción debe ser capaz de leer e interpretar archivos ráster en formato GeoTIFF (.tif) procedentes de WorldClim mediante GDAL, extrayendo matrices numéricas y metadatos de georreferenciación	Must
RF-03	Procesamiento topográfico	El sistema debe integrar Modelos Digitales del Terreno (MDT) para extraer variable de altitud, pendiente y orientación de laderas	Must
RF-04	Exportación de interoperabilidad	El motor debe exportar los mapas de probabilidad generados a formatos estándar y ligeros (GeoTIFF, GeoJSON, etc.), además de otros datos como nubes de puntos (PLY), gráficas (CSV), etc.	Must

Tabla 13 : Requisitos funcionales (I)

Bloque 2: Procesamiento de Machine Learning (GEU)

ID	Nombre requisito	Descripción	Prioridad
RF-05	Entrenamiento multimodelo	El sistema integrará algoritmos de IA (MaxEnt nativo y Random Forest con OpenCV) utilizando variables bioclimáticas como predictores	Must
RF-06	Inferencia predictiva espacial	Dado un conjunto de proyecciones climáticas futuras, el sistema debe generar una matriz continua de probabilidad de supervivencia (0.0-1.0) para cada pixel del área de estudio de la especie analizada	Must
RF-07	Generación de curvas de respuesta	El motor GEU debe calcular y renderizar gráficamente (ImPlot) cómo afecta individualmente cada variable ambiental a la probabilidad de supervivencia de la especie	Should

Tabla 14 : Requisitos funcionales (II)

Bloque 3: Visualización e interfaz web (Frontend TerraPredict)

ID	Nombre requisito	Descripción	Prioridad
RF-08	Visor Global 2D/3D	La plataforma web debe integrar un globo virtual multicapa (ESRI Satellite, ComosDB, etc.) usando CesiumJS, permitiendo superponer los mapas de probabilidad sobre	Must

		cartografía base a nivel macroescalar	
RF-09	Comparador de escenarios	La interfaz web debe permitir al usuario comparar visualmente la distribución de especies actual frente a escenarios futuros	Must
RF-10	Gemelo Digital local interactivo	El visor debe renderizar la orografía 3D (Three.js) e inyectar el mapa de calor predictivo como una textura dinámica sobre el relieve.	Must
RF-11	Panel analítico	La web debe incluir paneles de datos (Plotly/UI) para mostrar las métricas de evaluación del modelo (ej. Curvas ROC, AUC) calculadas por GEU	Should

Tabla 15 : Requisitos funcionales (III)

5.1.3 Catálogo de requisitos no funcionales (RNF)

Estos requisitos definen los atributos de calidad del sistema y las restricciones técnicas impuestas por la arquitectura de GEU.

- **RNF-01 Rendimiento computacional (GEU):** el motor en C++ debe optimizar el uso de memoria RAM y recursos para ser capaz de procesar matrices de datos globales (GeoTIFFs masivos de alta resolución) sin desbordamientos de memoria.
- **RNF-02 Rendimiento de renderizado (frontend):** la plataforma web basada en WebGL debe garantizar una tasa de refresco fluida (mínimo 30 FPS) durante la navegación e interacción con la malla topográfica 3D, asegurando una buena experiencia de usuario en navegadores estándar.
- **RNF-03 Desacoplamiento arquitectónico:** el sistema debe mantener una estricta separación estructural. El núcleo de cálculo de predicciones no debe tener dependencias directas del código de la interfaz de visualización web, comunicándose exclusivamente mediante el intercambio de archivos siguiendo el estándar.

- **RNF-04 Usabilidad:** la interfaz de usuario, tanto en GEU como en la herramienta web (TerraPredict) debe ser accesible y usable por cualquier usuario, siguiendo un estándar adecuado de color y estructura para que su manejo sea sencillo.
- **RNF-05 Extensibilidad:** la arquitectura del motor debe permitir la futura incorporación de nuevos algoritmos de Machine Learning o métricas de evaluación implementando interfaces comunes en C++.
- **RNF-06 Precisión espacial:** la proyección de las texturas predictivas sobre los Modelos Digitales del Terreno en el entorno 3D debe mantener coherencia a nivel topográfico (sin cambios inesperados en el mapa de calor y respetando la orografía de las zonas de estudio).

5.2 Modelado de objetivos y funcionalidad

Tras identificar los requisitos del sistema a desarrollar, se procede al modelado visual del sistema. Este proceso de diseño busca alinear las expectativas del usuario final con las capacidades técnicas de la arquitectura mencionadas en el apartado anterior.

5.2.1 Modelo de dependencias estratégicas (i-Star / Strategic Dependency Model)

Dado que la arquitectura del proyecto se distribuye en distintos componentes y que el sistema no opera de forma aislada, se ha utilizado el marco de trabajo i-Star para modelar el escenario organizativo de dicha arquitectura.

Específicamente, se ha diseñado un Modelo de Dependencias Estratégicas (SD) que ilustra cómo los distintos agentes de la arquitectura delegan responsabilidades para alcanzar sus objetivos funcionales y no funcionales. A continuación se expone un análisis de las dependencias establecidas del diagrama i-Star (Ilustración 31):

- **Dependencias de meta (*Goal dependencies*):** el actor Investigador depende de la plataforma web TerraPredict para alcanzar el objetivo crítico de *Visual la distribución de especies bajo escenarios de cambio climático*. Al usuario final le es indiferente cómo se realiza el cálculo por debajo, delegando toda la responsabilidad en el frontend.
- **Dependencias de tarea (*Task dependencies*):** para que la plataforma web cumpla la meta del usuario, esta depende estrictamente de *GEU*. Existe una delegación de la tarea *Calcular algoritmo predictivo*. La plataforma web no se encarga de realizar la predicción, por ello dependen del motor para que le entregue los resultados a mostrar.

- **Dependencias de recurso (*Resource dependencies*):** se identifican dos dependencias clave acerca del suministro de datos:
 - *GEU* depende completamente de las entidades externas (*WorldClim/GBIF*) para poder obtener los datos necesarios con los que será capaz de realizar la simulación.
 - *TerraPredict* depende directamente de *GEU* para la obtención de los recursos finales (archivos GeoTIFF, GeoJSON, etc.) para la visualización de los componentes útiles para el *Usuario final*.
- **Dependencias de calidad (*Softgoal dependencies*):** finalmente, acudimos a las dependencias referentes a los requisitos no funcionales. Se impone que el *Usuario final* obtenga una *Interactividad fluida en 3D* en *TerraPredict*. Esta meta blanda fuerza el uso de tecnologías aceleradas por hardware como WebGL para asegurar una experiencia de usuario óptima.

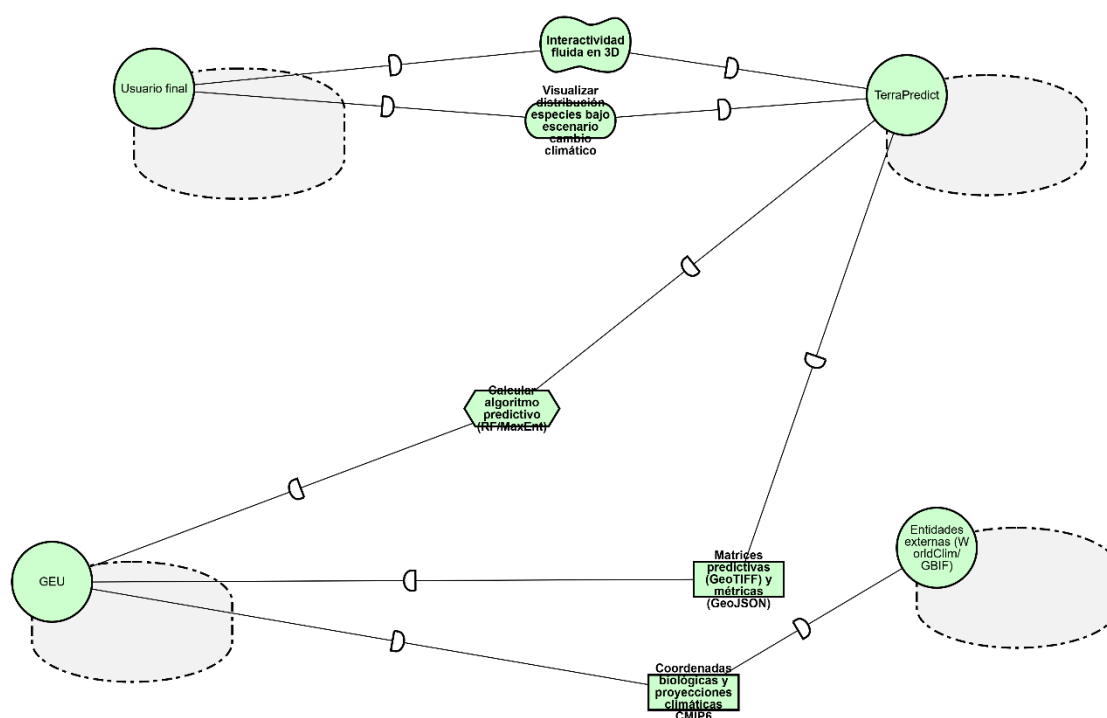


Ilustración 31 : Diagrama i-Star (autor)

5.2.2 Diagrama de casos de uso (UML)

Para formalizar la interacción funcional entre el usuario (o los distintos tipos de usuario que conforman el proyecto) y el sistema, se ha modelado el comportamiento mediante un diagrama de casos de uso (UML). Este diagrama define la frontera del sistema (*system boundary*) dividida en dos grandes bloques tecnológicos, estableciendo qué funcionalidades son usadas por cada actor y qué procesos son delegados a actores secundarios (entidades externas).

De acuerdo con la descripción de actores de apartados anteriores, las funcionalidades se estructuran de la siguiente manera:

1. Subsistema backend (GEU): este bloque es operado por el administrador del sistema, el cual tendría un perfil más técnico. Se encarga de la ingesta de datos provenientes de las entidades externas (WorldClim y GBIF) y el cálculo matemático para la predicción.
 - (a) Se le asignan los casos de uso: *Configurar parámetros de orografía (MDT)*, *Cargar matrices climáticas globales* y *Ejecutar entrenamiento del modelo (IA)*.
 - (b) El proceso de Exportar Gemelo Digital (Archivos de interoperabilidad) se modela como una extensión tras el cálculo.
2. Subsistema frontend (TerraPredict): este bloque es usado por el usuario final. Tiene responsabilidades únicamente analíticas (visualizar y analizar resultados).
 - (a) Se le asignan casos de uso como: *Navegar por el entorno 3D local*, *Comparar escenarios climáticos SSP* y *Evaluar curvas de respuesta ecológica*.

Como se muestra en la Ilustración 32, este modelado UML mediante un diagrama de casos de uso muestra la aplicación del principio de Separación de Responsabilidades (*Separation of Concerns* [63]).

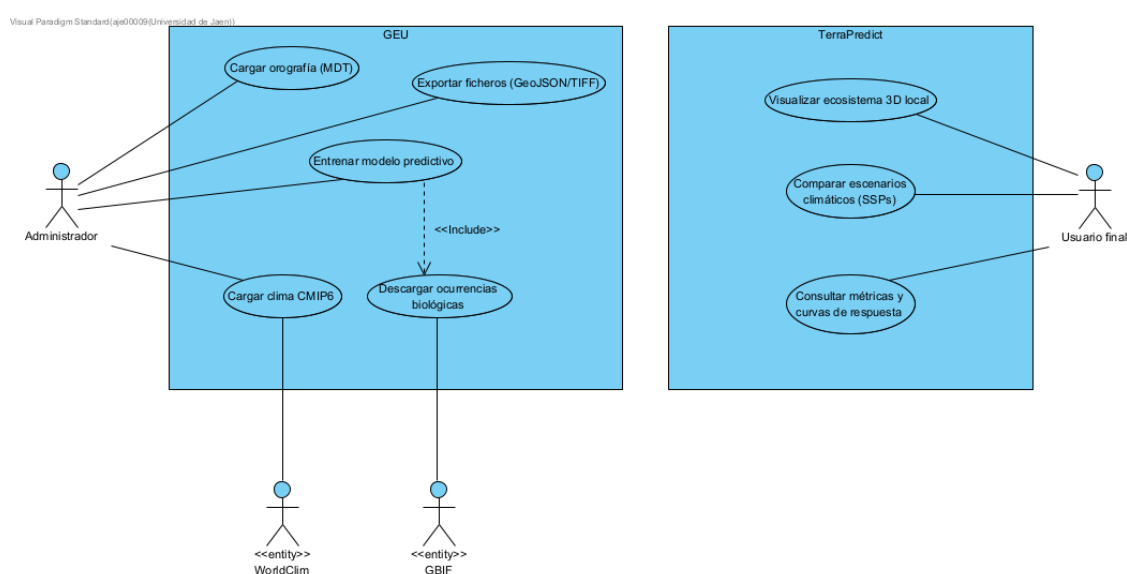


Ilustración 32 : Diagrama de casos de uso (autor)

5.2.3 Diagrama de actividades (UML)

Para complementar a los diagramas realizados en este apartado, se ha modelado el comportamiento dinámico mediante un diagrama de actividad. Este consiste en mostrar el flujo de trabajo secuencial (pipeline) desde la inicialización del entorno hasta el fin de la visualización en la web.

Para representar adecuadamente la separación de responsabilidades, el diagrama (Ilustración 33) se ha estructurado mediante calles (*swimlanes*). En el flujo se detallan las siguientes fases críticas:

1. **Fase de ingesta (GEU):** el administrador inicia el proceso definiendo el área de estudio. Después, se alcanza un nodo *fork* donde el sistema procesa tanto Modelos Digitales del Terreno (MDT) y recoge los datos de las entidades externas (GBIF y WorldClim).
2. **Fase de inferencia IA (OpenCV):** una vez que tenemos las matrices con los datos geoespaciales en memoria, el flujo entra en el apartado más analítico. Aquí se realizan los procesos lógicos más esenciales: generación de puntos de fondo (*background points*), la expansión de variables (características cuadráticas) y el entrenamiento del modelo.
3. **Transición y persistencia:** el ciclo finaliza con la exportación del Gemelo Digital. GEU escribe a disco los ficheros necesarios para la web (GeoJSON para métricas, PLY para topografía de la nube y GeoTIFF para texturas de probabilidad, etc.).
4. **Fase de análisis visual (frontend web):** el control pasa al navegador. El flujo básicamente finaliza en un *loop* interactivo donde el usuario final puede alternar entre diferentes escenarios climáticos (SSPs), evaluación de gráficas, visualización del modelo 3D, etc.

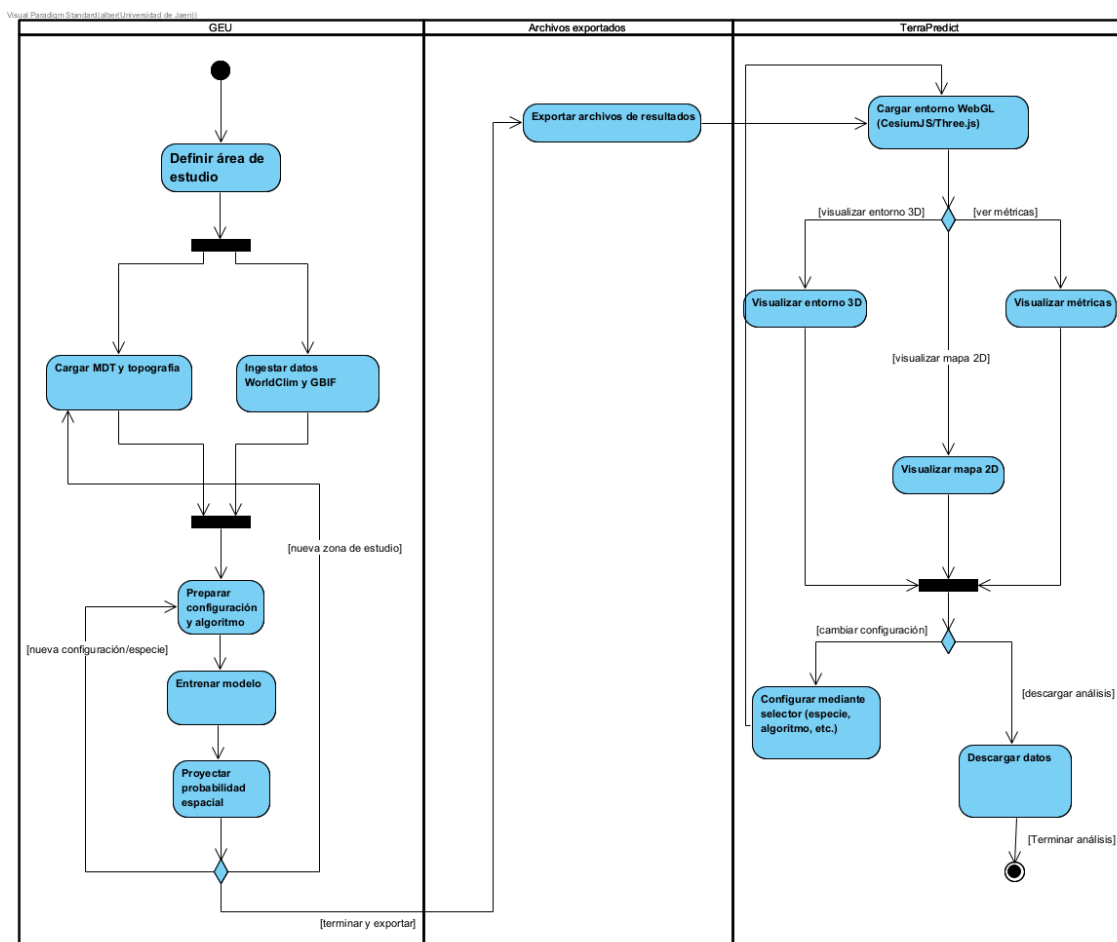


Ilustración 33 : Diagrama de actividad (autor)

5.3 Arquitectura del sistema

La complejidad inherente a la integración del modelado predictivo espacial, el procesamiento de datos climáticos enormes y la visualización interactiva requiere una arquitectura robusta y desacoplada. Por ello, como se ha comentado con anterioridad, se ha optado por un diseño basado en dos capas (motor de procesamiento: GEU/ aplicación web interactiva: TerraPredict), siguiendo el principio de inversión de dependencias (SOLID [64]) para garantizar la mantenibilidad del código.

5.3.1 Diagrama de componentes

El sistema se estructura en dos grandes bloques que se comunican mediante el intercambio de tipos de datos estandarizados. Como se muestra en la Ilustración 34, el diseño arquitectónico encapsula la complejidad del sistema en los siguientes módulos:

1. **GEU:** es el núcleo de procesamiento intensivo. Está compuesto por el controlador de simulación, la capa de abstracción de datos (GDAL) y el motor de Machine Learning (OpenCV).

2. **TerraPredict:** actúa como la capa de presentación de la información (resultados). Utiliza módulos de renderizado WebGL (CesiumJS/Three.js) y controladores para la interfaz. Consume los archivos generados por GEU.
3. **Capa de interoperabilidad:** conjunto de archivos (GeoTIFF, mallas PLY, etc.) que actúan de puente entre ambas capas.

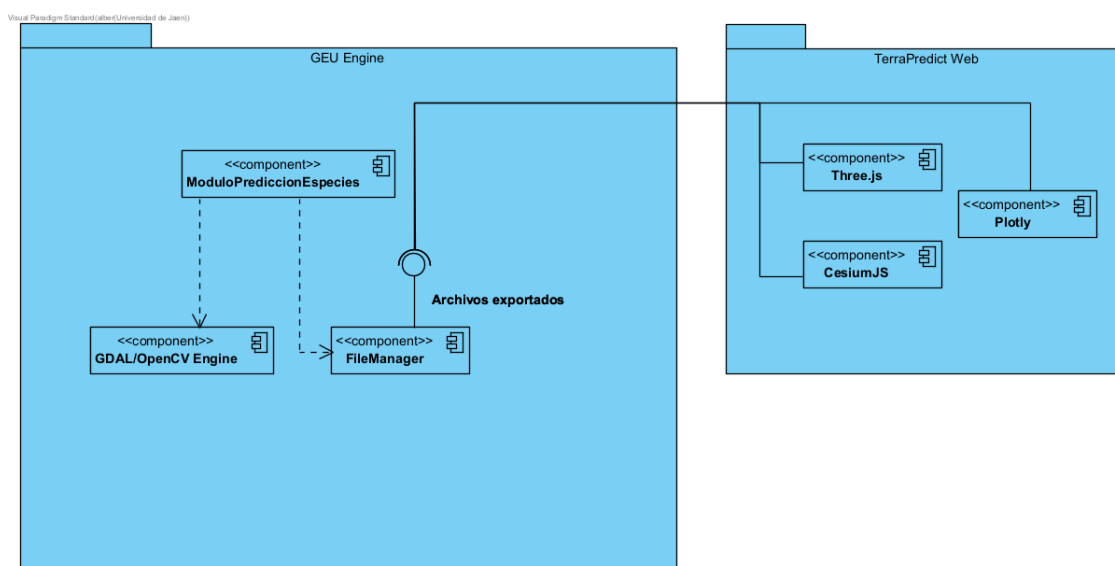


Ilustración 34 : Diagrama de componentes (autor)

5.3.2 Patrones de diseño

Para garantizar la calidad del software, se han seguido una serie de patrones de diseño tanto GoF, para la arquitectura interna, como GRASP, para la asignación de responsabilidades.

Patrones GoF (Gang of Four):

- Singleton (creacional):
 - Clase: SimulationManager
 - Justificación: asegura que haya un solo punto de acceso global y una única instancia para el gestor del ciclo de vida del backend, centralizando todo su comportamiento.
- Strategy (comportamiento):
 - IPredictionModel
 - Justificación: permite desacoplar el algoritmo matemático de su ejecución como tal. Permite cambiar fácilmente entre diferentes algoritmos de IA (RandomForest/MaxEnt) sin alterar el código fuente.

- Facade (estructural):
 - Clase: GeoDataLoader
 - Justificación: simplifica la API de GDAL ofreciendo una interfaz centralizada y simple para el manejo de coordenadas y bandas ráster.

Patrones GRASP:

- Controlador (Controller):
 - Aplicación: la clase SimulationManager actúa como controlador de fachada. Es el primero que recibe las peticiones de ejecución, coordinando todo el flujo entre los objetos de lectura, entramiento, etc.
- Experto en información (Information Expert):
 - Aplicación: asignación de la extracción de valores bioclimáticos y topográficos a la clase ClimateDataHandler.
 - Justificación: se asigna la responsabilidad a la clase que contiene la información necesaria para cumplirla. Esta clase es la única que abstrae el acceso a GDAL, rasters GeoTIFFs, etc.
- Creador (Creator):
 - Aplicación: la clase ExportManager es responsable de instanciar los objetos serializadores (TiffWriter, JsonWriter).
 - Justificación: se aplica el patrón porque la clase contiene la información para crear instancias de los archivos con los resultados en ellos.
- Bajo acoplamiento (Low Coupling):
 - Aplicación: uso de interfaces (IModule, IPredictionModel)
 - Justificación: minimizamos las dependencias directas entre clases. Garantiza que el cambio en alguna clase no afecte al resto del módulo o del sistema en general.

5.3.3 Diagrama de despliegue (UML)

Dada la separación tecnológica, se ha optado por modelar un diagrama de despliegue para ilustrar la topología física del sistema. Así, se especifica cómo los artefactos software se distribuyen en diferentes nodos de ejecución y la relación entre ellos.

En el diagrama se observan dos nodos: una máquina local o algún tipo de servidor donde con capacidad de cómputo intensivo (CPU/GPU) donde se encuentra el .exe de GEU; y un dispositivo cliente que ejecuta un navegador web

estándar con soporte WebGL. La conexión entre ambos se basa en la transferencia estática de ficheros a través de un protocolo HTTPS.

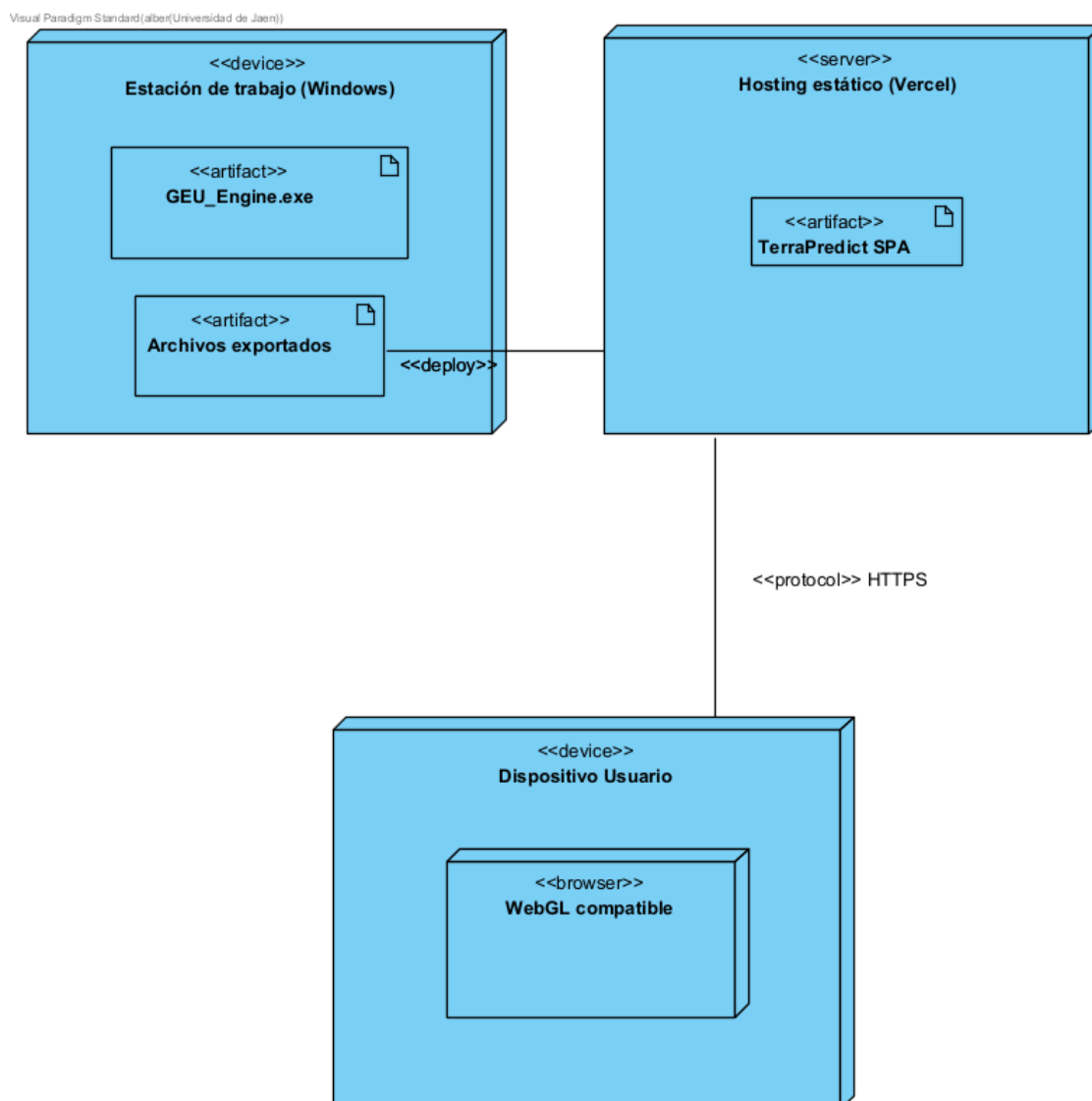


Ilustración 35 : Diagrama de despliegue (autor)

5.4 Diseño detallado

Tras definir la arquitectura de alto nivel y la asignación de responsabilidades, este apartado se define, a nivel de implementación, el detalle de las estructuras estáticas y el comportamiento dinámico del sistema con el flujo de comunicación entre componentes del mismo que esto conlleva.

5.4.1 Diseño estructural (Diagrama de clases)

El diagrama de clases representa la implementación exhaustiva orientada a objetos del procesamiento predictivo en C++. Como se observa en la Ilustración 36, el diseño se orquesta en torno a la clase *SimulationManager*, apoyándose en diferentes estructuras de datos e interfaces. Se destacan los siguientes puntos:

- **EEDD y tipados:** se ha diseñado un conjunto de enumeraciones o *enums* (*AlgorithmType*, *SSPScenario*, *FutureTimePeriod*) y algunos DTOs como *PredictionResult*, *ResponseCurveData* y *ModelMetrics*.
- **Módulo de preprocesamiento:** las clases *ClimateDataHandler*, *GeoDataLoader* y *DEMFeatureExtractor* encapsulan el proceso de la ingesta de las 19 variables bioclimáticas, los ficheros CSV de GBIF y la topografía de IGN. A esto se suma el *FeatureSelector*, por el cual se filtran las variables bioclimáticas realizando una multicolinealidad estadística (se explicará más adelante).
- **Machine Learning:** aquí es donde se aplica el comentado patrón Strategy con la interfaz *IPredictionModel*, donde aplicamos el polimorfismo de clases. De esta forma, heredan de esta interfaz *ForestPredictor* (usando el módulo de aprendizaje de OpenCV) y *MaxEntPredictor* (implementado nativamente).
- **Evaluación y métricas:** las clases *CrossValidator* (K-fold) y *MetricsCalculator* procesan las predicciones para generar la curva ROC, métricas (AUC, TSS, etc.) y comparativas.
- **Visualización y exportación:** *HeatMapGenerator* procesa la paleta de colores según la probabilidad de supervivencia de la especie y *FileManager* abstrae el guardado en archivos. Todo el sistema proporciona una interfaz gráfica de usuario mediante la clase *PrediccionEspeciesGUIAdapter*, que sigue el formato de los módulos existentes en GEU.

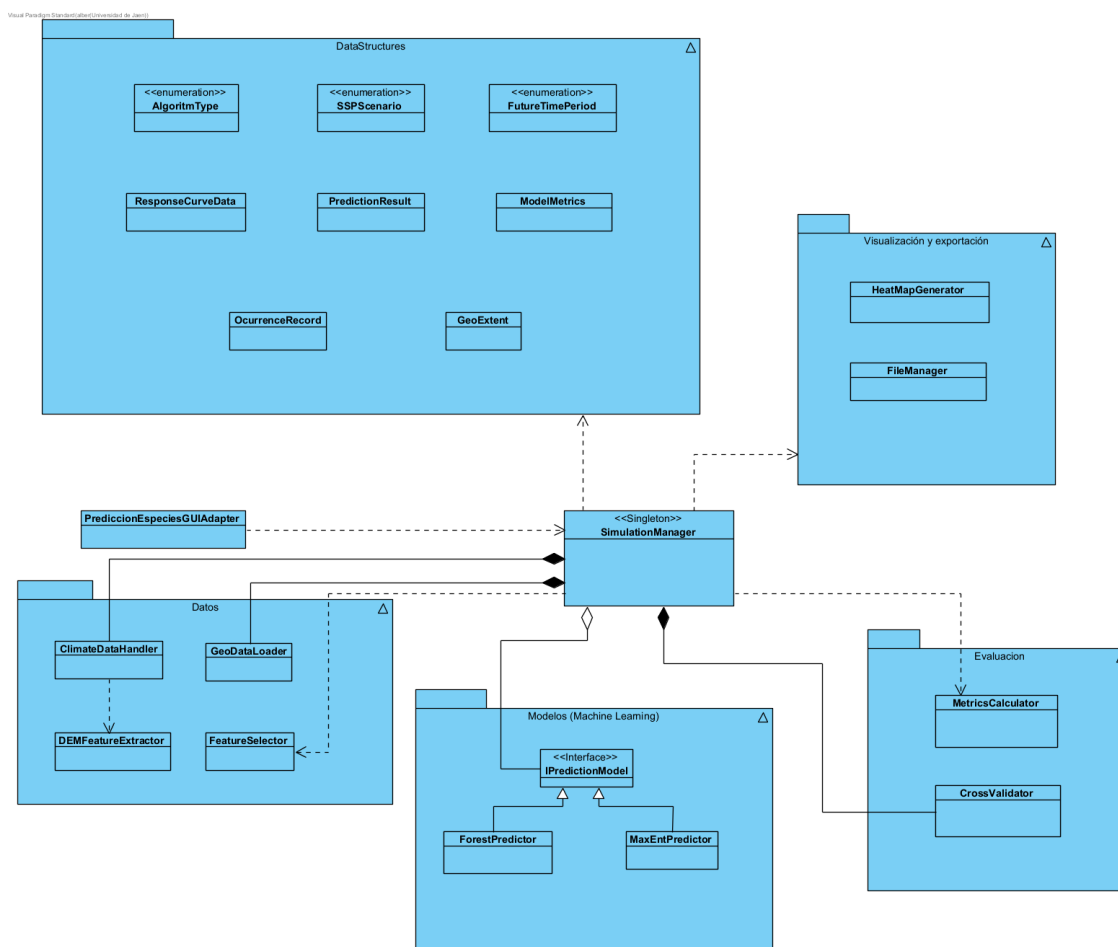


Ilustración 36 : Diagrama de clases (autor)

5.4.2 Diseño de comportamiento (Diagrama de secuencia)

Para validar la lógica temporal y el flujo de comunicación entre los distintos componentes, se ha modelado el escenario más esencial y crítico en GEU, el pipeline completo de entrenamiento y evaluación. Así, como se observa en la Ilustración 37, se detalla el intercambio de mensajes y la transformación de los datos desde su origen hasta el final de ese caso de uso. El flujo que describe el diagrama es el siguiente:

1. PrediccionEspeciesGUIAdapter captura la orden del usuario e invoca a SimulationManager
2. Este último solicita a ClimateDataHandler la creación de la matriz de entrenamiento (buildTrainingMatrix()), el cual a su vez pide a GeoDataLoader que parsee las coordenadas y genere los background points (pseudo-ausencias).
3. Se invoca a FeatureSelector::analyze() para descartar variables correlacionadas (VIF/Pearson).

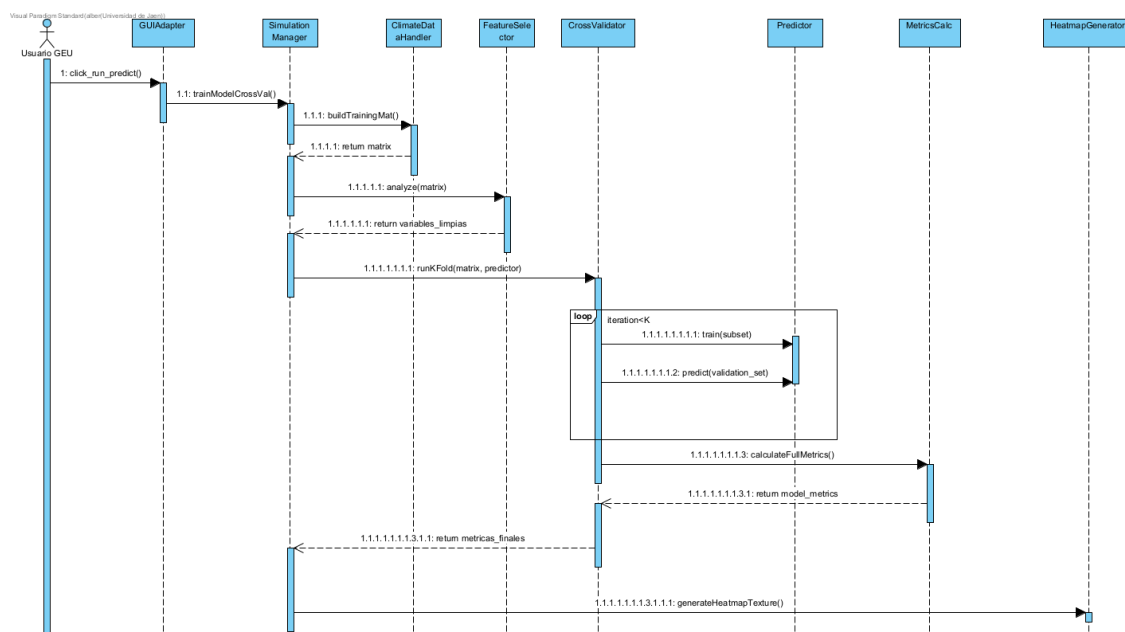


Ilustración 37 : Diagrama de secuencia: Entrenamiento y evaluación (autor)

5.4.3 Diseño del ciclo de vida (Diagrama de Máquina de Estados)

El núcleo del módulo nuevo de predicción en GEU se ha modelado mediante una Máquina de Estados Finitos (FSM) orientada a la concurrencia y la tolerancia a fallos. Como se muestra en la Ilustración 38, el estado lógico de SimulationManager viaja a diferentes estados a través de condiciones (“[]”):

- Idle: estado de reposo esperando interacción por parte del usuario.
- Loading data: carga de rasters, CSV, etc. Si GDAL reporta un puntero nulo o algún error, se va al estado de error.
- Preprocessing: limpieza espacial (*thinning*) y selección de características.
- Training/validating: ejecución de la parte de Machine Learning como tal. Se contemplan fallos por colapsos de RAM o problemas matemáticos en la regresión.
- Predicting: proyección espacial del modelo entrenado. Se incluyen comprobaciones.
- Exporting: volcado de resultados a disco por *FileManager*.
- Error state: captura excepciones, emite logs mediante *spdlog*, actualiza la UI y retorna al estado de Idle.

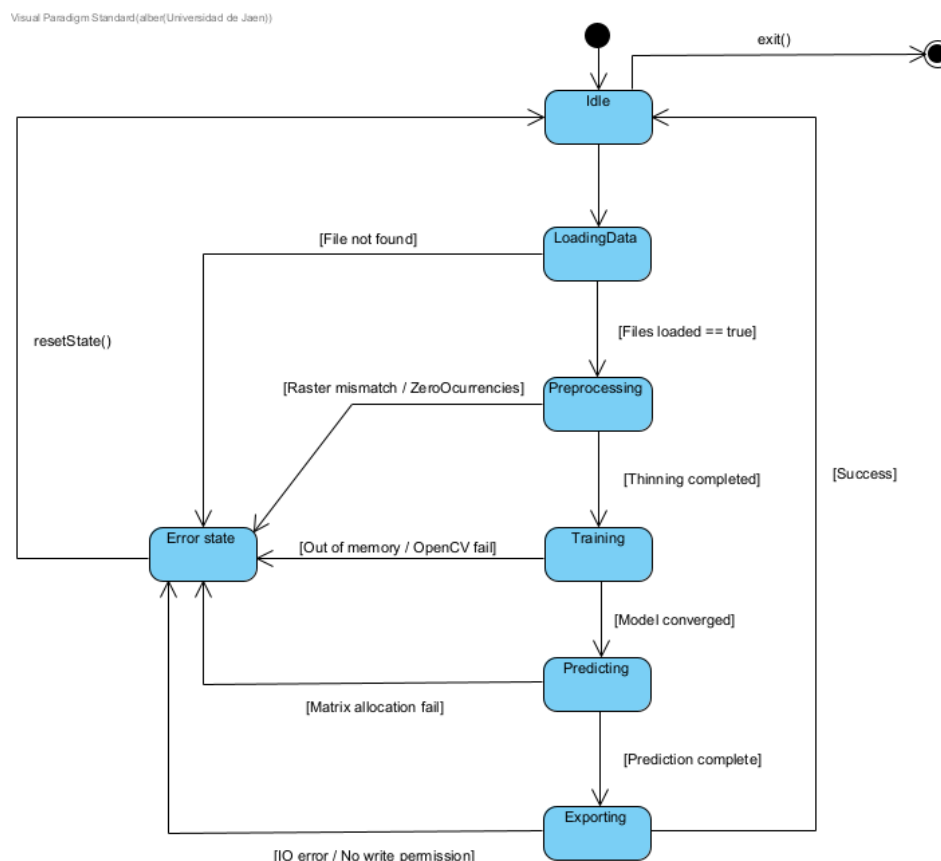


Ilustración 38 : Diagrama de máquina de estados (autor)

5.4.4 Diseño de la arquitectura web (Diagrama de componentes TerraPredict)

Como se ha comentado anteriormente, TerraPredict constituye la capa de presentación del sistema. Aunque no procesa datos directamente, su diseño debe ser claro, de forma que permita gestionar los múltiples componentes que contiene la web.

Como se observa en la Ilustración 39, se ha diseñado un diagrama de componentes detallado para entender como las distintas funcionalidades de TerraPredict se relacionan entre sí, así como la arquitectura a nivel conceptual. Se representan los siguientes elementos:

- Contenedor principal (index.html)
 - Estructura semántica de la SPA con secciones.
 - Incluye contenedores DOM sobre los que se montan los componentes JavaScript.
- Punto de entrada (main.js)
 - Maneja la inicialización de todos los componentes.
 - Gestiona el modo 2D/3D, la exportación en PNG/GLB, el menú móvil y las animaciones de scroll.

- Selector de escenario (ScenarioSelector.js)
 - Carga el catálogo dinámico.
 - Renderiza cuatro selectores (escenario, especie, etc.)
- Visor 2D (MapView.js)
 - Se usa CesiumJS.
 - Carga el GeoJSON y el PNG del heatmap.
 - Proporciona picking (con información) y selector de capa base.
- Visor 3D (Scene3D.js)
 - Escena Three.js con DEM, heatmap y nube de puntos PLY.
 - Incluye controles de órbita, picking con raycast, transiciones, etc.
- Componentes analíticos
 - Dashboards
 - Curvas de respuesta
 - Histograma de probabilidades
- Componentes comparativos
 - Mapas de diferencias
 - Comparador lado a lado
- Utilidades compartidas
 - Configuración
 - Utilidades de picking
 - Carga de DEM, nubes de punto, etc.

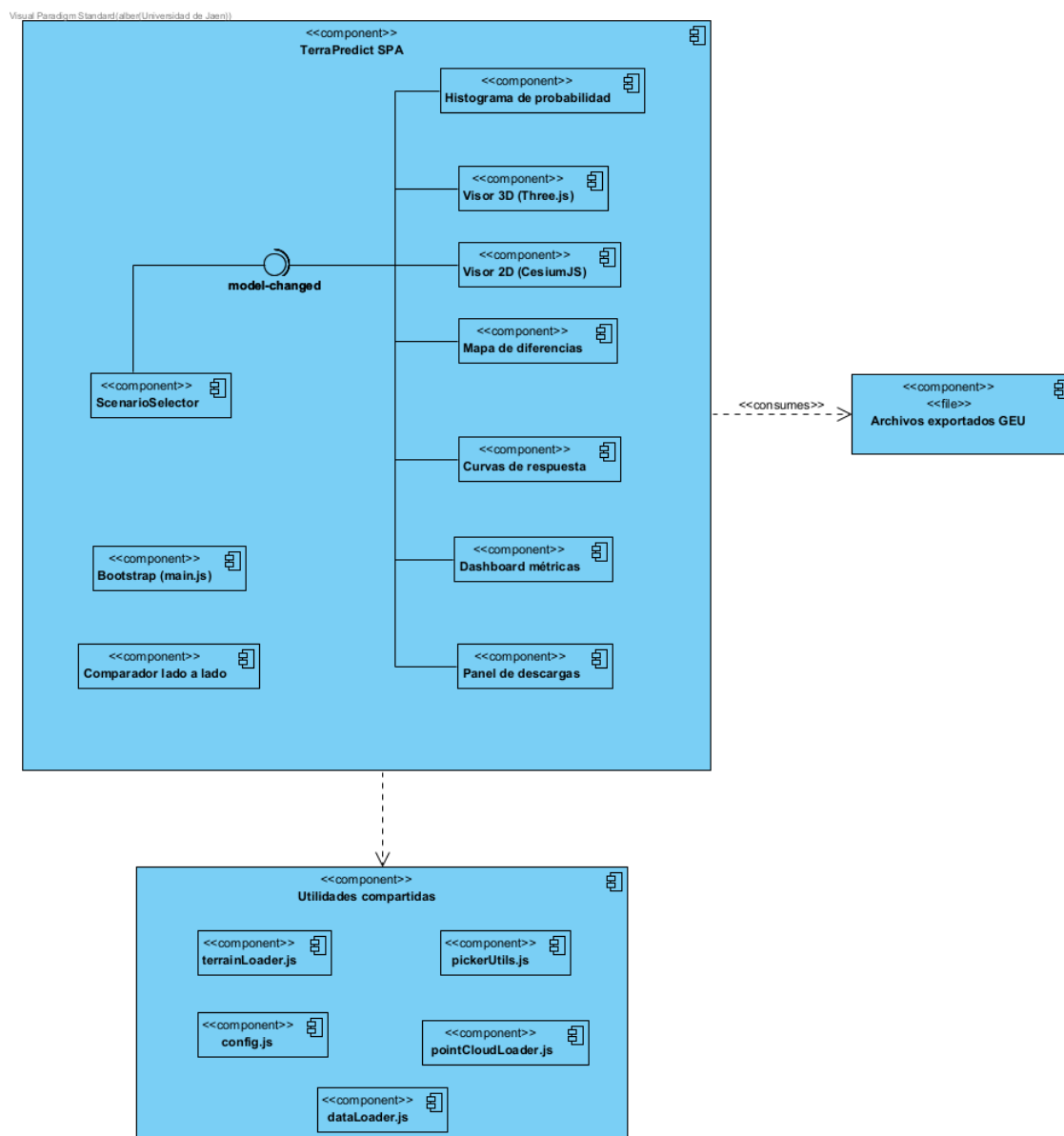


Ilustración 39 : Diagrama de componentes TerraPredict (autor)

CAPÍTULO 6

IMPLEMENTACIÓN

6. IMPLEMENTACIÓN

El presente capítulo describe la materialización técnica de la arquitectura y diseño detallado que se han expuesto en el capítulo anterior. Mientras que la fase de diseño se centró en la definición de componentes, responsabilidad, comunicación y flujo, este apartado responde a las decisiones de implementación concretas, la organización física del código, algoritmos de Machine Learning, sistema de exportación, entre otros. Se han seguido los diagramas UML definidos en el diseño y análisis, de forma que la trazabilidad y relación entre diseño e implementación sea real y tangible. Para este apartado también se seguirán los hitos/incrementos definidos con anterioridad.

6.1 Incremento I: Subsistema de datos (GeoTIFF/GDAL)

La organización física del módulo refleja la arquitectura lógica definida en el diagrama de componentes de GEU previamente definido (Ilustración 34). El código se encuentra en *GEUModules/PrediccionEspecies*, siguiendo así la convención de módulos en GEU. Como se muestra en la Ilustración 40, el directorio *Source/* se particiona en diferentes subcarpetas de forma que el código esté organizado y sea localizable fácilmente.

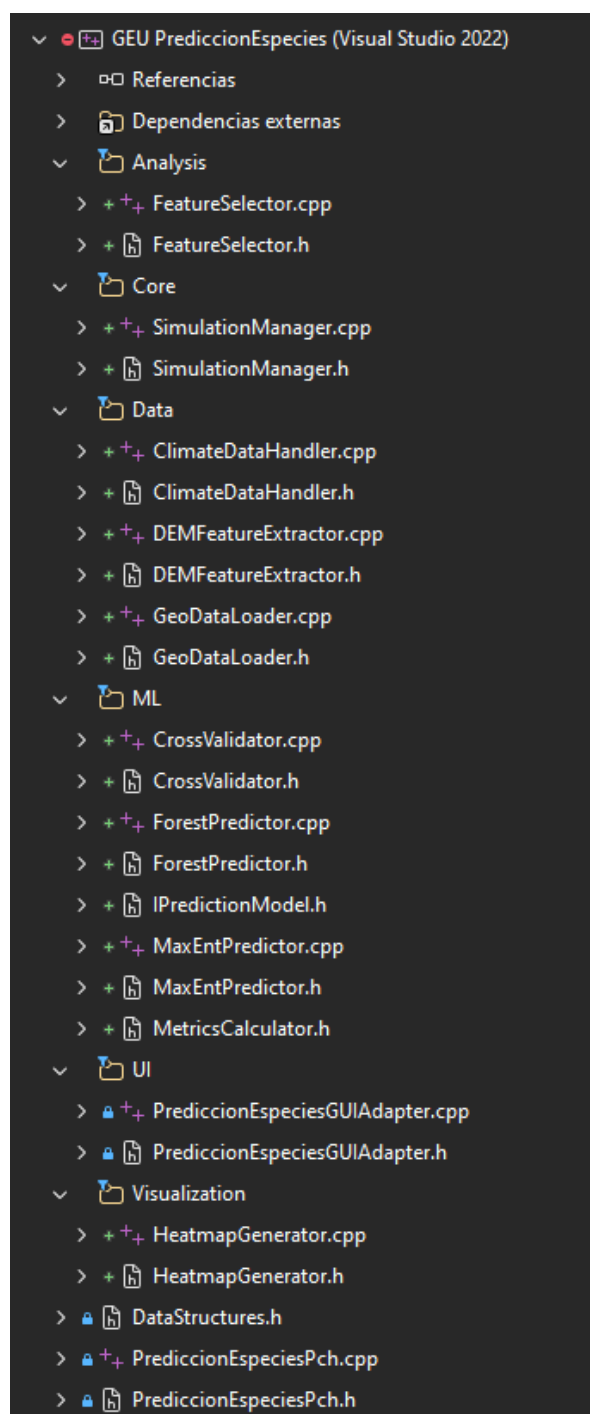


Ilustración 40 : Estructura del código en GEU (autor)

Con el fin de evitar colisiones con otros módulos de GEU, todo el código del módulo se ha encapsulado dentro de un espacio de nombres (*namespace*) anidado *GEU::SDM*. Así, la clase *SimulationManager* se referencia como *GEU::SDM::SimulationManager*, eliminando cualquier ambigüedad posible.

La cabecera *DataStructures.h* guarda toda la tipografía del módulo. Se definen enumeraciones que modelan el dominio del problema (*AlgorithmType*, *SSPScenario*, *FutureTimePeriod*), los DTOs (*PredictionResult*, *ModelMetrics*, *ResponseCurveData*) y algunos más. Con esta organización, todo se encuentra

centralizado en un solo archivo, permitiendo una mejor mantenibilidad del código.

El código respeta las convenciones establecidas en el resto del proyecto. Las más relevante son:

- **Nomenclatura del miembros:** los atributos privados utilizan guión bajo (ej. *_trainingMatrix*), mientras que los métodos públicos utilizan *camelCase* (ej. *trainModel()*). Las clases siguen la notación *PascalCase*.
- **Gestión de memoria:** se han usado contenedores estándar (*std::vector*, *std::map*, etc.). En cuanto a las matrices bidimensionales para el pipeline de ML se utiliza *cv::Mat* de *OpenCV*, evitando copias innecesarias al trabajar con ellas en diferentes pasos del proceso.
- **Sistema de logging:** todas las operaciones de I/O, entrenamiento y exportación registran mensajes mediante la librería *spdlog*, ya usada en el resto de GEU. Esta proporciona distintos niveles de “gravedad” (info, warn y error)
- **Precompiled header (PCH):** el módulo, al igual que todos los ya implementados en GEU, utiliza *PrediccionEspeciesPch.h* para agregar cabeceras de uso frecuente (*<vector>*, *<string>*, *<opencv2/opencv.hpp>*...), reduciendo el tiempo de compilación.

A continuación, en la Tabla 16, se recogen las clases y ficheros más relevantes del módulo, a modo de resumen, y también se relacionan con los componentes definidos en el diseño.

Fichero	Clase principal	Responsabilidad	Patrón Asociado
SimulationManager.cpp	SimulationManager	Gestión ciclo vida completo: carga datos, entrenamiento, predicción, cacheo y exportación	Singleton, Facade
ClimateDataHandler.cpp	ClimateDataHandler	Construye matrices entrenamiento y predicción, extrae valores puntuales en coordenadas	Information Expert
GeoDataLoader.cpp	GeoDataLoader	Encapsula GDAL para lectura GeoTIFF y CSVs	Facade
ForestPredictor.cpp	ForestPredictor	Implementación RF mediante OpenCV.	Strategy
MaxEntPredictor.cpp	MaxEntPredictor	Aproximación de MaxEnt mediante regresión logística L2	Strategy

FeatureSelector.cpp	FeatureSelector	Análisis selección variables mediante Pearson y factor VIF.	
HeatmapGenerator.cpp	HeatmapGenerator	Genera texturas 2D, nubes 3D elevadas con DEM, mapas de diferencias y cálculo área hábitat	
PrediccionEspeciesGUIAdaptor	PrediccionEspeciesGUIAdaptor	Adaptador de interfaz ImGui	Controller
DataStructures.h		Enums, DTOs, etc.	

Tabla 16 : Componentes principales módulo predicción de especies

El pipeline de predicción constituye el núcleo del módulo, definiendo así el flujo de transformaciones desde datos crudos (GeoTIFF, CSV) a resultados interpretables para un usuario no experimentado. La arquitectura está definida por un pipeline orientado a objetos, siendo posible controlar, monitorizar y estudiar cada fase del mismo de forma independiente. Esto permite tener un código mucho más modularizado y poco acoplado, permitiendo su futura extensión con más funcionalidades.

Como se observa en la Ilustración 41, el pipeline se desarrolla sobre ocho fases secuenciales, agrupadas en tres bloques lógicos (Adquisición y preprocesamiento, Entrenamiento e inferencia y Postprocesamiento y visualización). A continuación, también se detallan las fases de forma independiente.

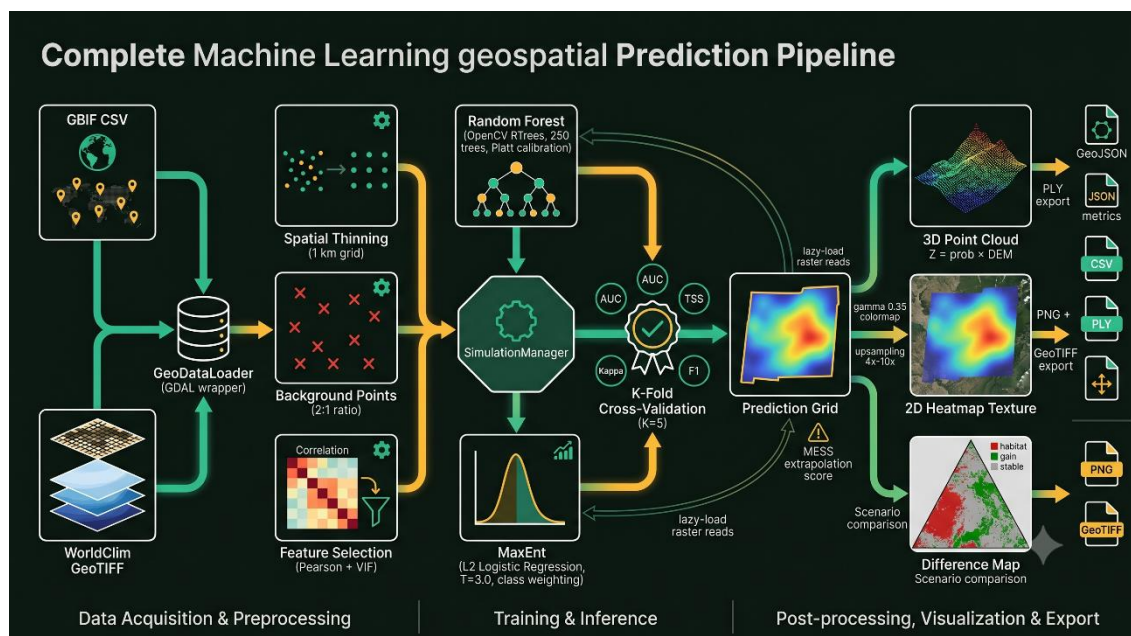


Ilustración 41 : Flujo completo de predicción (autor)

Fase 1: Ingesta de datos geoespaciales y biológicos

El punto de entrada del pipeline es la clase *SimulationManager::loadOcurrances* para datos biológicos (GBIF) y *loadCurrentClimate()/loadFutureClimate()* para variables ambientales (WorldClim). Estas operaciones delegan en *GeoDataLoader* que actúa como encapsulador sobre las funcionalidades de GDAL.

Para las ocurrencias de especies, *GeoDataLoader::loadCSV()* (Anexo I, Código 1) implementa un parser teniendo en cuenta el formato específico de GBIF. El método itera línea a línea del CSV, soportando distintos tipos de separadores (tab, punto, coma, etc.) para obtener la información individual de cada ocurrencia. Los resultados se almacenan en un vector de estructuras *OcurrenceRecord* (species, longitude, latitude), que posteriormente pasarán a ser una matriz *cv::Mat*.

En cuanto a las variables bioclimáticas, se complica un poco más, sobre todo cuando tratamos con formato GeoTIFF multibanda. El método *GeoDataLoader::loadMultibandGeoTIFF()* (Anexo I, Código 2) usa internamente *GDALDataset::GetRasterBand()* para acceder a cada banda individualmente y *RasterIO()* para extraer valores puntuales.

El área de estudio se deduce en el sistema a partir de los datos que se carga:

- **Ocurrencias de GBIF:** calcula el bounding box de todos los puntos de presencia: minLat, maxLat, minLon, maxLon.
- **Rásteres de WorldClim:** cada archivo GeoTIFF aporta su propio extent (geoTransform). El sistema utiliza la intersección de todos los rústeres cargados.
- **DEM:** cuando se carga un DEM, se restringe el área a dicho archivo (*_clipToDEMExtent*).
- **Margen de predicción:** para evitar que el modelo prediga solo dentro de la caja mínima, se amplía el extent 1.5 grados.

Fase 2: Preprocesamiento espacial y selección de características

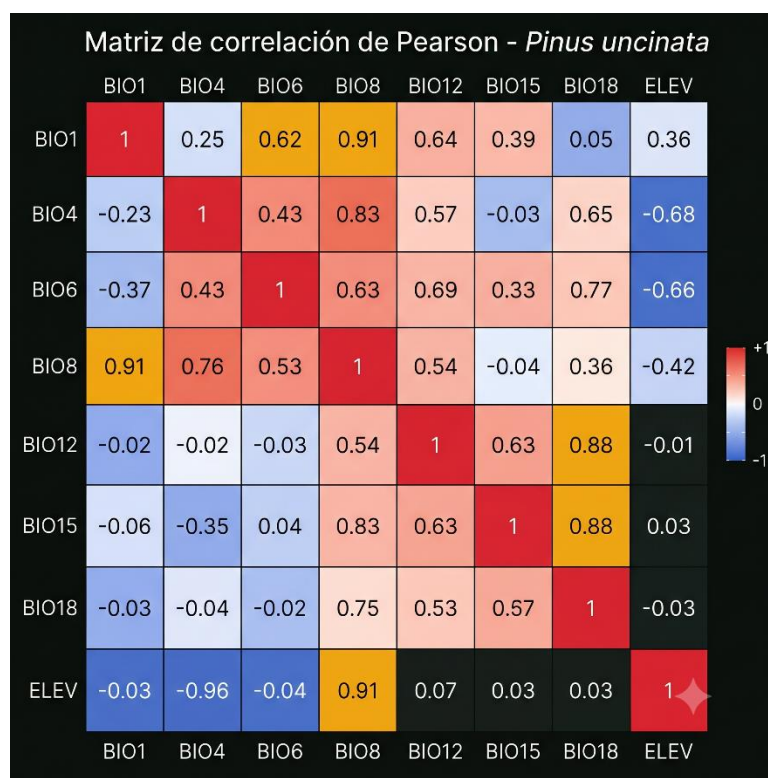
Una vez los datos están disponibles en memoria, el pipeline entra en la fase de preprocesamiento, manejada por *ClimateDataHandler::buildTrainingMatrix()* (Anexo I, Código 3). Esta fase consta de tres etapas:

- a) Thinning espacial [65] (Anexo I, Código 5): para mitigar el sesgo de autocorrelación espacial inherente a los datos de presencia, se implementa un algoritmo de *spatial thinning* con resolución configurable (1 km por defecto). Lo que se hace es discretizar el espacio geográfico en celdas cuadradas y conserva un único punto por celda.

- b) Generación de pseudo-ausencias (background points) (Anexo I, Código 4): dado que los datos de GBIF son de presencias de la especie, el modelo requiere ejemplos opuestos para entrenar. El sistema genera puntos de fondo mediante muestreo aleatorio dentro del *extent* del área de estudio con dos restricciones: no deben caer en celdas ya ocupadas por presencia, y se mantiene una proporción configurable respecto al número de presencias tras el *thinning*.
- c) Selección de variables (Anexo I, Código 7): para evitar la multicolinealidad de las 19 variables, el lugar realiza un análisis. Primero:
- a. Calcula la matriz de correlación de Pearson y descarta aquellas variables con correlación superior a un umbral (por defecto 0.85). La correlación de Pearson mide la relación lineal entre dos variables. Si dos de ellas están muy correlacionadas, aportan información redundante y el modelo podría sobreajustarse. Su fórmula matemática es:

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

Si $|r_{xy}| > 0.85$, se elimina una de las dos variables. Además, en la Ilustración 42, se observa un ejemplo de cómo podría ser la matriz de Pearson resultante.



- b. Después, se aplica un Factor de Inflación de la Varianza (VIF) (Anexo I, Código 8) mediante descomposición SVD sobre las variables que queden del paso anterior. Se eliminan las que queden por debajo del umbral. El resultado es un subconjunto de índices que se utiliza tanto para entrenamiento como para la predicción. El VIF mide cuánto aumenta la varianza de un coeficiente de regresión debido a la colinealidad con otras variables. Un VIF alto indica que una variable puede predecirse a partir de las demás. Su fórmula sería:

$$VIF_j = \frac{1}{1 - R_j^2}$$

donde R_j^2 es el coeficiente de determinación de la variable de la regresión de la variable j respecto al resto. Si $VIF > 10$, hay una multicolinealidad problemática.

- c. La descomposición SVD (Descomposición de valores singulares) es un método para calcular el VIF en lugar de usar una regresión lineal simple para ser más robusta. Su fórmula es:

$$X = U\Sigma V^T$$

Donde U y V son matrices ortogonales y Σ es una matriz diagonal con los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p$. El VIF se aproxima a partir de la inversa de los valores singulares cuadrados.

6.2 Incremento II: Subsistema de IA (OpenCV – RF/MaxEnt)

Continuando con las fases del pipeline, en referencia al motor de inferencia de IA con algoritmos de Machine Learning, tenemos diferentes fases interesantes que se definen a continuación. Después, se continuará con una explicación profunda sobre la implementación de los dos algoritmos de Machine Learning utilizados para el proyecto.

Fase 3: Entrenamiento del modelo predictivo

Con la matriz de características y el vector de etiquetas contruidos, el pipeline pasa a la fase de entrenamiento. Se elige un algoritmo dependiendo de lo que elija el usuario (*ForestPredictor* o *MaxEntPredictor*).

- Random Forest: la clase *ForestPredictor* encapsula a *cv::ml::RTrees* de OpenCV. Los hiperparámetros del algoritmo se configuran mediante el struct *RandomForestConfig*: número de árboles (250), profundidad máxima (7) y umbral mínimo de muestras por nodo (5). Tras el entrenamiento, se aplica calibración

de Platt mediante regresión logística para transformar las probabilidades al rango [0,1].

- MaxEnt (Máxima Entropía): se implementa una aproximación nativa basada en regresión logística con regularización L2 y características cuadráticas. La clase *MaxEntPredictor* utiliza descenso de gradiente estocástico con *class weighting* (peso 10:1 a favor de presencia) y temperatura de softmax $T = 3.0$ para suavizar la distribución de probabilidad.

Fase 4: Validación cruzada y métricas de rendimiento

Antes de proceder a la predicción, el sistema ejecuta una validación cruzada estratificada *K-Fold* ($K = 5$) mediante *CrossValidator::run()* (Anexo I, Código 16). Se garantiza así que la proporción de presencias/ausencias se mantenga constante en cada *fold*, evitando sesgos en métricas como AUC o TSS. Estas métricas se evaluarán en el capítulo de resultados. Sin embargo, forman parte del flujo, por lo que cabe mencionar que se almacenan en *ModelMetrics* y se propagan tanto a la interfaz para mostrarlas al usuario como a los archivos de exportación JSON.

La validación cruzada K-Fold es una técnica para evaluar un modelo sin necesidad de un conjunto de test independiente. El conjunto de datos se reparte en K partes (o *folds*). El modelo se entrena K veces: en cada iteración se usa $K - 1$ *folds* para entrenar y un *fold* para evaluar. Al final se promedian las métricas. La versión estratificada se diferencia en que a cada *fold* se garantiza la proporción original de presencias/ausencias, lo cual es muy importante en SDM porque las presencias son mucho menores que las ausencias.

Fase 5: Predicción espacial y ensemble

La predicción se ejecuta con *SimulationManager::runPrediction()*. Se comienza construyendo un grid regular de coordenadas geográficas que cubre el *extent* del área de estudio. Para cada celda del grid, *ClimateDataHandler::extractFeaturesAtPoint()* extrae los valores bioclimáticos interpolados del ráster y, si está activado el DEM, los factores topográficos mediante el algoritmo de Horn 3x3 implementado en *DEMFeatureExtractor*.

El algoritmo de Horn es un método para calcular la pendiente y orientación (aspecto) a partir de un DEM. Utiliza una ventana de 3x3 píxeles alrededor de un punto de interés, ponderando los píxeles laterales el doble que las esquinas.

El predictor aplica entonces el modelo entrenado a la matriz de características del grid. El sistema promedia las probabilidades de múltiples modelos entrenados con diferentes semillas y *jittering* (Anexo I, Código 6) geográfico (+500 m). El resultado se guarda en un objeto *PredictionResult*, que contiene: probabilidades por celda (para el heatmap posterior), desviación estándar de las múltiples ejecuciones, puntuaciones MESS (*Multivariate Enviromental Similarity Surfaces*) para detectar extrapolaciones y el *bouding box* geográfico.

Hay dos conceptos mencionados que cabe explicar más a fondo:

- **Jittering geográfico:** es una técnica de aumento de datos que desplaza ligeramente las coordenadas de las presencias antes de entrenar cada modelo del emsamble. Así cada modelo de este ve una versión ligeramente diferente de los datos, aumentando la robustez del modelo.
- **Puntuación MESS:** detecta cuándo una celda del grid futuro tiene valores de variables que caen fuera del rango observado en el entrenamiento. Sirve para identificar zonas de extrapolación climática.

6.2.1 Algoritmo Random Forest

El Random Forest es uno de los algoritmos de ensemble learning más consolidados en el ámbito de los SDMs [66]. Su robustez frente al sobreajuste, su capacidad para capturar relaciones no lineales entre variables bioclimáticas y su naturaleza probabilística entre dichas variables lo convierten en una elección trivial para implementar el módulo de predicción de especies en GEU. A diferencia de otros enfoques, este algoritmo ofrece un equilibrio entre precisión predictiva e interpretabilidad, ya que permite cuantificar la importancia de cada variable analizada en el modelo.

En el contexto de este TFG, el algoritmo se ha implementado mediante la clase `ForestPredictor`, que hereda de `IPredictionModel` y encapsula a la librería `cv::ml::RTrees` de OpenCV. Su módulo de Machine Learning es muy útil ya que incluye muchas funcionalidades interesantes para este caso y además ya se utilizaba en el stack base de GEU, por lo que era la opción predilecta (frente a otras como `mlpack`).

Fundamento teórico y funcionamiento

Random Forest pertenece a la familia de métodos de ensemble basados en árboles de decisión. El algoritmo se basa en construir un conjunto de árboles independientes entrenados sobre submuestras del conjunto de datos original (*bagging*) y, en cada nodo de división, considera un subconjunto de variables candidatas. La predicción final se obtiene promediando las salidas de todos los árboles, lo que reduce la varianza respecto a utilizar un único árbol.

Si profundizamos en el algoritmo, dado un conjunto de entrenamiento $D = \{(x_i, y_i)\}_{i=1}^n$ donde $x_i \in R^m$ representa el vector de m variables bioclimáticas y $y_i \in \{0,1\}$ indica la ausencia o presencia de la especie, el bosque se compone de B árboles $\{T_b\}_{b=1}^B$. Para una muestra de consulta x , la probabilidad de presencia se estima como:

$$\hat{p}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

donde $T_b(x) \in \{0,1\}$ es la predicción del árbol b -ésimo. En la implementación de OpenCV [67], la fracción de votos se obtiene directamente mediante el método `RTrees::getVotes()`, que devuelve la matriz de votos por clase de cada muestra.

Como se observa en la Ilustración 43, el proceso de inferencia en el módulo puede resumirse en cuatro etapas visuales: (1) ingesta de la matriz de características bioclimáticas, (2) construcción del bosque mediante bagging y selección aleatoria de variables, (3) agregación de las fracciones de votos de los árboles, y (4) transformación opcional mediante calibración de Platt para obtener las probabilidades calibradas.

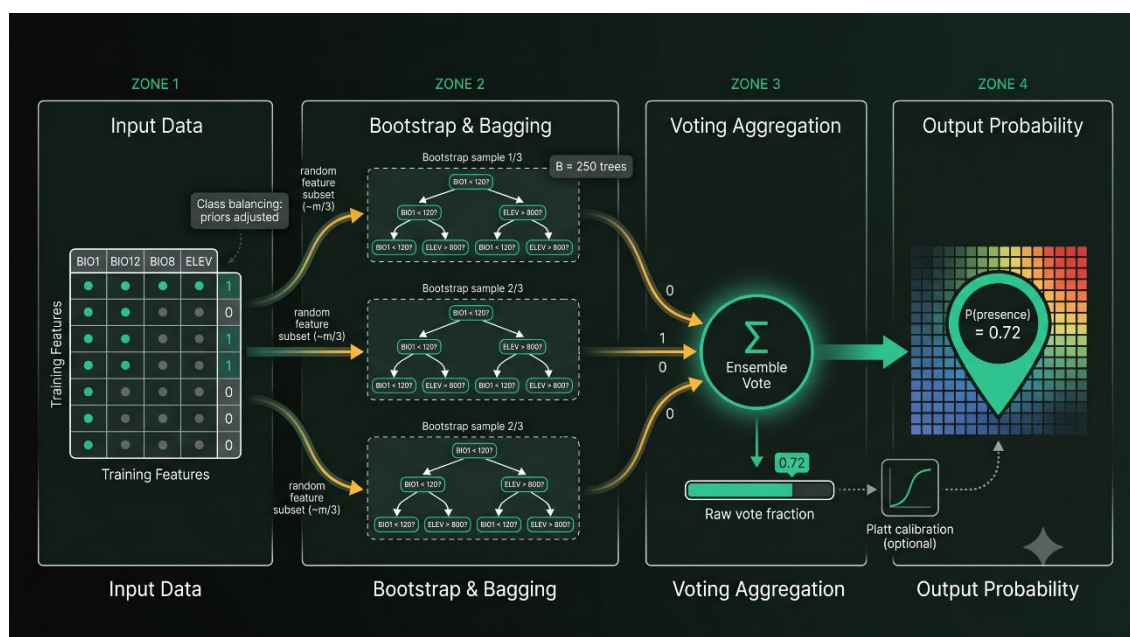


Ilustración 43 : Flujo de ejecución Random Forest (autor)

Datos de entrada y requisitos

Para que `ForestPredictor::train()` opere correctamente, el sistema debe proporcionarle dos matrices `cv::Mat`:

1. Matriz de características (features): de dimensiones $N \times m$, donde N es el número de muestras (presencias + pseudoausencias) y m es el número de variables bioclimáticas seleccionadas. Cada fila corresponde a un punto geográfico y cada columna a una variable. Los valores deben estar en escala continua (temperaturas en $^{\circ}\text{C} \times 10$, precipitaciones en mm, etc.).
2. Vector de etiquetas (labels): de dimensión $N \times 1$, con valores enteros 0 (ausencia/background point) o 1 (presencia). OpenCV espera etiquetas de tipo `CV_32S` o `CV_32F`.

Adicionalmente, el predictor requiere un vector de nombres de características (`_featureNames`) que se utiliza tanto para generar las etiquetas

del gráfico de importancia de variables como para validar la coherencia dimensional durante la predicción espacial. Si el número de columnas de la matriz de predicción no coincide con el de entrenamiento, el método *predict()* registrar un error crítico y devuelve un vector vacío, evitando así predicciones incorrectas.

Implementación en *ForestPredictor*

La clase *ForestPredictor* (*Source/ML/*) encapsula toda la lógica específica del algoritmo que se ha descrito. Su interfaz pública (heredada de *IPredictionModel*) consta de cuatro operaciones principales:

- *train(features, labels)* (Anexo I, Código 9): construye el bosque a partir de la matriz de entrenamiento.
- *predict(features)* (Anexo I, Código 10): devuelve el vector de probabilidades para un conjunto de muestras.
- *getVariableImportance()* (Anexo I, Código 11): extrae la importancia de cada variable según la reducción media de impureza Gini [67], [68].
- *saveModel(path)* / *loadModel(path)*: serializa y deserializa el modelo entrenado en formato XML de OpenCV.

El método *train()* comienza configurando los hiperparámetros del objeto *cv::Ptr<cv::ml::RTrees>*. OpenCV inicializa la semilla aleatoria interna de forma no determinista en cada ejecución, lo que introduce variabilidad entre entrenamientos. Esta característica resulta ventajosa, ya que permite entrenar multiples predictores con distintas semillas y promediar sus salidas para utilizar la media de los resultados y ser más precisos.

Un aspecto crítico de la implementación es el balanceo de clases. En los datos de SDM, la clase de presencia ($y=1$) suele estar muy subrepresentada respecto a las pseudoausencias ($y=0$). Si el bosque optimizara directamente la tasa de acierto global, tendería a predecir siempre ausencia. Para corregir este sesgo, el método *train()* asigna priores inversamente proporcionales a la frecuencia de cada clase mediante *_forest->setPriors()*. Concretamente, si el conjunto de entrenamiento contiene N_1 presencias y N_0 ausencias, los pesos asignados son $w_1 = N_0 / N$ y $w_0 = N_1 / N$, de forma que el error de clasificar incorrectamente una presencia penaliza tanto como el de clasificar incorrectamente una ausencia.

Hiperparámetros y justificación

Los hiperparámetros por defecto de *ForestPredictor* han sido ajustados empíricamente (mediante muchas ejecuciones con diferentes datasets y variables bioclimáticas seleccionadas tras la autocorrelación. En concreto, la Tabla 17 resumen la configuración final asignada para el modelo.

Hiperparámetro	Valor por defecto	Justificación
----------------	-------------------	---------------

Número de árboles (<code>_numTrees</code>)	250	Proporciona suficiente resolución en la fracción de votos para discriminar entre escenarios futuros sin penalizar excesivamente el tiempo de entrenamiento
Profundidad máxima (<code>_maxDepth</code>)	7	Regularización intermedia. Evita que el modelo memorice ruido en datasets pequeños, pero permite capturar interacciones no lineales entre variables
Muestras mínimas por hoja (<code>_minSamples</code>)	10	Suaviza las fracciones de votos y evita predicciones saturadas (0 o 1) al extrapolar a escenarios climáticos futuros
Variables activas por división (<code>ActiveVarCount</code>)	$\max(1, m/3)$	Estándar de Random Forest. Garantiza diversidad entre árboles y reduce la correlación entre predictores del ensemble.
Criterio de parada	<code>MAX_ITER=_numTrees</code>	Fuerza la construcción exacta del número de árboles especificado.

Tabla 17 : Hiperparámetros Random Forest

La combinación de la configuración de profundidad y el número de muestras por hoja, constituye el núcleo de la estrategia de regularización. Durante las pruebas que se realizaron con el dataset sobre la zona de Sierra Mágina con la especie *Pinus uncinata*, se observó que profundidades superiores a 10 producían predicciones con valores extremos (0.0 o 1.0) en las celdas de extrapolación climática, comprimiendo las diferencias entre escenarios SSP. Profundidades inferiores a 5, producían mapas excesivamente suavizados que perdían detalle espacial del nicho ecológica de la especie.

Para entender el proceso de forma más coloquial, fijémonos en un ejemplo concreto del caso de estudio. Si realizamos la predicción sobre Sierra Mágina para la especie *Pinus uncinata*: el conjunto de entrenamiento contiene 87 presencias (tras el *thinning* espacial) y 174 puntos de fondo generados aleatoriamente. Las variables más frecuentemente escogidas por los árboles son BIO4 (estacionalidad de la temperatura), BIO12 (precipitación anual) y BIO6 (temperatura mínima del mes más frío).

Un árbol concreto del bosque podría tomar decisiones como estas:

- ¿BIO6 < -5°C? -> probablemente no sobreviva.
- ¿BIO12 > 800 mm? -> sí, continúa.
- ¿BIO4 > 60? -> no, el clima es demasiado variable.

Como conclusión, cada árbol aprende reglas similares, pero ligeramente diferentes dada la naturaleza estocástica del algoritmo. Al promediar 250 de estas opiniones, el modelo suaviza los errores individuales y genera un mapa de probabilidad continuo.

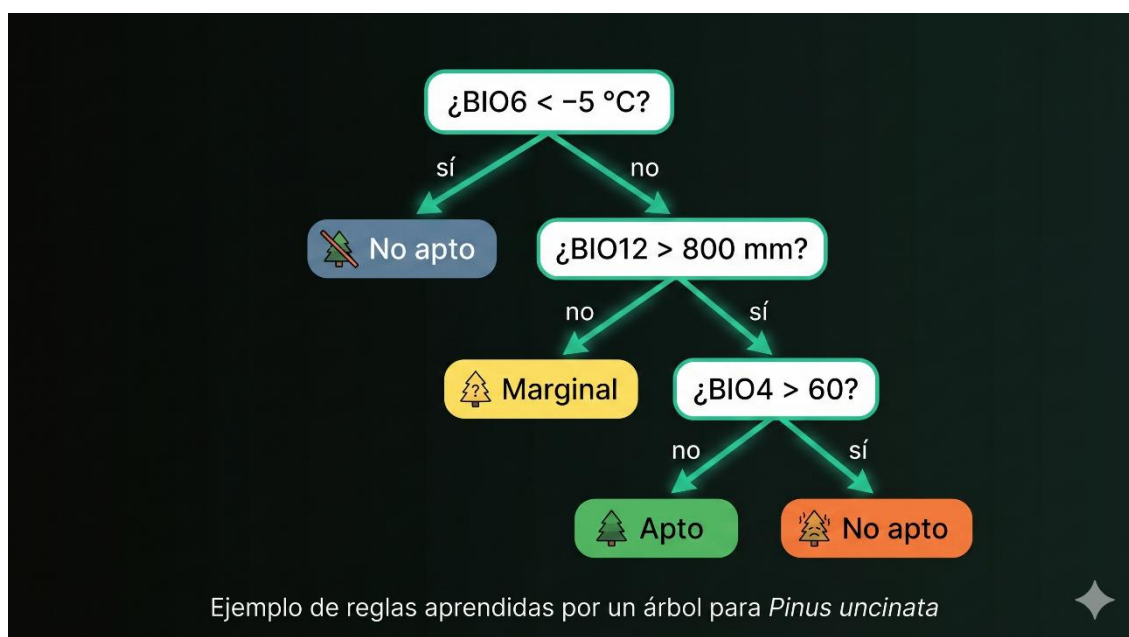


Ilustración 44 : Ejemplo árbol de decisión Random Forest (autor)

Calibración de Platt [69]

Por defecto, OpenCV devuelve la fracción de votos del bosque como estimación de probabilidad. Si bien esta fracción es numéricamente correcta (muestras con mayor fracción de votos indican mayor probabilidad de albergar la especie), no está necesariamente bien calibrada en el sentido probabilístico estricto: una fracción de 0.8 no garantiza que el 80% de muestras con esa puntuación sean realmente presencias.

Para resolver este problema, *ForestPredictor* implementa calibración de Platt, que transforma la fracción de votos r en una probabilidad calibrada p mediante una sigmoide logística:

$$p = \frac{1}{1 + e^{A \cdot r + B}}$$

Los parámetros A y B se estiman minimizando la *log-loss* sobre un conjunto de validación reservado durante el entrenamiento. El procedimiento utiliza gradiente descendiente durante 100 iteraciones con una tasa de aprendizaje de 0.1, acotando los valores de probabilidad mediante una ϵ de 10^{-9} para evitar inestabilidades numéricas en la función logarítmica.

No obstante, la calibración de Platt se encuentra desactivada por defecto (`_usePlattCalibration = false`). Durante la fase de validación del módulo se observó que, en el contexto de la comparación de escenarios futuros, la sigmoide comprimía las diferencias relativas entre el clima actual y los escenarios SSP.

Es decir, dos escenarios con fracciones de votos 0.4 y 0.6 podían calibrarse a probabilidades 0.48 y 0.52, dificultando la detección visual de cambios de hábitat. Por esta razón, el sistema utiliza las fracciones de votos crudos como probabilidad por defecto, aunque se mantiene la opción de forma configurable por el usuario en caso de que lo necesite.

Importancia de variables [67]

Una de las ventajas diferenciales de Random Forest es su capacidad para cuantificar la importancia de cada variable bioclimática. *ForestPredictor::getVariableImportance()* extrae la matriz de importancias calculada por OpenCV durante el entrenamiento y la normaliza al rango [0,1].

La importancia se calcula como la reducción media de impureza Gini atribuible a cada variable a lo largo de todos los nodos de división del bosque. Formalmente, para una variable bioclimática j :

$$\text{Importancia}(j) = \frac{1}{B} \sum_{b=1}^B \sum_{t \in T_b^j} p(t) \cdot \Delta i(t, j)$$

donde T_b^j es el conjunto de nodos del árbol b que utilizan la variable j , $p(t)$ es la proporción de muestras que llegan al nodo t , y $\Delta i(t, j)$ es la reducción de impureza al dividir por la variable j . Esta información se muestra en la interfaz de usuario mediante una barra de progreso horizontal y se exporta junto con las métricas del modelo en formato JSON.

Como se ve en la Ilustración 45, se muestra el resultado de calcular la importancia de las variables tras la ejecución del algoritmo Random Forest. Así, podemos analizar el peso o importancia que tiene cada variable sobre una especie concreta, además de mostrar el mapa de probabilidades.

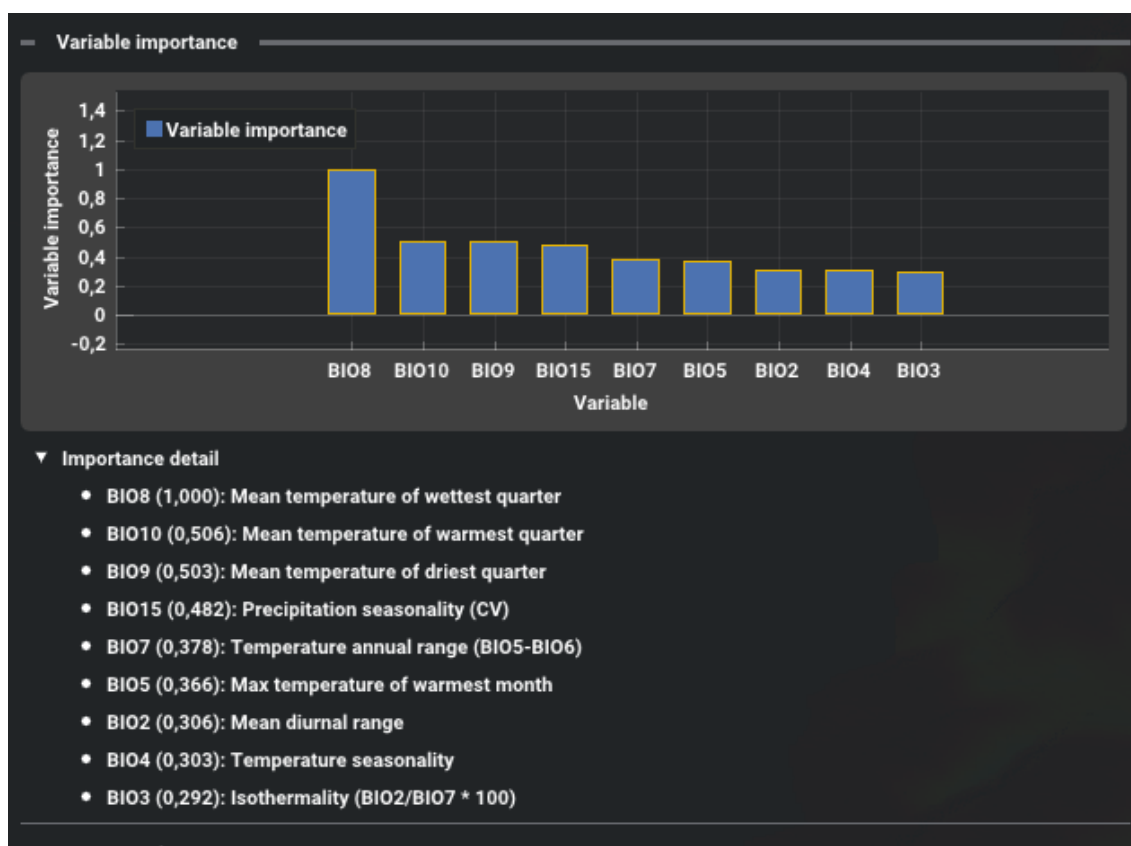


Ilustración 45 : Gráfico de importancia de variables en GEU (autor)

En resumen, para aclarar los conceptos técnicos y hablar de una forma más coloquial, Random Forest trata de realizar un proceso de votos, con una cantidad de expertos en la materia (árboles de decisión), y cada uno vota si o no a que una especie viva en una zona concreta o no (cada punto geográfico que estamos analizando). Cuando se termine la votación, la probabilidad es, básicamente, los votos que han salido positivos entre los votos totales. En la Ilustración 46, se aprecia un diagrama de flujo con todos los pasos por los que pasa el proceso de predicción de especies para el algoritmo Random Forest y algunos pasos previos para contextualizar el algoritmo dentro del proceso de simulación completo.

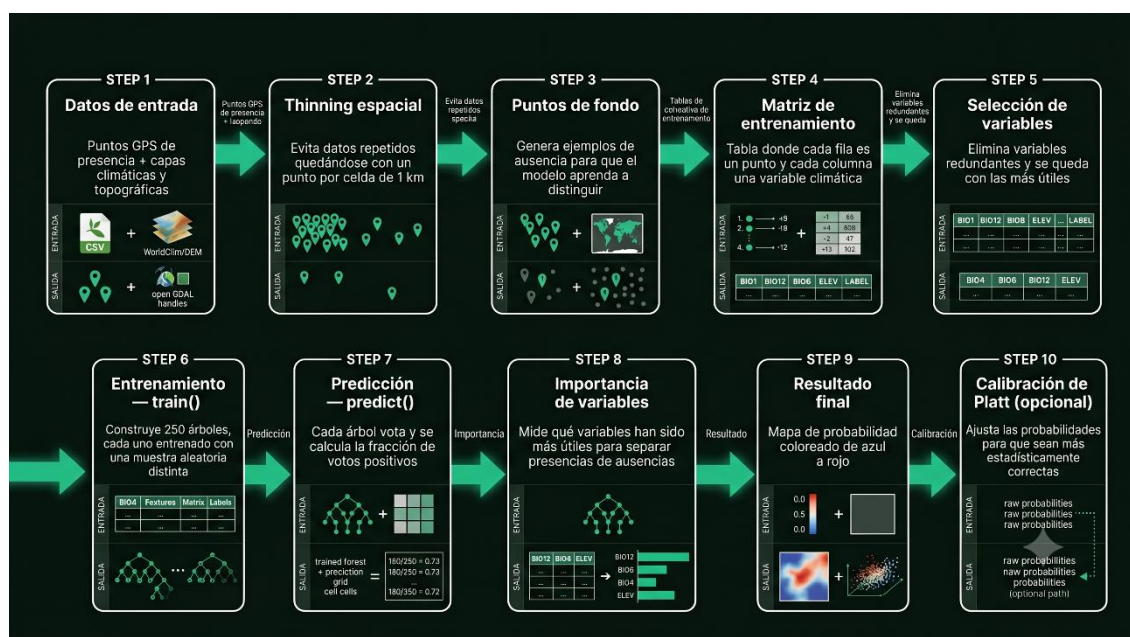


Ilustración 46 : Flujo contextualizado Random Forest (autor)

6.2.2 Algoritmo MaxEnt (Máxima Entropía)

El algoritmo MaxEnt (Maximum Entropy) fue desarrollado por Steven J. Phillips y colaboradores como una de las herramientas más utilizadas en Modelos de Distribución de Especies [70]. Su filosofía es diferente a la del Random Forest. En lugar de aprender reglas, MaxEnt modela la distribución de probabilidad de presencia que maximiza la entropía (es decir, la más neutral posible) teniendo en cuenta las restricciones dadas por las variables bioclimáticas.

El algoritmo original fue implementado en Java, así que para integrarlo en GEU sin utilizar una JVM, se ha realizado una aproximación nativa en MaxEntPredictor basada en una regresión logística con regularización L2 y características cuadráticas.

Fundamento teórico y funcionamiento

La idea de MaxEnt se puede entender con un ejemplo: supongamos que queremos repartir una cantidad fija de probabilidad por todo el territorio en la zona de estudio. Sabemos dónde ha aparecido la especie, así que esas zonas deberían tener más probabilidad de por sí. Sin embargo, en primera instancia asumimos que el territorio es uniforme. Así, MaxEnt trata de encontrar el reparto de probabilidad más justo respetando las restricciones observadas por los datos climáticos y de presencia.

Matemáticamente, el algoritmo busca una distribución de probabilidad $P(x)$ que:

1. Asigne probabilidad alta a los puntos donde se ha observado la especie
2. Maximice la entropía (la incertidumbre) en el resto del territorio.

3. Se ajuste a las restricciones de las variable bioclimáticas.

En la implementación del módulo, esta idea se aproxima mediante una regresión logística regularizada. Para cada punto geográfico x con variables $f(x)$, la probabilidad de presencia se modela como:

$$P(y = 1 | x) = \frac{1}{1 + e^{-(w \cdot f(x) + b)}}$$

donde w es el vector de pesos aprendido y b es el sesgo. Durante el entrenamiento, el algoritmo ajusta w y b minimizando una función de pérdida que penaliza dos cosas: los errores de clasificación y la magnitud de los pesos (regularización L2). La penalización a los pesos evita que el modelo se ajuste demasiado a los pocos puntos de presencia disponibles.

Como se observa en la Ilustración 47, el proceso de inferencia de este algoritmo se resume en cuatro etapas: (1) transformación de las variables ambientales en características lineales y cuadráticas, (2) ajuste de los pesos mediante descenso de gradiente, (3) aplicación de la función logística para obtener probabilidades, y (4) suavizado opcional con temperatura de *softmax* para evitar predicciones demasiado extremas.

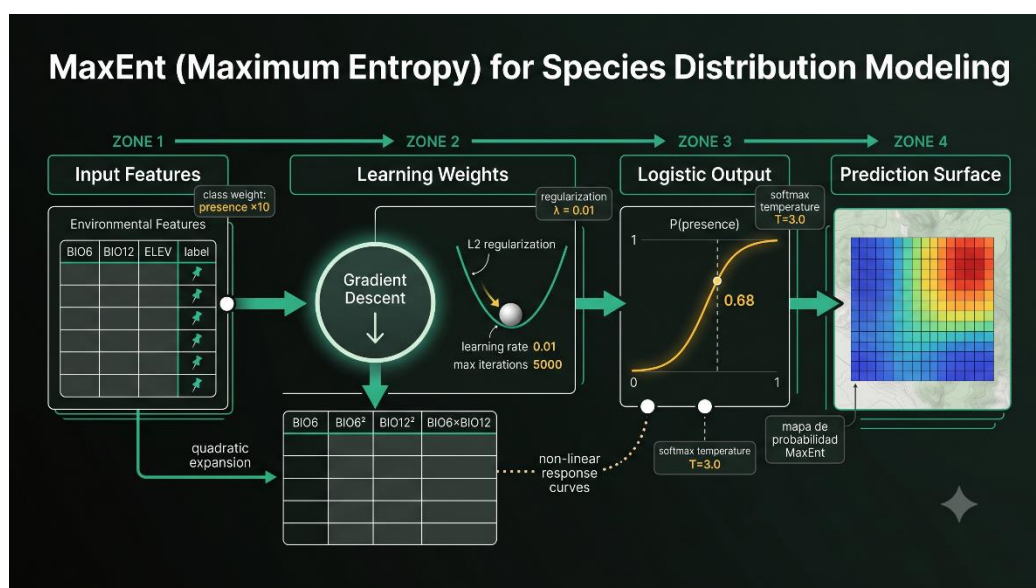


Ilustración 47 : Flujo de predicción MaxEnt (autor)

Implementación en *MaxEntPredictor*

La clase *MaxEntPredictor* (*Source/ML/*) encapsula toda la lógica del algoritmo. Su interfaz es la misma que *ForestPredictor*, garantizando su uso indistintamente desde *SimulationManager*.

Las estructuras de datos utilizadas principalmente son:

- *cv::Mat features* (entrada de *train()* y *predict()* (Anexo I, Código 14)): matriz de OpenCV que contiene las variables bioclimáticas.

Es igual que en Random Forest: las filas son puntos geográficos y las columnas las variables.

- cv::Mat labels (entrada de train()): vector columna con las etiquetas 0 (ausencia) o 1 (presencia). MaxEnt lo utiliza para saber qué filas de *features* corresponden a presencias reales.
- std::vector<float> weights (aprendida durante train()): lista de pesos $w_1, w_2, \dots, w_m$ más el sesgo b . Es el conocimiento del modelo. Cada peso indica cuánto influye una variable en la predicción final. Un peso positivo grande significa que, al aumentar esa variable, aumenta también la probabilidad. Un peso negativo, al contrario.
- std::vector<float> de salida de predict(): vector de probabilidades, uno por celda del *grid*. Posteriormente se encapsula en un *PredictionResult* junto con el *bounding box* geográfica y las dimensiones del *grid*, para poder pintar el mapa de calor.

En cuanto al método *train()* (Anexo I, Código 12), tras recibir las matrices *features* y *labels*, realiza las siguientes operaciones:

- Expansión de características (Anexo I, Código 13): se generan términos cuadráticos a partir de las variables. Por ejemplo, a partir de BIO6 se crea BIO6², y a partir de BIO6 y BIO12, se puede crear el producto de ambas. Esto aumenta el número de columnas de la matriz de entrenamiento, lo cual también permite capturar otro tipo de relacionales que nos ayuden a detectar el nicho ecológico de la especie.
- Balanceo de clases: al igual que con Random Forest, las presencias son mucho menos frecuentes que las ausencias, por lo que se asigna un peso 10 veces mayor a los errores en presencias que a los errores de background.
- Descenso de gradiente: los pesos se ajustan iterativamente, durante un máximo de 5000 iteraciones con una tasa de aprendizaje de 0.01. En cada iteración se calcula el gradiente de la función de pérdida, que mide cuanto se equivoca el modelo, y se actualizan los pesos en la dirección que reduce ese error.
- Regularización L2: se añade una penalización proporcional al cuadrado de los pesos. Esto evita que el modelo memorice ruido.
- Almacenamiento del modelo: tras converger, el vector de pesos *_weights* queda guardado en el objeto. Con este vector ya podemos pasar a predecir.

Una vez entrenado el modelo, *predict()* aplica la ecuación aprendida a cada celda del territorio. Recibe una matriz de *features* de dimensiones $K \times m$,

donde K es el número de celdas del *grid* de predicción y m es el número de variables seleccionadas. Por ejemplo, si el mapa tiene 200 x 200 celdas y usamos 4 variables, la matriz tendrá 40000 filas y 4 columnas. El proceso de este método es el siguiente:

- Expansión cuadrática: se expande la matriz de entrada como en el método *train()* al calcular variables cuadráticas.
- Cálculo del score lineal: para cada fila se calcula $score = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_m \cdot x_m + b$. Esto es una simple suma ponderada de las variables usando los pesos de *train()*.
- Aplicación de la función logística: el score se transforma en probabilidad mediante $p = \frac{1}{1+e^{-score}}$.
- Suavizado con temperatura: se aplica la temperatura de *softmax* $T = 3.0$: $p_{suavizada} = \frac{p^{1/T}}{p^{1/T} + (1-p)^{1/T}}$. Esto evita que el mapa tenga demasiadas zonas a 0 o a 1.

La salida es un vector *std::vector<float>* de tamaño K , donde cada posición contiene la probabilidad de presencia de la celda correspondiente.

Hiperparámetros y justificación

Los hiperparámetros por defecto de *MaxEntPredictor* se muestran en la Tabla 18. Han sido ajustado para ofrecer un comportamiento estable en datasets pequeños, es decir, de las zonas de estudio comentadas anteriormente.

Hiperparámetro	Valor por defecto	Justificación
Regularización L2 (<i>_meRegularization</i>)	0.01	Evita el sobreajuste en datasets pequeños
Tasa de aprendizaje (<i>_meLearningRate</i>)	0.01	Valor conservador que garantiza convergencia estable
Iteraciones máximas (<i>_meMaxIterations</i>)	5000	Suficiente para converger en datasets de SDM sin alargar innecesariamente el entrenamiento
Características cuadráticas (<i>_meUseQuadratic</i>)	true	Permite modelar nichos ecológicos con óptimos intermedios
Temperatura softmax (<i>_meSoftmaxTemperature</i>)	3.0	Suaviza las probabilidades para facilitar comparación visual entre escenarios actuales y futuros

Peso de presencias	10.0	Compensa el desbalanceo frente a los puntos de fondo, priorizando no perder presencias reales
---------------------------	-------------	--

Tabla 18 : Hiperparámetros MaxEnt

La combinación de la regularización y las características cuadráticas es clave. Sin la primera, el modelo cuadrático tendría demasiados parámetros para los pocos datos que tenemos y sobreajustaría. Sin la segunda, el modelo solo podría representar relaciones lineales y perdería capacidad de detección de óptimos de temperatura o precipitación.

Para visualizar de forma concreta el algoritmo, pongamos un ejemplo: tomando de referencia el *Pinus uncinata* en Sierra Mágina, supongamos que BIO12 es la variable más influyente. MaxEnt aprende un peso positivo para BIO12 y un peso negativo para BIO12¹². Esto significa que:

- A precipitaciones bajas, la probabilidad será baja.
- A precipitaciones medias/altas, la probabilidad crece.
- Si llegan a ser muy altas, el término cuadrático penaliza y la probabilidad vuelve a bajar.

Así, la curva de respuesta para cada variable bioclimática será mucho más suave, como se observa en la Ilustración 48.

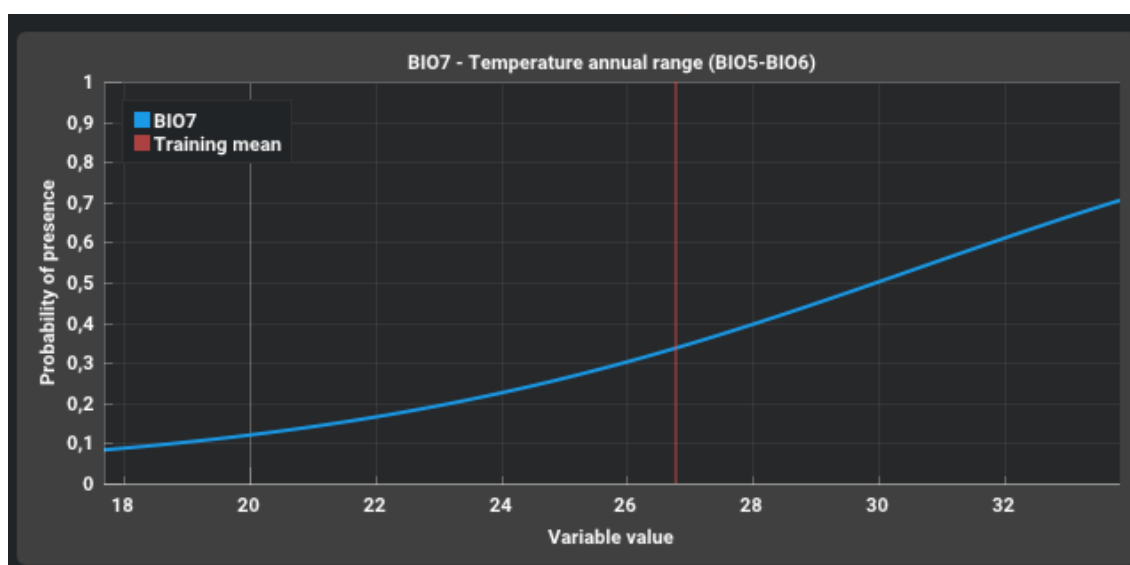


Ilustración 48 : Curva de respuesta MaxEnt (autor)

MaxEnt también implementa un método para calcular la importancia de las variables basada en los pesos que se le asignan a las mismas durante el entrenamiento y la predicción (Anexo I, Código 15).

A modo de resumen, en la Ilustración 49, se observa el flujo contextualizado al igual que se mostró para el algoritmo Random Forest, de

forma que se pueda ver donde encaja exactamente MaxEnt en el flujo de simulación.

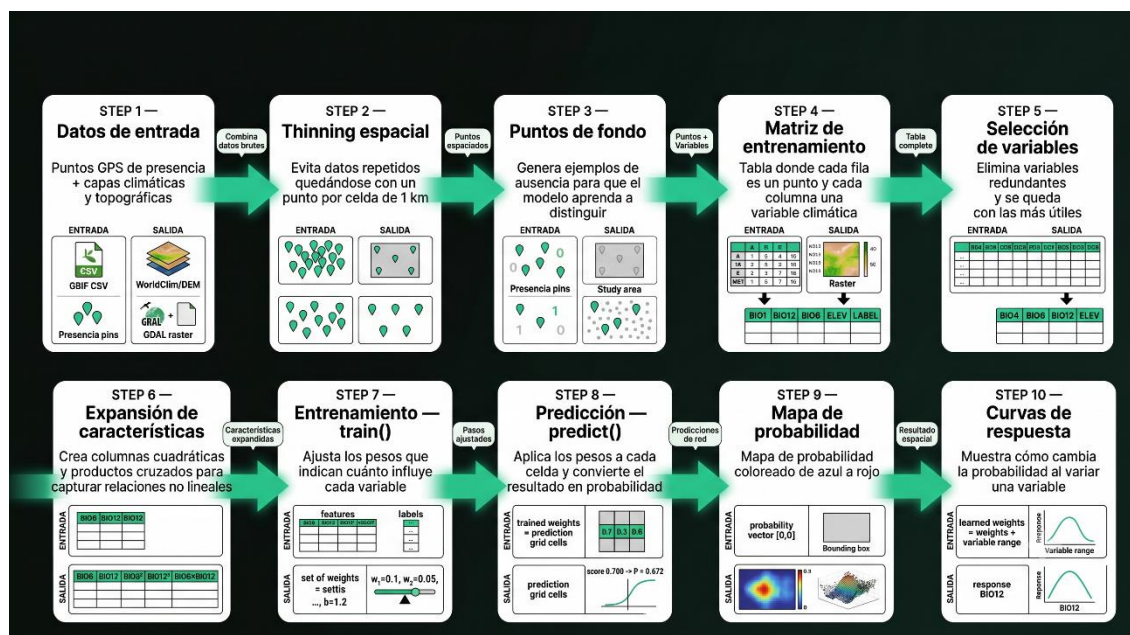


Ilustración 49 : Flujo de predicción MaxEnt contextualizado

6.3 Incremento III: Exportación y renderizado en GEU

En este tercer incremento, se transformará el vector de probabilidades resultante del incremento anterior en resultados tangibles y visuales en el propio motor GEU (Anexo I, Código 17) para su evaluación antes de trabajar en la parte web y demás. La implementación se concentra en dos clases principales: *HeatmapGenerator*, que gestiona todo lo relacionado con la visualización y *PredicciónEspeciesGUIAdapter*, que orquesta las exportaciones desde la interfaz de usuario.

Fase 6: Visualización y generación de representaciones intermedias

Tras la predicción, los datos numéricos se transforman en diferentes representaciones visuales dentro de GEU. Esta fase involucra tres objetos generados por *HeatmapGenerator*:

1. Heatmap 2D (Anexo I, Código 18): la textura se genera aplicando un *upsampling* bicúbico para suavizar la visualización pixelada dada la baja resolución por pixel de WorldClim. Se utiliza un *colormap* personalizado y la textura es gestionada por OpenGL (*GEU::Texture*).
2. Nube de puntos 3D (Anexo I, Código 19): *generateHeatmapPointCloud()* crea un *GEU::PointCloud* donde la coordenada Z de cada vértice es *probabilidad x elevación_DEM x exageración_vertical*, garantiza coherencia visual entre 2D y 3D.
3. Mapa de diferencias (Anexo I, Código 20): cuando ejecutamos predicciones futuras, se generan texturas a modo de comparación

(*generateDifferenceTexture()*) respecto a la situación actual con el resto de escenarios, usando el color rojo para pérdida de hábitat, verde para ganancia y gris para estabilidad.

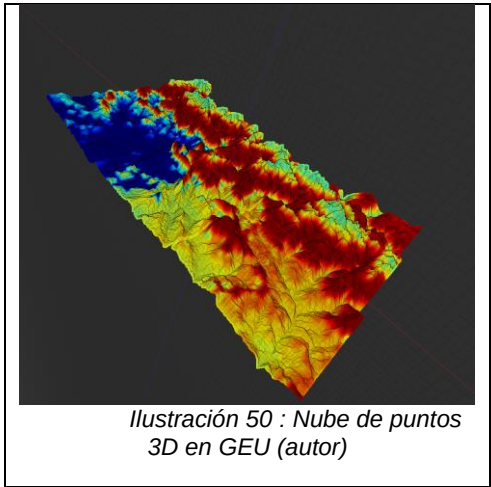


Ilustración 50 : Nube de puntos 3D en GEU (autor)

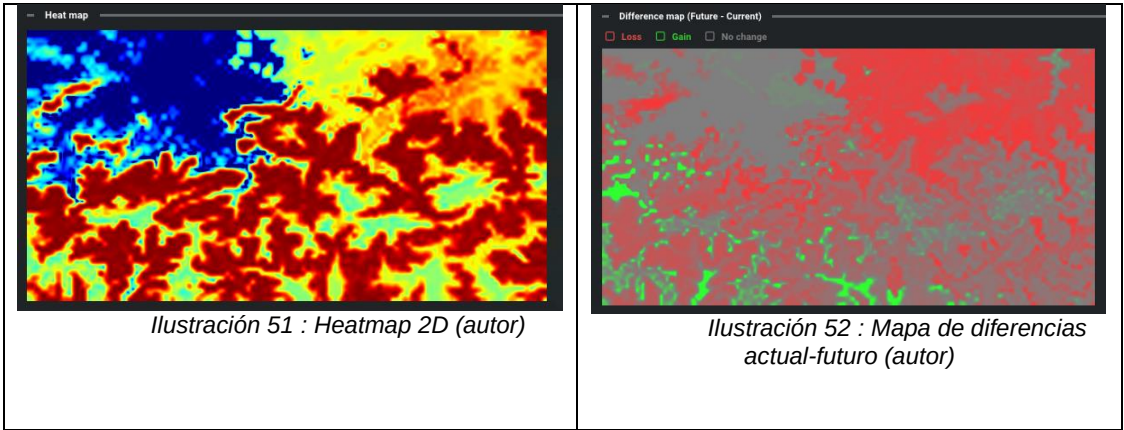


Ilustración 51 : Heatmap 2D (autor)

Ilustración 52 : Mapa de diferencias actual-futuro (autor)

Fase 7: Exportación multiformato y persistencia

La fase final del pipeline materializa los resultados en formatos persistentes para su uso de forma externa a GEU (además de la futura web, también son interoperables para otro software como QGIS, Unity, etc.). El sistema exporta (tras pulsar en botones correspondientes de la interfaz) a nueve formatos distintos organizados en subcarpetas dentro de *DataPredictionSpecies/Exports/*:

Formato	Subcarpeta	Contenido	Clase/método
CSV	CSV/	Tabla con coordenadas y probabilidades por celda	SimulationManager::exportResultsCSV()

GeoJSON	GeoJSON/	Puntos georreferenciados con propiedad probability	SimulationManager::exportResultsGeoJSON()
GeoTIFF	GeoTIFF/	Raster georreferenciado con el mismo extent y CRS que los datos de entrada	SimulationManager::exportResultsGeoTIFFAuto()
PNG heatmap	PNG/	Imagen del heatmap coloreado	HeatmapGenerator::exportHeatmapPNG()
PNG diferencias	PNG/	PNG por escenario futuro vs actual	HeatmapGenerator::exportDifferencePNG()
JSON métricas	JSON/	AUC, TSS, Kappa, etc. (por escenario)	SimulationManager::exportMetricsJSON()
JSON configuración	JSON/	Parámetros del modelo y del escenario	SimulationManager::exportConfigJSON()
JSON tablas de diferencias	JSON/	Tres tablas: área por umbral, área continua y balance de hábitat	SimulationManager::exportDifferenceTablesJSON()
Curvas de respuesta	ResponseCurves/	CSV y PNG por variable	SimulationManager::exportResponseCurvesCSV/PNG()
PLY nube de puntos	PLY/	Nube de puntos 3D con color y elevación	HeatmapGenerator::generateHeatmapPointCloud() + GEU::FileManager::savePointCloud()

Tabla 19 : Archivos exportados de GEU

Todos los métodos de exportación se apoyan en funciones auxiliares como `sanitizeFilename()` y `makeUniqueFilename()`, definidos en `DataStructures.h`, para garantizar que los nombres de archivo son válidos y no se repiten ni sobreescriben a anteriores. En la Ilustración 53, se pueden observar los distintos controles por los cuales el usuario puede exportar los archivos comentados anteriormente.



Ilustración 53 : Botones de exports de archivos

6.4 Incremento IV: Subsistema de interfaz GEU (ImGui)

El cuarto incremento añade la capa de presentación al módulo. Ya tenemos toda la parte de procesamiento, pero debemos de mostrar de forma clara al usuario cómo utilizar esa lógica de inferencia de simulación y todos los controles necesarios para mostrar resultados, métricas, predicción, etc. La elección de ImGui para la interfaz reside en que, básicamente, es el estándar y ya se utiliza en el resto de GEU. Toda esta lógica de la interfaz gráfica reside en *PrediccionEspeciesGUIAdapter*, que hereda de *InterfaceAdapter* y se registra en el gestor de módulo de la aplicación principal *GEUApp*.

6.4.1 Arquitectura del adaptador GUI

La clase *PrediccionEspeciesGUIAdapter* reside en *Source/UI/* y actúa como controlador entre el usuario y *SimulationManager*. Su ciclo de vida se rige por los siguiente métodos:

- *onAttach()*: se ejecuta cuando el módulo se activa en la interfaz principal. Se encarga de inicializar los valores por defecto de la interfaz.
- *renderGUIObject()* (Anexo I, Código 21): se ejecuta una vez por frame, dibuja todas las secciones del panel con sus respectivos components y comprueba si hay tareas asíncronas pendientes.
- *onDetach()*: se ejecuta al cerrar el módulo, liberando texturas, nubes e hilos pendientes.

Su función dentro del módulo de simulación es hacer de controlador en el patrón MVC (Modelo-Vista-Controlador). Se dedica a:

- Recoger las acciones del usuario.

- Validar que los datos necesarios estén cargados antes de llamar a una operación (no que los datos sean válidos como tal).
- Invocar los métodos públicos de *SimulationManager*.
- Mostrar los resultados devueltos.
- Gestionar el estado visual de la interfaz.

6.4.2 Secciones del panel de control

El panel se organiza en secciones, cada una implementada en un método privado diferente. Esta división facilita el mantenimiento, pues si se modifica la sección de exportación, no habría que cambiar nada de entrenamiento.

Sección	Método	Funcionalidad
Carga de datos	<code>renderDataLoadingSection()</code>	Botones para cargar CSV de ocurrencias, clima actual/futuro y DEM
Configuración	<code>renderConfigurationSection()</code>	Selector de algoritmo (RF/MaxEnt), hiperparámetros, jittering, etc.
Selección de variables	<code>renderVariableSelectionSection()</code>	Umbral de correlación y VIF, botón para ejecutar análisis.
Entrenamiento	<code>renderTrainingSection()</code>	Botón de entrenamiento, opción de cross-validation K-Fold, barra de progreso.
Escenarios	<code>renderScenarioSection()</code>	Selector de escenarios SSP y periodo futuro, botón para predecir futuro.
Resultados	<code>renderResultsSection()</code>	Visualización del heatmap, umbral de probabilidad, controles de la nube 3D, etc.
Comparación	<code>renderCompareSection()</code>	Comparación entre escenarios, mapas de diferencias, tablas.
Exportación	<code>renderExportSection()</code>	Botones para exportar a todos los formatos soportados
Barra de estado	<code>renderStatusBar()</code>	Mensaje de estado actual y barra de progreso.

Tabla 20 : Secciones interfaz gráfica GEU

Cada sección utiliza componentes típicos de ImGui como *Button()*, *SliderFloat()*, etc. Para las curvas de respuesta y gráfica se usa la librería *ImPlot*.

6.4.3 Progreso asíncrono y máquina de estados

El entramiento de un modelo y la predicción puede llegar a ser un proceso computacionalmente costoso, sobre todo si tenemos muchos datos. Si estas operaciones se ejecutan sobre el hilo principal, la interfaz se congelaría (al principio estaba implementado así). Para evitarlo, el adaptador utiliza un sistema de ejecución asíncrona basado en *std::thread*.

El flujo se basa en:

1. El usuario pulsa “Entrenar”.
2. El adaptador guarda *_pendingTask = PendingTask::Training*.
3. Se lanza *_workerThread* con la función *runAsync()* (Anexo I, Código 22), que ejecuta *SimulationManager::trainModel()* en segundo plano.
4. Durante la ejecución, *SimulationManager* reporta progreso mediante un *callback* (para actualizar la barra de progreso).
5. En cada frame, *onUpdate* lee *_progress* y dibuja una barra de progreso.
6. Cuando el hilo termina, *checkAsyncCompletion()* (Anexo I, Código 23) detecta que *_workerThread* ha finalizado y ejecuta *onTrainingComplete()*, que actualiza la interfaz con las métricas y el heatmap.

Además, *SimulationManager* mantiene una máquina de estados *SimulationState* con los estados: *Idle*, *LoadingData*, *Validating*, *Training*, *Predicting*, *Complete* y *Error*.

6.4.4 Localización

El módulo está preparado para soportar varios idiomas (y GEU en general) mediante un sistema de localización. Todas las cadenas visibles se obtienen a través de *LocaleStrings*, una utilidad de *GEUCore* que carga un fichero CSV de traducciones y devuelve un texto correspondiente al idioma activo (inglés/español). El formato del archivo CSV se muestra en la Ilustración 54.

PREDSP_BIO_16	Precipitation of wettest quarter	Precipitacion del trimestre mas humedo		
PREDSP_BIO_17	Precipitation of driest quarter	Precipitacion del trimestre mas seco		
PREDSP_BIO_18	Precipitation of warmest quarter	Precipitacion del trimestre mas calido		
PREDSP_BIO_19	Precipitation of coldest quarter	Precipitacion del trimestre mas frio		
PREDSP_BG_RATIO	BG ratio (x presences)	Ratio BG (x presencias)		
PREDSP_BIOCLIM_CURRENT	Current bioclimatic variables	Variables bioclimáticas actuales		
PREDSP_CLEAR_3D_SCENE	Clear 3D scene	Limpiar escena 3D		
PREDSP_COMPARE_NO_SCENARIO	Select a scenario	Selecciona un escenario		
PREDSP_COMPARE_SCENARIOS	Compare scenarios	Comparar escenarios		
PREDSP_COMPARE_SCENARIO_A	Scenario A	Escenario A		
PREDSP_COMPARE_SCENARIO_B	Scenario B	Escenario B		
PREDSP_COMPARE_SHOW_DIFF	Show difference map	Mostrar mapa de diferencias		
PREDSP_COMPARE_TITLE	Side-by-side comparison	Comparación lado a lado		
PREDSP_CONF_MATRIX	Confusion matrix (threshold %.1f)	Matriz de confusión (umbral %.1f)		
PREDSP_CONT_AREA	Cont. area (km2)	Area cont. (km2)		
PREDSP_CONTINUOUS_AREA	Continuous weighted area	Área continua ponderada		
PREDSP_CORR_MATRIX	Correlation matrix	Matriz de correlación		
PREDSP_CORR_THRESHOLD	Correlation threshold	Umbral de correlación		
PREDSP_CV_ENABLE	Cross-validation	Validación cruzada		
PREDSP_CV_FOLDS	Folds (K)	Pliegues (K)		
PREDSP_CV_MEAN_STD	Mean ± Std	Media ± Std		
PREDSP_CV_RESULTS	Cross-Validation (K-Fold)	Validación Cruzada (K-Fold)		
PREDSP_DELETE_SCENARIO	Delete scenario	Eliminar escenario		
PREDSP_DEM_ELEVATION	DEM / Terrain elevation	DEM / Elevación del terreno		
PREDSP_DEM_LOADED_ACTIVE	DEM loaded (active)	DEM cargado (activo)		
PREDSP_DEM_STATUS	DEM status	Estado DEM		

Ilustración 54 : Archivo de localización de cadenas de texto para idiomas (autor)

Simplemente, mediante una clave (ej. PREDSP_TRAIN_MODEL), se obtiene la cadena en español (“Entrenar modelo”) o en inglés (“Train model”), según el idioma seleccionado. Todas las claves del módulo implementado llevan el prefijo “PREDSP_” para evitar colisiones con otras cadenas existentes.

6.5 Incremento V: Subsistema frontend web

CAPÍTULO 7 RESULTADOS Y EVALUACIÓN

7. RESULTADOS Y EVALUACIÓN

7.1 Métricas de rendimiento del modelo

7.2 Mapas de predicción: actual y escenarios futuros

7.3 Visualización en TerraPredict: capturas y análisis

7.4 Comparativa de escenarios: áreas de hábitat

7.5 Rendimiento computacional y tiempos de ejecución

7.6 Validación visual

CAPÍTULO 8

CONCLUSIONES Y LÍNEAS FUTURAS

8. CONCLUSIONES Y LÍNEAS FUTURAS

8.1 Cumplimiento de objetivos

8.2 Aportaciones del trabajo

8.3 Limitaciones del sistema

8.4 Líneas futuras de trabajo

Referencias bibliográficas

- [1] «WorldClim». Accedido: 11 de febrero de 2026. [En línea]. Disponible en: <https://www.worldclim.org/>
- [2] «GBIF». Accedido: 2 de febrero de 2026. [En línea]. Disponible en: <https://www.gbif.org/>
- [3] «¿Qué es un gemelo digital? | IBM». Accedido: 21 de febrero de 2026. [En línea]. Disponible en: <https://www.ibm.com/es-es/think/topics/digital-twin>
- [4] J. Miller, «Species Distribution Modeling», *Geogr. Compass*, vol. 4, n.º 6, pp. 490-509, 2010, doi: 10.1111/j.1749-8198.2010.00351.x.
- [5] S. Beery, E. Cole, J. Parker, P. Perona, y K. Winner, «Species Distribution Modeling for Machine Learning Practitioners: A Review», en *Proceedings of the 4th ACM SIGCAS Conference on Computing and Sustainable Societies*, en COMPASS '21. New York, NY, USA: Association for Computing Machinery, sep. 2021, pp. 329-348. doi: 10.1145/3460112.3471966.
- [6] R. D. Crego, J. A. Stabach, y G. Connette, «Implementation of species distribution models in Google Earth Engine», *Divers. Distrib.*, vol. 28, n.º 5, pp. 904-916, 2022, doi: 10.1111/ddi.13491.
- [7] J. Velásquez-Tibatá, M. H. Olaya-Rodríguez, D. López-Lozano, C. Gutiérrez, I. González, y M. C. Londoño-Murcia, «BioModelos: A collaborative online system to map species distributions», *PLOS ONE*, vol. 14, n.º 3, p. e0214522, mar. 2019, doi: 10.1371/journal.pone.0214522.
- [8] «Figure AR6 WG1». Accedido: 12 de marzo de 2026. [En línea]. Disponible en: <https://www.ipcc.ch/report/ar6/wg1/figures/chapter-4/figure-4-40>
- [9] «QGIS — OSGeoLive 16.0 Documentation». Accedido: 12 de marzo de 2026. [En línea]. Disponible en: https://live.osgeo.org/es/overview/qgis_overview.html
- [10] B. Naimi y M. B. Araújo, «sdm: a reproducible and extensible R platform for species distribution modelling», *Ecography*, vol. 39, n.º 4, pp. 368-375, 2016, doi: 10.1111/ecog.01881.
- [11] C. GDAL/OGR contributors, «GDAL/OGR geospatial data abstraction software library», *Open Source Geospatial Found.*, 2020.
- [12] R. R. Curtin, M. Edel, M. Lozhnikov, Y. Mentekidis, S. Ghaisas, y S. Zhang, «mlpack 3: a fast, flexible machine learning library», *J. Open Source Softw.*, vol. 3, n.º 26, p. 726, 2018.
- [13] O. Cornut y others, «Dear imgui: a bloat-free graphical user interface library for c++». 2023.

- [14] P. Contributors, «PROJ coordinate transformation software library», *Open Source Geospatial Found. Beaverton USA*, 2022.
- [15] J. M. Jurado, F. García-Castillo, D. Jurado, y L. Ortega, «Detection of landscape features with visible and thermal imaging at the Castle of Puerta Arenas», *Archaeol. Anthropol. Sci.*, vol. 15, n.º 10, p. 164, 2023.
- [16] C. F. xyHt, «Creating Earth's Digital Twin», xyHt. Accedido: 12 de marzo de 2026. [En línea]. Disponible en: <https://www.xyht.com/lidarimaging/creating-earths-digital-twin/>
- [17] «WebGL - Low-Level 3D Graphics API Based on OpenGL ES», The Khronos Group. Accedido: 2 de junio de 2026. [En línea]. Disponible en: <https://www.khronos.org/webgl/>
- [18] «Modelos de distribución de especies de flora y fauna – Real Jardín Botánico CSIC». Accedido: 12 de marzo de 2026. [En línea]. Disponible en: <https://rjb.csic.es/evento/modelos-de-distribucion-de-especies-de-flora-y-fauna/>
- [19] «CesiumJS», Cesium. Accedido: 3 de junio de 2026. [En línea]. Disponible en: <https://cesium.com/platform/cesiumjs/>
- [20] «Three.js – JavaScript 3D Library». Accedido: 3 de junio de 2026. [En línea]. Disponible en: <https://threejs.org/>
- [21] «¿Qué es la gestión del ciclo de vida de las aplicaciones (ALM)?» Accedido: 1 de marzo de 2026. [En línea]. Disponible en: <https://www.redhat.com/es/topics/devops/what-is-application-lifecycle-management-alm>
- [22] T. D. P. Manager, «10 Herramientas ALM Para Entregar Mejores Proyectos», The Digital Project Manager. Accedido: 1 de marzo de 2026. [En línea]. Disponible en: <https://thedigitalprojectmanager.com/es/tools/herramientas-alm/>
- [23] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7.ª ed. Newtown Square, PA: Project Management Institute, 2021.
- [24] E. Gamma, R. Helm, R. Johnson, y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.
- [25] Agile Business Consortium, *The DSDM Agile Project Framework Handbook*. Agile Business Consortium, 2014.
- [26] T. Akenine-Möller, E. Haines, y N. Hoffman, *Real-Time Rendering*, 4.ª ed. CRC Press, 2018.
- [27] «Créditos ECTS frente a Créditos Tradicionales». Accedido: 3 de marzo de 2026. [En línea]. Disponible en: <https://www.campuseducacion.com/creditos-tradicionales-ects-cursos-oposiciones-maestros-profesores>
- [28] «Junior qa tester salario - Comprueba junior qa tester salario promedio en Jooble», Jooble. Accedido: 3 de marzo de 2026. [En línea]. Disponible en: <https://es.jooble.org/salary/junior-qa-tester>
- [29] T. A. R. Platform, «¿Cuánto gana un programador en España?  | 4Geeks Academy», The AI Reskilling Platform. Accedido: 3 de marzo de 2026. [En línea]. Disponible en: <https://ai-reskilling-platform.replit.app/>
- [30] «Ofertas de trabajo de Project manager junior». Accedido: 3 de marzo de 2026. [En línea]. Disponible en: <https://www.infojobs.net/ofertas-trabajo/project-manager-junior>

- [31] «Salario de Programador/a junior en España». Accedido: 15 de febrero de 2026. [En línea]. Disponible en: <https://es.indeed.com/career/programador-junior/salaries>
- [32] A. O. González, «Salario Project Manager en España 2025: Guía Completa», EIGP - Escuela Internacional de Gestión de Proyectos. Accedido: 3 de marzo de 2026. [En línea]. Disponible en: <https://eigp.es/blog/salario-project-manager.php/>
- [33] «idealo – tu comparador de precios». Accedido: 16 de febrero de 2026. [En línea]. Disponible en: <https://www.idealo.es/>
- [34] «PcComponentes.com | Tienda de Informática y Tecnología online», PcComponentes. Accedido: 16 de febrero de 2026. [En línea]. Disponible en: <https://www.pccomponentes.com/>
- [35] Chloe, «¿Cuál es la vida útil real de un PC gamer o portátil en 2026?», GEEKOM España. Accedido: 18 de febrero de 2026. [En línea]. Disponible en: <https://www.geekom.es/cuanto-dura-un-ordenador-o-pc-sobremesa-portatil/>
- [36] «OLED Vida útil de la pantalla: ¿Cuántos años durará? Consejos para prolongarla OLED Display Esperanza de vida - OLED/LCD Proveedor». Accedido: 18 de febrero de 2026. [En línea]. Disponible en: <https://www.panoxdisplay.com/es/solution/lifespan-of-oled-guide/>
- [37] «Vida útil del mouse para juegos: ¿cuánto debería durar?», MAMBASNAKE. Accedido: 18 de febrero de 2026. [En línea]. Disponible en: <https://mambasnake.com/es/blogs/news/how-long-can-a-gaming-mouse-last>
- [38] «Qué es un teclado mecánico: dónde empezar y qué buscar». Accedido: 18 de febrero de 2026. [En línea]. Disponible en: <https://store.epicgames.com/es-MX/news/what-is-a-mechanical-keyboard-where-to-start-and-what-to-look-for>
- [39] «Open-source software», *Wikipedia*. 17 de febrero de 2026. Accedido: 19 de febrero de 2026. [En línea]. Disponible en: https://en.wikipedia.org/w/index.php?title=Open-source_software&oldid=1338801929
- [40] «Inicio», GEU Docs. Accedido: 19 de febrero de 2026. [En línea]. Disponible en: <https://gggjaen.github.io/>
- [41] «Sierra Mágina», *Wikipedia, la enciclopedia libre*. 26 de abril de 2025. Accedido: 6 de febrero de 2026. [En línea]. Disponible en: https://es.wikipedia.org/w/index.php?title=Sierra_M%C3%A1gina&oldid=167070161
- [42] «SIERRA MÁGINA - Ventana del Visitante - Portal Ambiental de la CSMA», Ventana del Visitante. Accedido: 10 de abril de 2026. [En línea]. Disponible en: https://www.juntadeandalucia.es/medioambiente/portal/web/ventanadelvisitante/detalle-buscador-mapa/-/asset_publisher/Jlbxh2qB3NwR/content/sierra-m%C3%81gina/255035
- [43] «Lapiaces o lenares: qué son, cómo se forman, tipos y dónde hay - Resumen», Ecología Verde. Accedido: 10 de abril de 2026. [En línea]. Disponible en: <https://ecologiaverde.elperiodico.com/lapiaces-o-lenares-que-son-como-se-forman-tipos-y-donde-hay-5307.html>
- [44] «La Sima de San Pedro y su peculiar lago», Fundación Aquae. Accedido: 10 de abril de 2026. [En línea]. Disponible en: <https://www.fundacionaquae.org/wiki/el-lago-de-la-sima-de-san-pedro/>

- [45] «Pinus nigra», Arbolapp. Accedido: 11 de febrero de 2026. [En línea]. Disponible en: <https://www.arbolapp.es/especies/ficha/pinus-nigra/>
- [46] «Sendero Paso Puerto de la Mata», Jaén Paraíso Interior. Accedido: 10 de febrero de 2026. [En línea]. Disponible en: <https://www.jaenparaisointerior.es/es/w/sendero-paso-puerto-de-la-mata>
- [47] «PUERTO DE LA MATA - Ventana del Visitante - Portal Ambiental de la CSMA», Ventana del Visitante. Accedido: 10 de febrero de 2026. [En línea]. Disponible en: https://www.juntadeandalucia.es/medioambiente/portal/web/ventanadelvisitante/detalle-buscador-mapa/-/asset_publisher/Jlbxh2qB3NwR/content/puerto-de-la-mata/255035
- [48] «Detalle buscador mapa - Ventana del Visitante». Accedido: 10 de abril de 2026. [En línea]. Disponible en: https://www.juntadeandalucia.es/medioambiente/portal/web/ventanadelvisitante/detalle-buscador-mapa/-/asset_publisher/Jlbxh2qB3NwR/content/puerto-de-la-mata/255035
- [49] «Evapotranspiración: Cómo Medirla E Influencia En La Agricultura». Accedido: 10 de febrero de 2026. [En línea]. Disponible en: <https://eos.com/es/blog/evapotranspiracion/>
- [50] «Pinus nigra», *Wikipedia, la enciclopedia libre*. 23 de enero de 2026. Accedido: 10 de abril de 2026. [En línea]. Disponible en: https://es.wikipedia.org/w/index.php?title=Pinus_nigra&oldid=171665942
- [51] «Ordesa y Monte Perdido. Gobierno de Aragón», Portal del Gobierno de Aragón. Accedido: 2 de junio de 2026. [En línea]. Disponible en: <https://www.aragon.es/-/parque-nacional-de-ordesa-y-monte-perdido>
- [52] «Parque nacional de Ordesa y Monte Perdido», *Wikipedia, la enciclopedia libre*. 13 de mayo de 2026. Accedido: 2 de junio de 2026. [En línea]. Disponible en: https://es.wikipedia.org/w/index.php?title=Parque_nacional_de_Ordesa_y_Monte_Perdido&oldid=173438513
- [53] V. Estiber, «Las mejores rutas por Ordesa y Monte Perdido | Estiber.com», Blog Estiber. Accedido: 2 de junio de 2026. [En línea]. Disponible en: <https://www.estiber.com/blog/las-mejores-rutas-por-monte-perdido-y-ordesa/>
- [54] «flora - PARQUE NACIONAL ORDESA Y MONTE PERDIDO». Accedido: 2 de junio de 2026. [En línea]. Disponible en: <https://www.ordesa.net/flora/>
- [55] «timberline». Accedido: 2 de junio de 2026. [En línea]. Disponible en: <https://dictionary.cambridge.org/es/diccionario/ingles/timberline>
- [56] X. Zheng y R. Basher, «Thin-Plate Smoothing Spline Modeling of Spatial Climate Data and Its Application to Mapping South Pacific Rainfalls», *Mon. Weather Rev.*, vol. 123, n.º 10, pp. 3086-3102, oct. 1995, doi: 10.1175/1520-0493(1995)123<3086:TPSSMO>2.0.CO;2.
- [57] «Coordenadas geográficas (3ºESO Académicas) - Wikipedia». Accedido: 10 de abril de 2026. [En línea]. Disponible en: http://maralboran.org/wikipedia/index.php/Coordenadas_geogr%C3%A1ficas_%283%C2%BAESO_Acad%C3%A9micas%29
- [58] «GeoTIFF», *Wikipedia, la enciclopedia libre*. 30 de abril de 2026. Accedido: 2 de junio de 2026. [En línea]. Disponible en: <https://es.wikipedia.org/w/index.php?title=GeoTIFF&oldid=173226594>

- [59] C. N. de I. Geográfica, «Centro de Descargas del CNIG (IGN)», Centro de Descargas del CNIG. Accedido: 17 de junio de 2026. [En línea]. Disponible en: <http://centrodedescargas.cnig.es>
- [60] «GGGJ – Página del grupo de investigación GGGJ de la Universidad de Jaén». Accedido: 3 de junio de 2026. [En línea]. Disponible en: <https://gggj.ujaen.es/>
- [61] S. Yoon y D. Manocha, «Cache-Efficient Layouts of Bounding Volume Hierarchies», *Comput. Graph. Forum*, vol. 25, n.º 3, pp. 507-516, sep. 2006, doi: 10.1111/j.1467-8659.2006.00970.x.
- [62] «¿Qué son las Single Page Application (SPA) y cómo funcionan?», UNIR. Accedido: 8 de junio de 2026. [En línea]. Disponible en: <https://www.unir.net/revista/ingenieria/single-page-application-que-es/>
- [63] «Separation of Concerns (SoC)», GeeksforGeeks. Accedido: 4 de junio de 2026. [En línea]. Disponible en: <https://www.geeksforgeeks.org/software-engineering/separation-of-concerns-soc/>
- [64] M. J. Martín, «SOLID: los 5 principios para desarrollar software de calidad», Profile Software Services. Accedido: 5 de junio de 2026. [En línea]. Disponible en: <https://profile.es/blog/principios-solid-desarrollo-software-calidad/>
- [65] R. Anderson, «spThin: an R package for spatial thinning of species occurrence records for use in ecological niche models», *Ecography*, vol. 38, p. 541, ene. 2015.
- [66] R. Valavi, J. Elith, J. J. Lahoz-Monfort, y G. Guillera-Aroita, «Modelling species presence-only data with random forests», *Ecography*, vol. 44, n.º 12, pp. 1731-1742, 2021, doi: 10.1111/ecog.05615.
- [67] «Random Trees — OpenCV 2.3.2 documentation». Accedido: 16 de junio de 2026. [En línea]. Disponible en: https://www.opencv.org.cn/opencvdoc/2.3.2/html/modules/ml/doc/random_trees.html
- [68] «¿Que es el índice gini? | Foro Alura Latam», Alura Latam. Accedido: 16 de junio de 2026. [En línea]. Disponible en: <https://app.aluracursos.com/forum/topico-que-es-el-indice-gini-221879>
- [69] S. V. R, «The Complete Guide to Platt Scaling», Train in Data's Blog. Accedido: 16 de junio de 2026. [En línea]. Disponible en: <https://www.blog.trainindata.com/complete-guide-to-platt-scaling/>
- [70] J. Elith, S. J. Phillips, T. Hastie, M. Dudík, Y. E. Chee, y C. J. Yates, «A statistical explanation of MaxEnt for ecologists: Statistical explanation of MaxEnt», *Divers. Distrib.*, vol. 17, n.º 1, pp. 43-57, ene. 2011, doi: 10.1111/j.1472-4642.2010.00725.x.

Anexos

Anexo I – Código fuente GEU

A continuación, se presentan fragmentos de código fuente que se consideran los más importantes para mostrar de una manera más técnica cómo se ha llevado la teoría a la práctica en C++ (solo se muestran algunos y solo del motor GEU dado que si no la extensión de la memoria sería mucho más larga).


```

std::vector<OccurrenceRecord> GeoDataLoader::loadCSV(const std::string& path) {
    std::vector<OccurrenceRecord> records;

    std::ifstream file(path);
    if (!file.is_open()) {
        return records;
    }

    std::string headerLine;
    if (!std::getline(file, headerLine)) {
        return records;
    }

    char separator = '\t';
    if (headerLine.find('\t') == std::string::npos) {
        separator = ',';
    }

    std::vector<std::string> headers;
    {
        std::stringstream ss(headerLine);
        std::string token;
        while (std::getline(ss, token, separator)) {
            token.erase(0, token.find_first_not_of(" \t\r\n"));
            token.erase(token.find_last_not_of(" \t\r\n") + 1);
            headers.push_back(token);
        }
    }

    int latCol = -1, lonCol = -1, speciesCol = -1;
    for (int i = 0; i < static_cast<int>(headers.size()); ++i) {
        std::string h = headers[i];
        std::transform(h.begin(), h.end(), h.begin(), ::tolower);

        if (h == "decimallatitude" || h == "latitude" || h == "lat") {
            latCol = i;
        }
        else if (h == "decimallongitude" || h == "longitude" || h == "lon" || h == "long") {
            lonCol = i;
        }
        else if (h == "species" || h == "scientificname" || h == "taxon") {
            speciesCol = i;
        }
    }

    if (latCol == -1 || lonCol == -1) {
        return records;
    }

    std::string line;
    while (std::getline(file, line)) {
        if (line.empty()) continue;

        std::vector<std::string> fields;
        {
            std::stringstream ss(line);
            std::string token;
            while (std::getline(ss, token, separator)) {
                token.erase(0, token.find_first_not_of(" \t\r\n"));
                token.erase(token.find_last_not_of(" \t\r\n") + 1);
                fields.push_back(token);
            }
        }

        if (static_cast<int>(fields.size()) <= std::max(latCol, lonCol)) {
            continue;
        }

        OccurrenceRecord record;
        record.latitude = parseDoubleSafe(fields[latCol]);
        record.longitude = parseDoubleSafe(fields[lonCol]);

        if (record.latitude < -90.0 || record.latitude > 90.0 ||
            record.longitude < -180.0 || record.longitude > 180.0) {
            continue;
        }

        if (speciesCol >= 0 && speciesCol < static_cast<int>(fields.size())) {
            record.species = fields[speciesCol];
        }

        records.push_back(std::move(record));
    }

    file.close();
    return records;
}

```


Código 1 : Carga de archivos de ocurrencias CSV de GBIF (autor)

```

std::vector<RasterLayer> GeoDataLoader::loadMultibandTiff(const std::string& path, const
GeoExtent* clipExtent) {
    initialize();

    std::vector<RasterLayer> layers;

    GDALDataset* dataset = static_cast<GDALDataset*>(
        GDALOpen(path.c_str(), GA_ReadOnly));

    if (!dataset) {
        return layers;
    }

    const int bandCount = dataset->GetRasterCount();
    const int fullWidth = dataset->GetRasterXSize();
    const int fullHeight = dataset->GetRasterYSize();

    double geoTransform[6];
    dataset->GetGeoTransform(geoTransform);

    const char* projWkt = dataset->GetProjectionRef();
    std::string projection = projWkt ? std::string(projWkt) : "";

    int xOff = 0, yOff = 0, xSize = fullWidth, ySize = fullHeight;
    if (clipExtent && clipExtent->isValid()) {
        int minCol = static_cast<int>(std::floor((clipExtent->minLon - geoTransform[0]) /
geoTransform[1]));
        int maxCol = static_cast<int>(std::ceil((clipExtent->maxLon - geoTransform[0]) /
geoTransform[1]));
        int minRow = static_cast<int>(std::floor((clipExtent->maxLat - geoTransform[3]) /
geoTransform[5]));
        int maxRow = static_cast<int>(std::ceil((clipExtent->minLat - geoTransform[3]) /
geoTransform[5]));

        xOff = std::max(0, std::min(fullWidth, minCol));
        yOff = std::max(0, std::min(fullHeight, minRow));
        int xEnd = std::max(0, std::min(fullWidth, maxCol));
        int yEnd = std::max(0, std::min(fullHeight, maxRow));
        xSize = xEnd - xOff;
        ySize = yEnd - yOff;

        if (xSize <= 0 || ySize <= 0) {
            GDALClose(dataset);
            return layers;
        }

        geoTransform[0] += xOff * geoTransform[1];
        geoTransform[3] += yOff * geoTransform[5];
    }

    for (int b = 1; b <= bandCount; ++b) {
        RasterLayer layer;
        layer.width = xSize;
        layer.height = ySize;
        std::memcpy(layer.geoTransform, geoTransform, sizeof(double) * 6);
        layer.projection = projection;
        layer.name = "bio" + std::to_string(b);

        GDALRasterBand* band = dataset->GetRasterBand(b);
        if (!band) {
            continue;
        }

        int hasNoData = 0;
        double noData = band->GetNoDataValue(&hasNoData);
        if (hasNoData) {
            layer.noDataValue = static_cast<float>(noData);
        }

        layer.data.resize(static_cast<size_t>(xSize) * ySize);
        CPLErr err = band->RasterIO(
            GF_Read, xOff, yOff, xSize, ySize,
            layer.data.data(), xSize, ySize,
            GDT_Float32, 0, 0);

        if (err != CE_None) {
            layer.data.clear();
        }

        layers.push_back(std::move(layer));
    }

    GDALClose(dataset);
    return layers;
}

```

Código 2 : Carga de archivos GeoTIFF multibanda WorldClim (autor)

```

std::pair<cv::Mat, cv::Mat> ClimateDataHandler::buildTrainingMatrix(
    const std::vector<OccurrenceRecord>& occurrences,
    int numBackgroundPoints) const {

    if (!isCurrentLoaded()) {
        return {};
    }

    if (numBackgroundPoints < 0) {
        numBackgroundPoints = static_cast<int>(occurrences.size()) * 2;
    }

    std::vector<OccurrenceRecord> validPresences;
    for (const auto& occ : occurrences) {
        if (hasValidDataAtPoint(occ.latitude, occ.longitude)) {
            validPresences.push_back(occ);
        }
    }

    if (validPresences.empty()) {
        return {};
    }

    resetRandomSeed(12345);
    auto backgroundPoints = generateBackgroundPoints(numBackgroundPoints, validPresences);

    const int totalSamples = static_cast<int>(validPresences.size() +
        backgroundPoints.size());
    const int numClimateFeatures = static_cast<int>(_currentLayers.size());
    const int numDemFeatures = (hasDEM() && _useDEMFeature) ? 1 : 0;
    const int numFeatures = numClimateFeatures + numDemFeatures;

    cv::Mat features(totalSamples, numFeatures, CV_32F);
    cv::Mat labels(totalSamples, 1, CV_32S);

    int idx = 0;

    for (const auto& occ : validPresences) {
        auto feats = extractFeaturesAtPoint(occ.latitude, occ.longitude);
        for (int j = 0; j < numFeatures; ++j) {
            features.at<float>(idx, j) = feats[j];
        }
        labels.at<int>(idx, 0) = 1;
        ++idx;
    }

    for (const auto& bg : backgroundPoints) {
        auto feats = extractFeaturesAtPoint(bg.latitude, bg.longitude);
        for (int j = 0; j < numFeatures; ++j) {
            features.at<float>(idx, j) = feats[j];
        }
        labels.at<int>(idx, 0) = 0;
        ++idx;
    }

    // Ruido gaussiano relativo para evitar memorización
    std::default_random_engine rng(12345);
    std::normal_distribution<float> noise(0.0f, 0.10f);
    for (int i = 0; i < features.rows; ++i) {
        for (int j = 0; j < features.cols; ++j) {
            float val = features.at<float>(i, j);
            if (std::isfinite(val) && val != 0.0f) {
                features.at<float>(i, j) = val * (1.0f + noise(rng));
            }
        }
    }

    // Guardar rangos de entrenamiento para detección de extrapolación (MESS)
    _trainFeatureMin.resize(numFeatures, std::numeric_limits<float>::max());
    _trainFeatureMax.resize(numFeatures, -std::numeric_limits<float>::max());
    _trainFeatureMean.assign(numFeatures, 0.0f);
    std::vector<int> meanCounts(numFeatures, 0);
    for (int i = 0; i < features.rows; ++i) {
        for (int j = 0; j < numFeatures; ++j) {
            float v = features.at<float>(i, j);
            if (std::isfinite(v)) {
                _trainFeatureMin[j] = std::min(_trainFeatureMin[j], v);
                _trainFeatureMax[j] = std::max(_trainFeatureMax[j], v);
                _trainFeatureMean[j] += v;
                ++meanCounts[j];
            }
        }
    }
    for (int j = 0; j < numFeatures; ++j) {
        if (meanCounts[j] > 0) {
            _trainFeatureMean[j] /= static_cast<float>(meanCounts[j]);
        }
    }

    return { features, labels };
}

```

Código 3 : Construcción matriz de entrenamiento (autor)

```

std::vector<OccurrenceRecord> ClimateDataHandler::generateBackgroundPoints(
    int n, const std::vector<OccurrenceRecord>& presences) const {

    std::vector<OccurrenceRecord> background;

    if (!isCurrentLoaded() || n <= 0 || presences.empty()) return background;

    GeoExtent occurrenceExtent;
    occurrenceExtent.minLon = occurrenceExtent.maxLon = presences[0].longitude;
    occurrenceExtent.minLat = occurrenceExtent.maxLat = presences[0].latitude;
    for (const auto& p : presences) {
        occurrenceExtent.minLon = std::min(occurrenceExtent.minLon, p.longitude);
        occurrenceExtent.maxLon = std::max(occurrenceExtent.maxLon, p.longitude);
        occurrenceExtent.minLat = std::min(occurrenceExtent.minLat, p.latitude);
        occurrenceExtent.maxLat = std::max(occurrenceExtent.maxLat, p.latitude);
    }
    constexpr double BUFFER_KM = 25.0;
    constexpr double KM_PER_DEG_LAT = 110.54;
    double meanLat = 0.0;
    for (const auto& p : presences) meanLat += p.latitude;
    meanLat /= presences.size();
    double kmPerDegLon = 111.32 * std::cos(meanLat * 3.14159265358979323846 / 180.0);
    double bufferDegLat = BUFFER_KM / KM_PER_DEG_LAT;
    double bufferDegLon = BUFFER_KM / kmPerDegLon;

    GeoExtent extent;
    extent.minLon = occurrenceExtent.minLon - bufferDegLon;
    extent.maxLon = occurrenceExtent.maxLon + bufferDegLon;
    extent.minLat = occurrenceExtent.minLat - bufferDegLat;
    extent.maxLat = occurrenceExtent.maxLat + bufferDegLat;

    GeoExtent rasterExtent = getExtent();
    if (rasterExtent.isValid()) {
        extent.minLon = std::max(extent.minLon, rasterExtent.minLon);
        extent.maxLon = std::min(extent.maxLon, rasterExtent.maxLon);
        extent.minLat = std::max(extent.minLat, rasterExtent.minLat);
        extent.maxLat = std::min(extent.maxLat, rasterExtent.maxLat);
    }
    if (!extent.isValid()) return background;

    // Distancia mínima a presencias (en grados, ~1km aprox)
    constexpr double minDistDeg = 0.01;
    const double minDistSq = minDistDeg * minDistDeg;

    std::uniform_real_distribution<double> lonDist(extent.minLon, extent.maxLon);
    std::uniform_real_distribution<double> latDist(extent.minLat, extent.maxLat);

    int maxAttempts = n * 100;
    int attempts = 0;

    while (static_cast<int>(background.size()) < n && attempts < maxAttempts) {
        ++attempts;

        double lon = lonDist(_rng);
        double lat = latDist(_rng);

        if (!isValidDataAtPoint(lat, lon)) continue;

        bool tooClose = false;
        for (const auto& pres : presences) {
            double dLon = lon - pres.longitude;
            double dLat = lat - pres.latitude;
            if ((dLon * dLon + dLat * dLat) < minDistSq) {
                tooClose = true;
                break;
            }
        }
        if (tooClose) continue;

        OccurrenceRecord bg;
        bg.latitude = lat;
        bg.longitude = lon;
        bg.species = "background";
        background.push_back(std::move(bg));
    }

    return background;
}

```

Código 4 : Generación de puntos de fondo/ausencias (autor)

```

std::vector<OccurrenceRecord> SimulationManager::thinOccurrences(
    const std::vector<OccurrenceRecord>& occurrences, double cellSizeKm) const {
    if (occurrences.empty() || cellSizeKm <= 0.0) return occurrences;

    double minLat = occurrences[0].latitude;
    double minLon = occurrences[0].longitude;
    double sumLat = 0.0;
    for (const auto& occ : occurrences) {
        minLat = std::min(minLat, occ.latitude);
        minLon = std::min(minLon, occ.longitude);
        sumLat += occ.latitude;
    }
    double meanLat = sumLat / occurrences.size();
    double latRad = meanLat * 3.14159265358979323846 / 180.0;

    double kmPerDegLat = 110.54;
    double kmPerDegLon = 111.32 * std::cos(latRad);
    double cellSizeDegLat = cellSizeKm / kmPerDegLat;
    double cellSizeDegLon = cellSizeKm / kmPerDegLon;

    // Celdas locales (offset por min) para evitar problemas con coordenadas negativas
    // y con la referencia global (ecuador/meridiano)
    std::set<std::pair<int, int>> occupiedCells;
    std::vector<OccurrenceRecord> thinned;
    thinned.reserve(occurrences.size());

    for (size_t i = 0; i < occurrences.size(); ++i) {
        const auto& occ = occurrences[i];
        int cellLat = static_cast<int>(std::floor((occ.latitude - minLat) / cellSizeDegLat));
        int cellLon = static_cast<int>(std::floor((occ.longitude - minLon) /
cellSizeDegLon));

        auto key = std::make_pair(cellLat, cellLon);
        bool inserted = occupiedCells.insert(key).second;

        if (inserted) {
            thinned.push_back(occ);
        }
    }

    return thinned;
}

```

Código 5 : Proceso de thinning espacial sobre ocurrencias (autor)


```
std::vector<OccurrenceRecord> SimulationManager::jitterOccurrences(  
    const std::vector<OccurrenceRecord>& occurrences,  
    double jitterMeters, int seed) const {  
    if (occurrences.empty() || jitterMeters <= 0.0) return occurrences;  
  
    std::vector<OccurrenceRecord> jittered = occurrences;  
    std::mt19937 rng(seed);  
    std::uniform_real_distribution<double> dist(-1.0, 1.0);  
    constexpr double METERS_PER_DEG_LAT = 111320.0;  
  
    for (auto& occ : jittered) {  
        double dLat = jitterMeters * dist(rng) / METERS_PER_DEG_LAT;  
        double dLon = jitterMeters * dist(rng) / (METERS_PER_DEG_LAT * std::cos(occ.latitude  
* 3.14159265358979323846 / 180.0));  
        occ.latitude += dLat;  
        occ.longitude += dLon;  
    }  
    return jittered;  
}
```

Código 6 : Proceso de jittering geográfico sobre ocurrencias (autor)

```

VariableSelectionResult FeatureSelector::analyze(
    const cv::Mat& features,
    const std::vector<std::string>& featureNames,
    const std::vector<std::string>& featureDescriptions,
    float corrThreshold,
    float vifThreshold) {
    VariableSelectionResult result;
    result.corrThreshold = corrThreshold;
    result.vifThreshold = vifThreshold;

    if (features.empty() || features.cols == 0) {
        return result;
    }

    const int numVars = features.cols;
    if (static_cast<int>(featureNames.size()) != numVars) {
        return result;
    }

    result.correlationMatrix = computeCorrelationMatrix(features);

    std::vector<bool> active(numVars, true);
    std::vector<std::string> reasons(numVars, "");

    result.variableInfos.resize(numVars);
    for (int i = 0; i < numVars; ++i) {
        result.variableInfos[i].name = featureNames[i];
        result.variableInfos[i].description = (i < static_cast<int>(
            featureDescriptions.size()))
            ? featureDescriptions[i] : "";
        result.variableInfos[i].selected = true;

        float maxCorr = 0.0f;
        int maxPartner = -1;
        for (int j = 0; j < numVars; ++j) {
            if (i == j) continue;
            if (result.correlationMatrix[i][j] > maxCorr) {
                maxCorr = result.correlationMatrix[i][j];
                maxPartner = j;
            }
        }
        result.variableInfos[i].maxCorrelation = maxCorr;
        if (maxPartner >= 0) {
            result.variableInfos[i].maxCorrPartner = featureNames[maxPartner];
        }
    }

    // Paso 1: Filtrado por correlacion de Pearson.
    // Para cada par (i,j) con |r| > umbral, eliminamos la variable
    bool removedCorr;
    do {
        removedCorr = false;

        float maxCorr = -1.0f;
        int varToRemove = -1;
        int partner = -1;

        for (int i = 0; i < numVars; ++i) {
            if (!active[i]) continue;
            for (int j = i + 1; j < numVars; ++j) {
                if (!active[j]) continue;
                float r = result.correlationMatrix[i][j];
                if (r > corrThreshold && r > maxCorr) {
                    maxCorr = r;
                    partner = j;
                    float avgI = 0.0f, avgJ = 0.0f;
                    int countI = 0, countJ = 0;
                    for (int k = 0; k < numVars; ++k) {
                        if (k == i || !active[k]) continue;
                        avgI += result.correlationMatrix[i][k];
                        ++countI;
                    }
                    for (int k = 0; k < numVars; ++k) {
                        if (k == j || !active[k]) continue;
                        avgJ += result.correlationMatrix[j][k];
                        ++countJ;
                    }
                    avgI = (countI > 0) ? avgI / countI : 0.0f;
                    avgJ = (countJ > 0) ? avgJ / countJ : 0.0f;
                    varToRemove = (avgI > avgJ) ? i : j;
                }
            }
        }

        if (varToRemove >= 0) {
            active[varToRemove] = false;
            char buf[128];
            snprintf(buf, sizeof(buf), "r=%.3f (%s)", maxCorr,
                featureNames[partner].c_str());
            reasons[varToRemove] = std::string(buf);
            removedCorr = true;
        }
    } while (removedCorr);

    // Paso 2: Filtrado por VIF.
    // Calcular VIF para cada variable activa contra las demas activas.
    // Eliminar iterativamente la de mayor VIF hasta que todas esten
    // por debajo del umbral.
    std::vector<int> activeIndices;
    for (int i = 0; i < numVars; ++i) {
        if (active[i]) activeIndices.push_back(i);
    }

    bool removedVif;
    do {
        removedVif = false;
        if (activeIndices.size() <= 2) break;

        std::vector<float> vifs = computeVIF(features, activeIndices);

        float maxVif = -1.0f;
        size_t idxToRemove = static_cast<size_t>(-1);
        int originalIdx = -1;

        for (size_t k = 0; k < activeIndices.size(); ++k) {
            if (vifs[k] > vifThreshold && vifs[k] > maxVif) {
                maxVif = vifs[k];
                idxToRemove = k;
                originalIdx = activeIndices[k];
            }
        }

        if (idxToRemove != static_cast<size_t>(-1)) {
            active[originalIdx] = false;
            char buf[64];
            snprintf(buf, sizeof(buf), "VIF=%.1f", maxVif);
            reasons[originalIdx] = std::string(buf);
            activeIndices.erase(activeIndices.begin() + idxToRemove);
            removedVif = true;
        }
    } while (removedVif);

    result.selectedIndices.clear();
    for (int i = 0; i < numVars; ++i) {
        result.variableInfos[i].selected = active[i];
        result.variableInfos[i].exclusionReason = reasons[i];
        if (active[i]) {
            result.selectedIndices.push_back(i);
        }
    }

    if (activeIndices.size() >= 2) {
        std::vector<float> finalVifs = computeVIF(features, activeIndices);
        for (size_t k = 0; k < activeIndices.size(); ++k) {
            result.variableInfos[activeIndices[k]].vif = finalVifs[k];
        }
    }

    result.valid = true;
    return result;
}

```

Código 7 : Selección de variables con Pearson y VIF (autor)

```
std::vector<float> FeatureSelector::computeVIF(const cv::Mat& features,
const std::vector<int>& activeIndices) {

    const int nVars = static_cast<int>(activeIndices.size());
    std::vector<float> vifs(nVars, 0.0f);

    if (nVars <= 1) return vifs;

    for (int k = 0; k < nVars; ++k) {
        int targetCol = activeIndices[k];

        const int nPred = nVars - 1;
        cv::Mat X(features.rows, nPred, CV_32F);
        int colIdx = 0;
        for (int m = 0; m < nVars; ++m) {
            if (m == k) continue;
            features.col(activeIndices[m]).copyTo(X.col(colIdx++));
        }

        cv::Mat y = features.col(targetCol);

        float r2 = computeRSquared(X, y);
        if (r2 >= 0.9999f) {
            vifs[k] = 9999.0f;
        } else if (r2 < 0.0f) {
            vifs[k] = 1.0f;
        } else {
            vifs[k] = 1.0f / (1.0f - r2);
        }
    }

    return vifs;
}
```

Código 8 : Cálculo del VIF para variables bioclimáticas (autor)

```

bool ForestPredictor::train(const cv::Mat& features, const cv::Mat& labels) {
    if (features.empty() || labels.empty()) {
        return false;
    }

    if (features.rows != labels.rows) {
        return false;
    }

    // No fijamos cv::theRNG() para preservar la aleatoriedad del bagging,
    // que es parte fundamental del algoritmo Random Forest.

    _forest = cv::ml::RTrees::create();
    _forest->setMaxDepth(_maxDepth);
    _forest->setMinSampleCount(_minSamples);
    _forest->setRegressionAccuracy(0.0f);
    _forest->setUseSurrogates(false);
    _forest->setMaxCategories(2);
    _forest->setCalculateVarImportance(true);
    int activeVarCount = std::max(1, features.cols / 3);
    _forest->setActiveVarCount(activeVarCount);

    // Pesos iguales para presencia/ausencia para contrarrestar el desbalance
    float priors[] = { 0.5f, 0.5f };
    cv::Mat priorsMat(1, 2, CV_32F, priors);
    _forest->setPriors(priorsMat);
    _forest->setTermCriteria(
        cv::TermCriteria(cv::TermCriteria::MAX_ITER, _numTrees, 0));

    // Split estratificado: 80% de cada clase
    std::vector<int> idxClass0, idxClass1;
    for (int i = 0; i < labels.rows; ++i) {
        if (labels.at<int>(i, 0) == 0) idxClass0.push_back(i);
        else idxClass1.push_back(i);
    }
    std::mt19937 splitRng(12345);
    std::shuffle(idxClass0.begin(), idxClass0.end(), splitRng);
    std::shuffle(idxClass1.begin(), idxClass1.end(), splitRng);
    size_t train0 = static_cast<size_t>(idxClass0.size() * 0.8);
    size_t train1 = static_cast<size_t>(idxClass1.size() * 0.8);

    int trainCount = static_cast<int>(train0 + train1);
    cv::Mat trainFeatures(trainCount, features.cols, CV_32F);
    cv::Mat trainLabels(trainCount, 1, CV_32S);
    int row = 0;
    for (size_t i = 0; i < train0; ++i, ++row) {
        features.row(idxClass0[i]).copyTo(trainFeatures.row(row));
        trainLabels.at<int>(row, 0) = 0;
    }
    for (size_t i = 0; i < train1; ++i, ++row) {
        features.row(idxClass1[i]).copyTo(trainFeatures.row(row));
        trainLabels.at<int>(row, 0) = 1;
    }

    int testCount = static_cast<int>((idxClass0.size() - train0) + (idxClass1.size() -
train1));
    cv::Mat testFeatures(testCount, features.cols, CV_32F);
    cv::Mat testLabels(testCount, 1, CV_32S);
    row = 0;
    for (size_t i = train0; i < idxClass0.size(); ++i, ++row) {
        features.row(idxClass0[i]).copyTo(testFeatures.row(row));
        testLabels.at<int>(row, 0) = 0;
    }
    for (size_t i = train1; i < idxClass1.size(); ++i, ++row) {
        features.row(idxClass1[i]).copyTo(testFeatures.row(row));
        testLabels.at<int>(row, 0) = 1;
    }

    try {
        _forest->train(trainFeatures, cv::ml::ROW_SAMPLE, trainLabels);
    }
    catch (const cv::Exception&) {
        _trained = false;
        return false;
    }

    _trained = true;

    // Calibración Platt y métricas sobre validación
    if (!testFeatures.empty()) {
        std::vector<float> rawProbs;
        rawProbs.reserve(testFeatures.rows);
        for (int i = 0; i < testFeatures.rows; ++i) {
            cv::Mat sample = testFeatures.row(i);
            cv::Mat votes;
            _forest->getVotes(sample, votes, 0);
            if (votes.cols >= 2) {
                float v0 = votes.at<int>(1, 0);
                float v1 = votes.at<int>(1, 1);
                rawProbs.push_back(v1 / (v0 + v1 + 1e-9f));
            } else {
                rawProbs.push_back(0.0f);
            }
        }
        calibratePlatt(rawProbs, testLabels);
        computeMetrics(testFeatures, testLabels);
    }

    return true;
}

```

Código 9 : Entrenamiento Random Forest (autor)

```
std::vector<float> ForestPredictor::predict(const cv::Mat& features) const {
    std::vector<float> probabilities;

    if (!_trained || features.empty()) {
        return probabilities;
    }

    probabilities.resize(features.rows, 0.0f);

    for (int i = 0; i < features.rows; ++i) {
        cv::Mat sample = features.row(i);
        cv::Mat votes;

        _forest->getVotes(sample, votes, 0);

        if (votes.cols >= 2) {
            float votesClass0 = votes.at<int>(1, 0);
            float votesClass1 = votes.at<int>(1, 1);
            float totalVotes = votesClass0 + votesClass1;

            if (totalVotes > 0.0f) {
                probabilities[i] = votesClass1 / totalVotes;
                if (_usePlattCalibration) {
                    float z = _plattA * probabilities[i] + _plattB;
                    if (z > 10.0f) probabilities[i] = 0.9999f;
                    else if (z < -10.0f) probabilities[i] = 0.0001f;
                    else probabilities[i] = 1.0f / (1.0f + std::exp(-z));
                }
            }
        }
    }

    return probabilities;
}
```

Código 10 : Predicción Random Forest (autor)

```

std::vector<VariableImportance> ForestPredictor::getVariableImportance() const {
    std::vector<VariableImportance> importance;

    if (!_trained) return importance;

    cv::Mat varImportance = _forest->getVarImportance();
    if (varImportance.empty()) return importance;

    double maxVal;
    cv::minMaxLoc(varImportance, nullptr, &maxVal);

    for (int i = 0; i < varImportance.rows; ++i) {
        VariableImportance vi;

        if (i < static_cast<int>(_featureNames.size())) {
            vi.name = _featureNames[i];
            bool foundDesc = false;
            for (int j = 0; j < NUM_BIOCLIM_VARS; ++j) {
                if (BIOCLIM_NAMES[j] == vi.name) {
                    vi.description = getBioclimDescription(j);
                    foundDesc = true;
                    break;
                }
            }
            if (!foundDesc) {
                if (vi.name == "ELEV") vi.description = "Elevacion del terreno (m)";
                else if (vi.name == "SLOPE") vi.description = "Pendiente del terreno
(grados)";
                else if (vi.name == "ASPECT") vi.description = "Orientacion del terreno
(grados)";
            }
        }
        else if (i < NUM_BIOCLIM_VARS) {
            vi.name = BIOCLIM_NAMES[i];
            vi.description = getBioclimDescription(i);
        }
        else if (i == NUM_BIOCLIM_VARS) {
            vi.name = "ELEV";
            vi.description = "Elevacion del terreno (m)";
        }
        else if (i == NUM_BIOCLIM_VARS + 1) {
            vi.name = "SLOPE";
            vi.description = "Pendiente del terreno (grados)";
        }
        else if (i == NUM_BIOCLIM_VARS + 2) {
            vi.name = "ASPECT";
            vi.description = "Orientacion del terreno (grados)";
        }
        else {
            vi.name = "VAR" + std::to_string(i + 1);
            vi.description = "Variable " + std::to_string(i + 1);
        }

        vi.importance = (maxVal > 0.0)
            ? static_cast<float>(varImportance.at<float>(i, 0) / maxVal)
            : 0.0f;

        importance.push_back(vi);
    }

    std::sort(importance.begin(), importance.end(),
        [](const VariableImportance& a, const VariableImportance& b) {
            return a.importance > b.importance;
        });

    return importance;
}

```

Código 11 : Obtención de importancia de variables Random Forest (autor)

```

bool MaxEntPredictor::train(const cv::Mat& features, const cv::Mat& labels) {
    if (features.empty() || labels.empty()) {
        return false;
    }

    _numOriginalFeatures = features.cols;

    cv::Mat expanded = expandFeatures(features);

    _featureMeans.resize(expanded.cols);
    _featureStds.resize(expanded.cols);

    for (int j = 0; j < expanded.cols; ++j) {
        cv::Mat col = expanded.col(j);
        cv::Scalar mean, stddev;
        cv::meanStdDev(col, mean, stddev);
        _featureMeans[j] = static_cast<float>(mean[0]);
        _featureStds[j] = static_cast<float>(stddev[0]);
        if (_featureStds[j] < 1e-8f) {
            _featureStds[j] = 1.0f;
        }
    }

    cv::Mat normalizedFeatures = normalizeFeatures(expanded);

    const int totalSamples = normalizedFeatures.rows;
    const int numTrain = static_cast<int>(totalSamples * 0.8);
    const int numVal = totalSamples - numTrain;

    std::vector<int> indices(totalSamples);
    std::iota(indices.begin(), indices.end(), 0);
    std::mt19937 rng(42);
    std::shuffle(indices.begin(), indices.end(), rng);

    cv::Mat trainFeatures(numTrain, normalizedFeatures.cols, CV_32F);
    cv::Mat trainLabels(numTrain, 1, CV_32S);
    cv::Mat valFeatures(numVal, normalizedFeatures.cols, CV_32F);
    cv::Mat valLabels(numVal, 1, CV_32S);

    for (int i = 0; i < numTrain; ++i) {
        normalizedFeatures.row(indices[i]).copyTo(trainFeatures.row(i));
        trainLabels.at<int>(i, 0) = labels.at<int>(indices[i], 0);
    }
    for (int i = 0; i < numVal; ++i) {
        normalizedFeatures.row(indices[numTrain + i]).copyTo(valFeatures.row(i));
        valLabels.at<int>(i, 0) = labels.at<int>(indices[numTrain + i], 0);
    }

    // Peso inverso de frecuencia para compensar desbalance presencia/background
    int nPos = 0, nNeg = 0;
    for (int i = 0; i < numTrain; ++i) {
        if (trainLabels.at<int>(i, 0) == 1) ++nPos;
        else ++nNeg;
    }
    float posWeight = 1.0f, negWeight = 1.0f;
    if (nPos > 0 && nNeg > 0) {
        posWeight = static_cast<float>(numTrain) / (2.0f * nPos);
        negWeight = static_cast<float>(numTrain) / (2.0f * nNeg);
    }

    const int numWeights = normalizedFeatures.cols + 1;
    _weights.resize(numWeights, 0.0f);

    float prevLoss = std::numeric_limits<float>::max();
    int earlyStopCount = 0;
    constexpr int earlyStopPatience = 50;

    for (int iter = 0; iter < _maxIterations; ++iter) {
        std::vector<float> gradients(numWeights, 0.0f);
        float totalLoss = 0.0f;

        float totalWeight = 0.0f;
        for (int i = 0; i < numTrain; ++i) {
            float z = _weights[numWeights - 1];
            for (int j = 0; j < trainFeatures.cols; ++j) {
                z += _weights[j] * trainFeatures.at<float>(i, j);
            }

            float pred = sigmoidTemperature(z, 3.0f);
            float label = static_cast<float>(trainLabels.at<int>(i, 0));
            float w = (label == 1.0f) ? posWeight : negWeight;
            float error = pred - label;

            for (int j = 0; j < trainFeatures.cols; ++j) {
                gradients[j] += w * error * trainFeatures.at<float>(i, j);
            }
            gradients[numWeights - 1] += w * error;

            float clippedPred = std::clamp(pred, 1e-7f, 1.0f - 1e-7f);
            totalLoss += w * (-label * std::log(clippedPred) + (1.0f - label) *
                std::log(1.0f - clippedPred));
            totalWeight += w;
        }

        float invW = (totalWeight > 0.0f) ? 1.0f / totalWeight : 1.0f / numTrain;
        for (int j = 0; j < numWeights; ++j) {
            gradients[j] *= invW;
        }

        for (int j = 0; j < numWeights - 1; ++j) {
            gradients[j] += _regularization * _weights[j];
        }

        for (int j = 0; j < numWeights; ++j) {
            _weights[j] -= _learningRate * gradients[j];
        }

        totalLoss *= invW;

        if (totalLoss >= prevLoss - 1e-6f) {
            ++earlyStopCount;
            if (earlyStopCount >= earlyStopPatience) {
                break;
            }
        }
        else {
            earlyStopCount = 0;
        }
        prevLoss = totalLoss;
    }

    _trained = true;

    if (numVal > 0) {
        std::vector<float> valProbs;
        valProbs.reserve(numVal);
        for (int i = 0; i < numVal; ++i) {
            float z = _weights[numWeights - 1];
            for (int j = 0; j < valFeatures.cols; ++j) {
                z += _weights[j] * valFeatures.at<float>(i, j);
            }
            valProbs.push_back(sigmoidTemperature(z, 2.0f));
        }
        calculateFullMetrics(valProbs, valLabels, _metrics);
    }

    return true;
}

```


Código 12 : Entrenamiento MaxEnt (autor)

```

cv::Mat MaxEntPredictor::expandFeatures(const cv::Mat& features) const {
    if (!_useQuadratic) return features.clone();

    const int n = features.rows;
    const int m = features.cols;
    const int expandedCols = m + m;

    cv::Mat expanded(n, expandedCols, CV_32F);

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            expanded.at<float>(i, j) = features.at<float>(i, j);
        }
        for (int j = 0; j < m; ++j) {
            float val = features.at<float>(i, j);
            expanded.at<float>(i, m + j) = val * val;
        }
    }

    return expanded;
}

```

Código 13 : Expansión de características cuadráticas (autor)

```

std::vector<float> MaxEntPredictor::predict(const cv::Mat& features) const {
    std::vector<float> probabilities;

    if (!_trained || features.empty()) {
        return probabilities;
    }

    cv::Mat expanded = expandFeatures(features);
    cv::Mat normalized = normalizeFeatures(expanded);

    const int numWeights = static_cast<int>(_weights.size());
    probabilities.resize(normalized.rows);

    for (int i = 0; i < normalized.rows; ++i) {
        float z = _weights[numWeights - 1];
        for (int j = 0; j < normalized.cols; ++j) {
            z += _weights[j] * normalized.at<float>(i, j);
        }
        z = std::clamp(z, -10.0f, 10.0f);
        probabilities[i] = sigmoidTemperature(z, 2.0f);
    }

    return probabilities;
}

```

Código 14 : Predicción MaxEnt (autor)

```

std::vector<VariableImportance> MaxEntPredictor::getVariableImportance() const {
    std::vector<VariableImportance> importance;

    if (!_trained) return importance;

    // Valor absoluto de los pesos lineales como indicador de importancia
    std::vector<float> linearImportance(_numOriginalFeatures, 0.0f);

    for (int i = 0; i < _numOriginalFeatures && i < static_cast<int>(_weights.size()); ++i) {
        linearImportance[i] = std::abs(_weights[i]);

        if (_useQuadratic) {
            int quadIdx = _numOriginalFeatures + i;
            if (quadIdx < static_cast<int>(_weights.size()) - 1) {
                linearImportance[i] += std::abs(_weights[quadIdx]);
            }
        }
    }

    float maxVal = *std::max_element(linearImportance.begin(), linearImportance.end());

    for (int i = 0; i < _numOriginalFeatures; ++i) {
        VariableImportance vi;

        if (i < static_cast<int>(_featureNames.size())) {
            vi.name = _featureNames[i];
            bool foundDesc = false;
            for (int j = 0; j < NUM_BIOCLIM_VARS; ++j) {
                if (BIOCLIM_NAMES[j] == vi.name) {
                    vi.description = getBioclimDescription(j);
                    foundDesc = true;
                    break;
                }
            }
            if (!foundDesc) {
                if (vi.name == "ELEV") vi.description = "Elevacion del terreno (m)";
                else if (vi.name == "SLOPE") vi.description = "Pendiente del terreno
(grados)";
                else if (vi.name == "ASPECT") vi.description = "Orientacion del terreno
(grados)";
            }
        }
        else if (i < NUM_BIOCLIM_VARS) {
            vi.name = BIOCLIM_NAMES[i];
            vi.description = getBioclimDescription(i);
        }
        else {
            vi.name = "VAR" + std::to_string(i + 1);
            vi.description = "Variable " + std::to_string(i + 1);
        }

        vi.importance = (maxVal > 0.0f) ? linearImportance[i] / maxVal : 0.0f;
        importance.push_back(vi);
    }

    std::sort(importance.begin(), importance.end(),
        [](const VariableImportance& a, const VariableImportance& b) {
            return a.importance > b.importance;
        });

    return importance;
}

```

Código 15 : Obtención de importancia de variables MaxEnt (autor)

```

KFoldResults CrossValidator::runCrossValidation(
    const cv::Mat& features,
    const cv::Mat& labels,
    const IPredictionModel* predictorTemplate,
    int k) {

    KFoldResults results;
    results.k = k;
    results.valid = false;

    if (features.empty() || labels.empty() || features.rows != labels.rows || k < 2) {
        return results;
    }

    if (predictorTemplate == nullptr) {
        return results;
    }

    const int nSamples = features.rows;

    auto folds = createStratifiedFolds(labels, k);

    for (int i = 0; i < k; ++i) {
        if (folds[i].empty()) {
            return results;
        }
    }

    results.foldMetrics.resize(k);

    for (int fold = 0; fold < k; ++fold) {
        const auto& testIndices = folds[fold];

        int nTrain = nSamples - static_cast<int>(testIndices.size());
        cv::Mat trainFeatures(nTrain, features.cols, features.type());
        cv::Mat trainLabels(nTrain, 1, labels.type());
        cv::Mat testFeatures(static_cast<int>(testIndices.size()), features.cols,
features.type());
        cv::Mat testLabels(static_cast<int>(testIndices.size()), 1, labels.type());

        int trainIdx = 0;
        for (int i = 0; i < nSamples; ++i) {
            bool isTest = false;
            for (int idx : testIndices) {
                if (idx == i) { isTest = true; break; }
            }
            if (isTest) continue;
            features.row(i).copyTo(trainFeatures.row(trainIdx));
            trainLabels.at<int>(trainIdx, 0) = labels.at<int>(i, 0);
            ++trainIdx;
        }

        for (size_t i = 0; i < testIndices.size(); ++i) {
            int idx = testIndices[i];
            features.row(idx).copyTo(testFeatures.row(static_cast<int>(i)));
            testLabels.at<int>(static_cast<int>(i), 0) = labels.at<int>(idx, 0);
        }

        auto predictor = clonePredictor(predictorTemplate);
        if (!predictor || !predictor->train(trainFeatures, trainLabels)) {
            continue;
        }

        auto probs = predictor->predict(testFeatures);
        calculateFullMetrics(probs, testLabels, results.foldMetrics[fold]);
    }

    auto [mean, stddev] = ModelMetrics::computeMeanAndStd(results.foldMetrics);
    results.meanMetrics = mean;
    results.stdMetrics = stddev;
    results.valid = true;

    return results;
}

```

Código 16 : Validación cruzada (autor)

```
glm::vec3 HeatmapGenerator::probabilityToColor(float probability, float minProb, float
maxProb) {
    if (maxProb <= minProb) return glm::vec3(0.0f, 0.0f, 1.0f);
    float normalized = (probability - minProb) / (maxProb - minProb);
    normalized = glm::clamp(normalized, 0.0f, 1.0f);
    // Gamma correction para expandir el rango bajo y hacer visible la variacion
    normalized = std::pow(normalized, 0.35f);
    float r = glm::clamp(1.5f - std::abs(4.0f * normalized - 3.0f), 0.0f, 1.0f);
    float g = glm::clamp(1.5f - std::abs(4.0f * normalized - 2.0f), 0.0f, 1.0f);
    float b = glm::clamp(1.5f - std::abs(4.0f * normalized - 1.0f), 0.0f, 1.0f);
    return glm::vec3(r, g, b);
}
```

Código 17 : Paso de probabilidad a color para heatmap (autor)

```

GEU::Texture* HeatmapGenerator::generateHeatmapTexture(const PredictionResult& result,
DEMFeatureExtractor* demExtractor, const TopographicFactorConfig& topoConfig) {
    if (!result.isValid()) {
        return nullptr;
    }

    int coarseW = 1;
    if (result.coords.size() > 1) {
        double firstLat = result.coords[0].second;
        for (size_t i = 1; i < result.coords.size(); ++i) {
            if (std::abs(result.coords[i].second - firstLat) > 1e-8) {
                coarseW = static_cast<int>(i);
                break;
            }
        }
    }

    int coarseH = static_cast<int>(result.coords.size()) / coarseW;
    if (coarseW <= 0 || coarseH <= 0) {
        return nullptr;
    }

    // Rango fijo [0, 1] para que los colores sean comparables entre escenarios
    constexpr float minProb = 0.0f;
    constexpr float maxProb = 1.0f;

    bool useDEM = (demExtractor && demExtractor->isLoaded());

    cv::Mat coarseProb(coarseH, coarseW, CV_32F);
    for (int r = 0; r < coarseH; ++r) {
        for (int c = 0; c < coarseW; ++c) {
            int idx = r * coarseW + c;
            float prob = result.proBABILITIES[idx];
            if (topoConfig.enabled && useDEM) {
                double lon = result.coords[idx].first;
                double lat = result.coords[idx].second;
                auto dem = demExtractor->getFeaturesOrDefault(lon, lat);
                if (std::isfinite(dem.elevation)) {
                    float factor = computeTopographicFactor(dem, topoConfig);
                    prob = glm::clamp(prob * factor, 0.0f, 1.0f);
                }
            }
            coarseProb.at<float>(r, c) = prob;
        }
    }

    int targetW, targetH;
    if (useDEM) {
        targetW = coarseW * 10;
        targetH = coarseH * 10;
        const int MAX_HIRES = 4096;
        if (targetW > MAX_HIRES || targetH > MAX_HIRES) {
            float scale = std::min(static_cast<float>(MAX_HIRES) / targetW,
static_cast<float>(MAX_HIRES) / targetH);
            targetW = static_cast<int>(targetW * scale);
            targetH = static_cast<int>(targetH * scale);
        }
    }
    else {
        targetW = coarseW * 4;
        targetH = coarseH * 4;
        const int MAX_TEX_SIZE = 2048;
        if (targetW > MAX_TEX_SIZE || targetH > MAX_TEX_SIZE) {
            float scale = std::min(static_cast<float>(MAX_TEX_SIZE) / targetW,
static_cast<float>(MAX_TEX_SIZE) / targetH);
            targetW = static_cast<int>(targetW * scale);
            targetH = static_cast<int>(targetH * scale);
        }
    }

    // Interpolación bicúbica para suavizar los bordes del grid grueso
    cv::Mat fineProb;
    cv::resize(coarseProb, fineProb, cv::Size(targetW, targetH), 0, 0, cv::INTER_CUBIC);

    std::vector<uint8_t> pixels(static_cast<size_t>(targetW) * targetH * 4, 0);

    double fineCellSizeX = result.extent.width() / targetW;
    double fineCellSizeY = result.extent.height() / targetH;

    for (int row = 0; row < targetH; ++row) {
        for (int col = 0; col < targetW; ++col) {
            float prob = fineProb.at<float>(row, col);
            prob = glm::clamp(prob, 0.0f, 1.0f);

            if (topoConfig.enabled && useDEM) {
                double lon = result.extent.minLon + (col + 0.5) * fineCellSizeX;
                double lat = result.extent.maxLat - (row + 0.5) * fineCellSizeY;
                auto dem = demExtractor->getFeaturesOrDefault(lon, lat);
                if (std::isfinite(dem.elevation)) {
                    float factor = computeTopographicFactor(dem, topoConfig);
                    prob = glm::clamp(prob * factor, 0.0f, 1.0f);
                }
            }

            glm::vec3 color = probabilityToColor(prob, minProb, maxProb);
            size_t idx = static_cast<size_t>(row * targetW + col) * 4;
            pixels[idx + 0] = static_cast<uint8_t>(color.r * 255.0f);
            pixels[idx + 1] = static_cast<uint8_t>(color.g * 255.0f);
            pixels[idx + 2] = static_cast<uint8_t>(color.b * 255.0f);
            pixels[idx + 3] = 255;
        }
    }

    auto* img = new GEU::Image(pixels.data(), static_cast<uint16_t>(targetW),
static_cast<uint16_t>(targetH), 4);
    img->flipImageVertically(); // Para cuadra

    auto* tex = new GEU::Texture(img, GL_CLAMP_TO_EDGE, GL_CLAMP_TO_EDGE,
GL_LINEAR, GL_LINEAR, false, false);

    delete img;

    return tex;
}

```

Código 18 : Generar heatmap (autor)

```

GEU::PointCloud* HeatmapGenerator::generateHeatmapPointCloud(const PredictionResult& result,
const std::string& name, DEMFeatureExtractor* demExtractor, const PointCloudConfig& config,
const TopographicFactorConfig& topoConfig) {
    if (!result.isValid() || result.probabilities.empty()) {
        return nullptr;
    }

    bool useDEM = (demExtractor && demExtractor->isLoaded());

    int coarseW = 1;
    if (result.coords.size() > 1) {
        double firstLat = result.coords[0].second;
        for (size_t i = 1; i < result.coords.size(); ++i) {
            if (std::abs(result.coords[i].second - firstLat) > 1e-8) {
                coarseW = static_cast<int>(i);
                break;
            }
        }
    }

    int coarseH = static_cast<int>(result.coords.size()) / coarseW;
    if (coarseW <= 0 || coarseH <= 0 || coarseW * coarseH > static_cast<int>(
result.probabilities.size())) {
        coarseW = static_cast<int>(result.probabilities.size());
        coarseH = 1;
    }

    cv::Mat coarseProb(coarseH, coarseW, CV_32F);
    for (int r = 0; r < coarseH; ++r) {
        for (int c = 0; c < coarseW; ++c) {
            int idx = r * coarseW + c;
            float prob = result.probabilities[idx];
            if (topoConfig.enabled && useDEM) {
                double lon = result.coords[idx].first;
                double lat = result.coords[idx].second;
                auto dem = demExtractor->getFeaturesOrDefault(lon, lat);
                if (std::isfinite(dem.elevation)) {
                    float factor = computeTopographicFactor(dem, topoConfig);
                    prob = glm::clamp(prob * factor, 0.0f, 1.0f);
                }
            }
            coarseProb.at<float>(r, c) = prob;
        }
    }

    int fineW = coarseW * config.subdiv;
    int fineH = coarseH * config.subdiv;
    int cvInterp = cv::INTER_CUBIC;
    if (config.interpolation == 0) cvInterp = cv::INTER_NEAREST;
    else if (config.interpolation == 1) cvInterp = cv::INTER_LINEAR;
    cv::Mat fineProb;
    cv::resize(coarseProb, fineProb, cv::Size(fineW, fineH), 0, 0, cvInterp);

    // Sistema de coordenadas local centrado y escalado
    double minLon = result.extent.minLon;
    double maxLon = result.extent.maxLon;
    double minLat = result.extent.minLat;
    double maxLat = result.extent.maxLat;
    double centerLon = (minLon + maxLon) * 0.5;
    double centerLat = (minLat + maxLat) * 0.5;
    double latRad = centerLat * 3.14159265358979323846 / 180.0;
    double metersPerDegLon = 111320.0 * std::cos(latRad);
    double metersPerDegLat = 110540.0;
    double worldWidth = (maxLon - minLon) * metersPerDegLon;
    double worldHeight = (maxLat - minLat) * metersPerDegLat;
    double maxWorldDim = std::max(worldWidth, worldHeight);
    float worldScale = (maxWorldDim > 0.0) ? static_cast<float>(config.targetSize /
maxWorldDim) : 1.0f;

    double fineCellSizeX = result.extent.width() / fineW;
    double fineCellSizeY = result.extent.height() / fineH;

    // Rango fijo [0, 1] para colores (coherente con textura 2D)
    constexpr float minProb = 0.0f;
    constexpr float maxProb = 1.0f;
    for (int r = 0; r < fineH; ++r) {
        for (int c = 0; c < fineW; ++c) {
            float p = fineProb.at<float>(r, c);
            p = glm::clamp(p, 0.0f, 1.0f);
            fineProb.at<float>(r, c) = p;
        }
    }

    // Generar nube de puntos 3D de alta resolución
    std::vector<GEU::VAO::Point> points;
    points.reserve(static_cast<size_t>(fineW * fineH));

    glm::ivec3 minPos(std::numeric_limits<float>::max(), std::numeric_limits<float>::max(),
std::numeric_limits<float>::max());
    glm::ivec3 maxPos(std::numeric_limits<float>::lowest(),
std::numeric_limits<float>::lowest(), std::numeric_limits<float>::lowest());

    for (int row = 0; row < fineH; ++row) {
        for (int col = 0; col < fineW; ++col) {
            double lon = result.extent.minLon + (col + 0.5) * fineCellSizeX;
            double lat = result.extent.maxLat - (row + 0.5) * fineCellSizeY;
            float prob = fineProb.at<float>(row, col);

            float z = 0.0f;
            bool hasDem = true;
            if (useDEM) {
                auto dem = demExtractor->getFeatures(lon, lat);
                if (!std::isfinite(dem.elevation)) {
                    if (config.skipNoData) continue; // Evita fantasmas en Z=0
                    hasDem = false;
                }
            }
            else {
                z = dem.elevation * worldScale + config.verticalExaggeration;
            }

            float x = static_cast<float>((lon - centerLon) * metersPerDegLon * worldScale);
            float y = static_cast<float>((lat - centerLat) * metersPerDegLat * worldScale);

            GEU::VAO::Point p;
            p._point = glm::vec3(x, y, z);

            glm::vec3 color = probabilityToColor(prob, minProb, maxProb);
            p._color = glm::vec3(color * 255.0f);

            points.push_back(p);

            minPos = glm::min(minPos, p._point);
            maxPos = glm::max(maxPos, p._point);
        }
    }

    if (points.empty()) {
        return nullptr;
    }

    AABB aabb;
    aabb.update(minPos);
    aabb.update(maxPos);

    auto* cloud = new GEU::PointCloud(name);
    cloud->addNewPoints(points);
    cloud->updateAABB(aabb);
    cloud->spatialOrdering(GEU::PointCloud::SortingMethod::Hilbert);
    return cloud;
}

```


Código 19 : Generar nube de puntos 3D con colores del heatmap (autor)

```

GEU::Texture* HeatmapGenerator::generateDifferenceTexture(const PredictionResult& current,
const PredictionResult& future) {
    if (!current.isValid() || !future.isValid()) {
        return nullptr;
    }

    int coarseW = 1;
    if (current.coords.size() > 1) {
        double firstLat = current.coords[0].second;
        for (size_t i = 1; i < current.coords.size(); ++i) {
            if (std::abs(current.coords[i].second - firstLat) > 1e-8) {
                coarseW = static_cast<int>(i);
                break;
            }
        }
    }
    int coarseH = static_cast<int>(current.coords.size()) / coarseW;
    if (coarseW <= 0 || coarseH <= 0) {
        return nullptr;
    }

    cv::Mat currentProb(coarseH, coarseW, CV_32F);
    cv::Mat futureProb(coarseH, coarseW, CV_32F);
    for (int r = 0; r < coarseH; ++r) {
        for (int c = 0; c < coarseW; ++c) {
            int idx = r * coarseW + c;
            currentProb.at<float>(r, c) = current.proBABILITIES[idx];
            futureProb.at<float>(r, c) = future.proBABILITIES[idx];
        }
    }

    int targetW = coarseW * 5;
    int targetH = coarseH * 5;
    cv::Mat currentFine, futureFine;
    cv::resize(currentProb, currentFine, cv::Size(targetW, targetH), 0, 0, cv::INTER_CUBIC);
    cv::resize(futureProb, futureFine, cv::Size(targetW, targetH), 0, 0, cv::INTER_CUBIC);

    std::vector<uint8_t> pixels(static_cast<size_t>(targetW) * targetH * 4, 0);
    for (int row = 0; row < targetH; ++row) {
        for (int col = 0; col < targetW; ++col) {
            float cur = glm::clamp(currentFine.at<float>(row, col), 0.0f, 1.0f);
            float fut = glm::clamp(futureFine.at<float>(row, col), 0.0f, 1.0f);

            float diff = (cur > 0.0f) ? ((fut - cur) / (cur + 0.05f)) : (fut / 0.05f);
            float visDiff = glm::clamp(diff, -1.0f, 1.0f);

            float r = 0.5f, g = 0.5f, b = 0.5f;
            if (visDiff > 0.0f) {
                g = 0.5f + 0.5f * visDiff;
                r = 0.5f - 0.3f * visDiff;
                b = 0.5f - 0.3f * visDiff;
            }
            else if (visDiff < 0.0f) {
                r = 0.5f + 0.5f * (-visDiff);
                g = 0.5f - 0.3f * (-visDiff);
                b = 0.5f - 0.3f * (-visDiff);
            }

            size_t idx = static_cast<size_t>(row * targetW + col) * 4;
            pixels[idx + 0] = static_cast<uint8_t>(r * 255.0f);
            pixels[idx + 1] = static_cast<uint8_t>(g * 255.0f);
            pixels[idx + 2] = static_cast<uint8_t>(b * 255.0f);
            pixels[idx + 3] = 255;
        }
    }

    auto* img = new GEU::Image(pixels.data(), static_cast<uint16_t>(targetW),
static_cast<uint16_t>(targetH), 4);
    img->flipImageVertically();

    auto* tex = new GEU::Texture(img, GL_CLAMP_TO_EDGE, GL_CLAMP_TO_EDGE,
GL_LINEAR, GL_LINEAR, false, false);
    delete img;

    return tex;
}

```

Código 20 : Generación textura de diferencia entre escenarios (autor)

```

void GEU::PrediccionEspeciesGUIAdapter::renderGuiObject(
    ApplicationState* appState, Scene* sceneContent)
{
    auto* sim = SimulationManager::getInstance();
    _currentScene = sceneContent;

    checkAsyncCompletion();

    renderStatusBar();
    ImGui::Separator();

    auto nowMs = std::chrono::duration_cast<std::chrono::milliseconds>({
        std::chrono::steady_clock::now().time_since_epoch().count()});
    bool showProgress = _isBusy.load() || nowMs < _progressHideTimeMs.load();
    if (showProgress) {
        float prog = _progress.load();
        std::string msg;
        {
            std::lock_guard<std::mutex> lock(_statusMutex);
            msg = _statusMsg;
        }
        GuiUtilities::ProgressBar(msg, prog, ImGui::GetContentRegionAvail().x, 0, 0);
        ImGui::Spacing();
    }

    if (ImGui::BeginTabBar("##predspp_tabs")) {

        if (ImGui::BeginTabItem((LocaleStrings::getInstance()->getString(PREDSPP_TAB_DATA) +
"##predspp_datatab").c_str())) {
            renderDataLoadingSection();
            ImGui::EndTabItem();
        }

        if (ImGui::BeginTabItem((LocaleStrings::getInstance()->getString(PREDSPP_TAB_MODEL) +
"##predspp_modeltab").c_str())) {
            renderConfigurationSection();
            ImGui::Separator();
            renderVariableSelectionSection();
            ImGui::Separator();
            renderTrainingSection();
            ImGui::EndTabItem();
        }

        ImGui::BeginDisabled(!sim->hasTrainedModel());
        if (ImGui::BeginTabItem((LocaleStrings::getInstance()->getString(PREDSPP_TAB_SCENARIOS) +
"##predspp_scenariotab").c_str())) {
            renderScenarioSection();
            ImGui::EndTabItem();
        }
        ImGui::EndDisabled();

        ImGui::BeginDisabled(!sim->hasTrainedModel());
        if (ImGui::BeginTabItem((LocaleStrings::getInstance()->getString(PREDSPP_TAB_RESULTS) +
"##predspp_resultstab").c_str())) {
            renderResultsSection();
            ImGui::Separator();
            renderExportSection();
            ImGui::EndTabItem();
        }
        ImGui::EndDisabled();

        ImGui::BeginDisabled(!sim->hasTrainedModel());
        if (ImGui::BeginTabItem((LocaleStrings::getInstance()-
>getString(PREDSPP_TAB_RESPONSE_CURVES) + "##predspp_curvestab").c_str())) {
            renderResponseCurvesSection();
            ImGui::EndTabItem();
        }
        ImGui::EndDisabled();

        if (ImGui::BeginTabItem((std::string("Comparar") + "##predspp_comparetab").c_str())) {
            renderCompareSection();
            ImGui::EndTabItem();
        }

        ImGui::EndTabBar();
    }
}

```

Código 21 : Renderización de la interfaz del módulo con ImGui (autor)

```

void GEU::PrediccionEspeciesGUIAdapter::runAsync(std::function<void()> task, const std::string&
status) {
    if (_isBusy.load()) return;
    auto* sim = SimulationManager::getInstance();
    sim->setProgressCallback([this](float p) {
        _progress.store(p);
    });
    _isBusy = true;
    _progress = 0.05f;
    {
        std::lock_guard<std::mutex> lock(_statusMutex);
        _statusMsg = status;
    }
    std::thread worker([this, task]() {
        std::atomic<bool> taskDone{false};
        std::thread taskThread([&]() {
            task();
            taskDone = true;
        });
        taskThread.detach();

        while (!taskDone.load()) {
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
            float p = _progress.load();
            if (p < 0.9f) {
                _progress = p + 0.01f;
            }
        }

        _progress = 1.0f;
        auto hideTime = std::chrono::duration_cast<std::chrono::milliseconds>(
            std::chrono::steady_clock::now().time_since_epoch()).count() + 1000;
        _progressHideTimeMs.store(hideTime);
        _isBusy = false;
        auto* sim2 = SimulationManager::getInstance();
        sim2->setProgressCallback(nullptr);
    });
    worker.detach();
}

```

Código 22 : Realizar tarea asíncrona para entrenamiento o predicción (autor)

```

void GEU::PrediccionEspeciesGUIAdapter::checkAsyncCompletion() {
    if (_pendingTask == PendingTask::None) return;

    auto* sim = SimulationManager::getInstance();
    switch (_pendingTask) {
        case PendingTask::Training:
            // El entrenamiento ya actualiza el estado interno de SimulationManager
            break;
        case PendingTask::CurrentPrediction:
            updateHeatmapVisualizations();
            cacheScenario("Actual", sim->getLastResult());
            switchToScenario("Actual");
            break;
        case PendingTask::FuturePrediction: {
            updateHeatmapVisualizations();
            std::string sspShort;
            switch (sim->getClimateHandler().getLoadedScenario()) {
                case SSPScenario::SSP1_26: sspShort = "SSP12.6"; break;
                case SSPScenario::SSP2_45: sspShort = "SSP24.5"; break;
                case SSPScenario::SSP3_70: sspShort = "SSP37.0"; break;
                case SSPScenario::SSP5_85: sspShort = "SSP58.5"; break;
            }
            std::string periodShort;
            switch (sim->getClimateHandler().getLoadedPeriod()) {
                case FutureTimePeriod::Period_2021_2040: periodShort = "20212040"; break;
                case FutureTimePeriod::Period_2041_2060: periodShort = "20412060"; break;
                case FutureTimePeriod::Period_2061_2080: periodShort = "20612080"; break;
                case FutureTimePeriod::Period_2081_2100: periodShort = "20812100"; break;
            }
            std::string niceName = "SDM_" + sspShort;
            if (!periodShort.empty()) niceName += "_" + periodShort;
            cacheScenario(niceName, sim->getFutureResult());
            switchToScenario(niceName);
            break;
        }
        default:
            break;
    }
    _pendingTask = PendingTask::None;
}

```

Código 23 : Comprobar finalización tarea asíncrona (autor)

Anexo II – Manual de usuario GEU (módulo de predicción de especies)